

Fachbereich Maschinenbau – Automatisierungstechnik

P R O J E K T D O K U M E N T A T I O N

Smarte Mülltonne

Interdisziplinäres Projekt

Semester: Wintersemester 2025/26

Studierende:

Philipp Zajackowski	Matr.-Nr. 10039196
Christoph Harges	Matr.-Nr. 30259993
Daniel Tüben	Matr.-Nr. 30549236

**Betreuende
Professoren:**

Prof. Dr. Frank Hellweg
Prof. Dr. Jens Bechthold
Prof. Markus Strick

Tabellenverzeichnis

Tabelle 1: Anforderungsliste.....	12
Tabelle 2: Morphologischer Kasten	13
Tabelle 3: Prinziplösung	14
Tabelle 4: Gewichtungsmatrix.....	14
Tabelle 5: Erfüllungsgrade	15
Tabelle 6: Erfüllungsgrade für Rot.....	16
Tabelle 7: Erfüllungsgrade für Blau	17
Tabelle 8: Übersicht der identifizierten Use Cases.....	26
Tabelle 9: Pinbelegungstabelle des Raspberry Pi Pico 2 W.....	52
Tabelle 10: Visuelles Anzeigesystem der Betriebszustände	57
Tabelle 11: Visuelles Anzeigesystem der Serverzustände.....	58
Tabelle 12: Akustisches Anzeigesystem	58
Tabelle 13: Komponentenübersicht nach Subsystemschicht	62
Tabelle 14: Modulübersicht der Client-Software auf dem Raspberry Pi Pico 2 W.....	64
Tabelle 15: Parameter des PD-Reglers und der Motoransteuerung.....	73
Tabelle 16: Übersicht der Nachrichten, die der Client sendet.....	81
Tabelle 17: Übersicht der Nachrichten, die der Server an den Client sendet.....	81
Tabelle 18: Use Case Erfüllungsmatrix.....	86
Tabelle 19: Bewertungskriterien.....	86
Tabelle 20: Reifegrad-Übersicht.....	87
Tabelle 21: Technische Trade-offs.....	88

Abbildungsverzeichnis

Abbildung 1: Sprint-Backlog in Jira mit Aufgaben, Kategorien und Statusinformationen	9
Abbildung 2: Zeitliche Planung der Sprints und Aufgabenpakete im Projekt "Smarte Mülltonne"	9
Abbildung 3: Struktur der Projektdokumentation mit zentralen Seiten und Arbeitsbereichen	10
Abbildung 4: Frontalansicht des Korpus vor dem Schnitt.....	30
Abbildung 5: Unterer Korpus nach dem Schnitt.....	31
Abbildung 6: CAD-Modell Korpus mit Deckel.....	31
Abbildung 7: Zusammengefügtter Korpus.....	32
Abbildung 8: Deckel des Korpus.....	33
Abbildung 9: Fahrwerkanbindung am Korpus	34
Abbildung 10: Ansicht 1 des Liniensensorhalters	34

Abbildung 11: Ansicht 2 des Liniensensorhalters	35
Abbildung 12: Anbindung der Liniensensoren am Korpus	35
Abbildung 13: Halter Ultraschallsensor.....	35
Abbildung 14: Deckel des Halters	36
Abbildung 15: Anzeige/Bedienplatte.....	36
Abbildung 16: Montierte Anzeige/Bedienplatte	36
Abbildung 17: Vorläufiger Schaltplan (Fritzing-Entwurf der Elektronikarchitektur)	49
Abbildung 18: Finaler Schaltplan des Bordnetzes mit L298N, Pico, Sensorik und UI	50
Abbildung 19: Blockschaltbild der Client-Server-Architektur und Datenflüsse	65
Abbildung 20: State-Machine-Diagramm für Client und Server	66
Abbildung 21: State-Machine des Clients.....	68
Abbildung 22: Gewichtung der Liniensensoren im Quellcode	70
Abbildung 23: Veranschaulichung der Funktionsweise des P-Reglers.....	70
Abbildung 24: Berechnung der Stellgröße beim PD-Regler	71
Abbildung 25: Visualisierung des Unterschieds des P- und D-Anteils beim PD-Regler.....	71
Abbildung 26: Konfiguration der Trim-Werte im Quellcode.....	72
Abbildung 27: Positionierung der Liniensensoren auf der Unterseite des Fahrwerks.....	75
Abbildung 28: State-Machine des Servers.....	79
Abbildung 29: Methode des Servers zum Erkennen eines bereits verbundenen Clients.....	82
Abbildung 30: Erweiterung MicroPico in Visual Studio Code	91
Abbildung 31: Markierung der Start- und Endposition.....	94

Inhaltsverzeichnis

<i>Tabellenverzeichnis.....</i>	<i>2</i>
<i>Abbildungsverzeichnis</i>	<i>2</i>
Inhaltsverzeichnis	4
1 Einleitung	7
1.1 <i>Motivation und Problemstellung.....</i>	<i>7</i>
1.2 <i>Zielsetzung und Abgrenzung</i>	<i>8</i>
1.3 <i>Abgrenzung.....</i>	<i>8</i>
1.4 <i>Vorgehensmodell und Projektorganisation.....</i>	<i>9</i>
1.5 <i>Anforderungsliste und Technische Wertigkeit.....</i>	<i>10</i>
2 Anforderungen und Use Cases.....	19
2.1 <i>Use-Case-Übersicht.....</i>	<i>19</i>
2.2 <i>Detaillierte Use-Case-Beschreibungen</i>	<i>19</i>
2.2.1 UC1: Autonome Fahrt zur Bereitstellungsposition.....	20
2.2.2 UC2: Überwachung des Füllstands.....	21
2.2.3 UC3: Status- und Zustandsanzeige.....	22
2.2.4 UC4: Zentrale Server-Verwaltung und Steuerung.....	23
2.3 <i>Priorisierung / Use-Case-Matrix</i>	<i>24</i>
2.4 <i>Übersichtstabelle der Use Cases.....</i>	<i>25</i>
2.5 <i>Einordnung der Prioritäten</i>	<i>27</i>
2.6 <i>Out-of-Scope im Projekt.....</i>	<i>28</i>
3 Mechanik und Konstruktion.....	30
3.1 <i>Konzept und Aufbau</i>	<i>30</i>
3.2 <i>Antriebskonzept.....</i>	<i>36</i>
3.3 <i>Iterationen und Optimierungen</i>	<i>39</i>
3.3.1 Iteration – Anpassung des Innenkorpus	39
3.3.2 Iteration – Motorwechsel zur Lösung des Lastproblems.....	40
3.3.3 Iteration – Strukturelle Versteifung des Fahrwerks.....	41
4 Elektronik, Energie und Verdrahtung	44
4.1 <i>Komponentenübersicht.....</i>	<i>44</i>

4.2	<i>Blockschaltbild</i>	46
4.2.1	Funktionsblöcke.....	46
4.2.2	Energiepfad	47
4.2.3	Signalfluss	48
4.3	<i>Schaltplan und Pinbelegung</i>	48
4.3.1	Vorläufiger Schaltplan (Fritzing-Entwurf)	49
4.3.2	Finaler Schaltplan	50
4.3.3	Verbindliche Pinbelegung des Raspberry Pi Pico 2 W	51
4.4	<i>Motoransteuerung</i>	52
4.5	<i>Energieversorgung</i>	53
4.6	<i>Fachliche Probleme, Analyse und Lösungen</i>	55
5	Sensorik und User Interface	57
5.1	<i>Statusanzeige</i>	57
5.2	<i>Füllstandssensorik</i>	59
5.3	<i>Hinderniserkennung</i>	60
5.4	<i>Linienerkennung</i>	60
5.5	<i>Fachliche Probleme, Analyse und Lösungen</i>	60
6	Architektur der Software	62
6.1	<i>Systemübersicht und Komponenten</i>	62
6.2	<i>Softwareziele</i>	62
6.3	<i>Dateistruktur und Modulübersicht</i>	63
6.4	<i>Architektur Client–Server</i>	64
6.5	<i>Zustandskonzept</i>	66
6.6	<i>Sicherheits- und Betriebsfälle</i>	66
7	Software: Client	68
7.1	<i>Client-Zustandsmaschine</i>	68
7.2	<i>Linienfolgeregelung</i>	70
7.3	<i>Kalibrierung</i>	72
7.4	<i>Software-Qualität und Performance</i>	73
7.5	<i>Fachliche Probleme, Analyse und Lösungen</i>	74
8	Software: Server	78

8.1	<i>Ziele der Server-Software</i>	78
8.2	<i>Dateistruktur und Modulübersicht</i>	78
8.3	<i>Server-Zustandsmaschine</i>	79
8.4	<i>Kommunikationsprotokoll und Nachrichtenformat</i>	80
8.5	<i>Missionsablauf</i>	81
8.6	<i>Verbindungsmanagement und Reconnect-Logik</i>	82
8.7	<i>Software-Qualität und Robustheit</i>	84
9	Evaluation der Arbeitsergebnisse	85
9.1	<i>Bewertung: Zielerreichung pro Use Case</i>	85
9.2	<i>Grenzen des Prototyps</i>	86
10	Diskussion und Lessons Learned	88
10.1	<i>Technische Trade-offs</i>	88
10.2	<i>Stärken und Verbesserungspotenziale des Projekts</i>	89
11	Übergabe: Betrieb und Weiterentwicklung (für Folgegruppen)	91
11.1	<i>Voraussetzungen und Systemkomponenten</i>	91
11.2	<i>Inbetriebnahme des Clients</i>	91
11.3	<i>Inbetriebnahme des Servers</i>	92
11.4	<i>Bedienung im Regelbetrieb</i>	93
11.5	<i>Hinweise zum Demonstrationsbetrieb</i>	93
11.6	<i>Weiterentwicklung</i>	94
	Quellenverzeichnis	98

1 Einleitung

1.1 Motivation und Problemstellung

Die konventionelle Abfallwirtschaft folgt überwiegend starren, intervallbasierten Entsorgungsroutinen, die auf Erfahrungswerten statt auf objektiven Echtzeitdaten beruhen. Dadurch entstehen strukturelle Ineffizienzen: Sammelfahrzeuge werden häufig unabhängig vom tatsächlichen Füllstand der Behälter eingesetzt, was zu unvollständig ausgelasteten Touren, erhöhtem Ressourcenverbrauch und vermeidbaren Emissionen führt. Zudem fehlen verlässliche Informationen über Behälterzustände, Füllstandsentwicklungen oder Störereignisse wie Überfüllungen oder Beschädigungen.

Die unzureichende Datenbasis erschwert sowohl operative als auch strategische Entscheidungen. Ohne kontinuierliche Messdaten lassen sich weder präzise Prognosen noch adaptive Planungsprozesse realisieren. Dies limitiert die dynamische Routenoptimierung, die bedarfsgerechte Kapazitätsverteilung und die vorausschauende Wartungsplanung. Gleichzeitig bleibt die Bewertung der Wirtschaftlichkeit und Umweltwirkung bestehender Prozesse unvollständig.

Intelligente, sensor- und datenbasierte Systeme setzen genau hier an. Durch IoT-Sensorik, drahtlose Kommunikation und automatisierte Datenanalyse entsteht ein kontinuierlicher Informationsfluss über Füllstände, Nutzungsprofile und Anomalien. Auf dieser Grundlage können Sammelprozesse bedarfsorientiert gesteuert, starre Routinen durch dynamische Einsatzpläne ersetzt und Ressourcen effizienter disponiert werden. Zustandsbasierte Wartungszyklen und eine verbesserte strategische Entscheidungsgrundlage erhöhen zusätzlich die Prozessqualität.

Insgesamt ermöglichen intelligente Abfallwirtschaftssysteme eine deutliche Steigerung der Effizienz, eine Reduktion von Kosten und Emissionen sowie eine höhere Transparenz entlang des gesamten Entsorgungsprozesses. Sie bilden damit eine wesentliche Grundlage für die digitale und nachhaltige Transformation kommunaler und industrieller Entsorgungsstrukturen.

1.2 Zielsetzung und Abgrenzung

Das Projekt verfolgt das Ziel, eine **autonom agierende Mülltonne** zu entwickeln, die definierte Fahr- und Messaufgaben selbstständig ausführt und über eine **Serverinstanz** gesteuert sowie überwacht werden kann. Die wesentlichen Ziele lassen sich in folgende überprüfbare Zielaussagen gliedern:

Autonome Navigation:

- Die Mülltonne soll in der Lage sein, **eine Linie zuverlässig zu verfolgen**.
- Hindernisse werden mittels Sensorik erkannt, und die Mülltonne **bleibt rechtzeitig davorstehen**.

Serverbasierte Interaktion

- Die Mülltonne empfängt **Serverkommandos** (z. B. Start, Stopp, Statusabfrage) über eine Netzwerkverbindung.
- Betriebszustände, Sensordaten und Positionsinformationen werden **regelmäßig an den Server übertragen**.

Mess- und Erfassungsaufgaben

- Relevante Umgebungs- oder Systemdaten (z. B. Abstandswerte, Füllstand, Position) werden **automatisch erfasst und gespeichert**.
- Die erfassten Daten stehen sowohl lokal als auch serverseitig zur **Auswertung** bereit.

1.3 Abgrenzung

Um den Projektumfang klar zu definieren, werden folgende Aspekte bewusst **nicht umgesetzt** oder nur **konzeptionell betrachtet**:

Nicht Bestandteil der Umsetzung:

- **Komplexe globale Navigation** (z. B. SLAM, GPS-basierte Außenlokalisierung) – es wird ausschließlich in einer **vordefinierten, kontrollierten Umgebung** navigiert.
- **Autonomes Entleeren oder Heben** der Mülltonne – mechanische Aktoren zur Müllverarbeitung sind **nicht vorgesehen**.
- **Sicherheitskritische Funktionen** wie Not-Aus-Systeme nach Industriestandard werden **nicht vollständig implementiert**.
- **Energieoptimierung oder Batteriemanagement**, weder Optimierungsmechanismen noch Statusabfragen zum Energiezustand sind Teil des Projektumfangs.

1.4 Vorgehensmodell und Projektorganisation

Vorgehensmodell

Das Projekt wurde nach dem agilen Vorgehensmodell Scrum durchgeführt. Die Entwicklung erfolgte iterativ in mehreren Sprints, um technische Ergebnisse frühzeitig zu evaluieren und Anpassungen flexibel vornehmen zu können. Zu Beginn jedes Sprints wurden im Rahmen der Sprint-Planung die umzusetzenden Backlog-Einträge festgelegt. Regelmäßige kurze Abstimmungen dienten der Fortschrittskontrolle. Sprint Reviews und Retrospektiven ermöglichten eine systematische Bewertung der Ergebnisse sowie eine kontinuierliche Prozessverbesserung.

Die zeitliche Struktur der Sprints sowie die Zuordnung der wesentlichen Arbeitspakete sind in Abbildung 1 dargestellt.

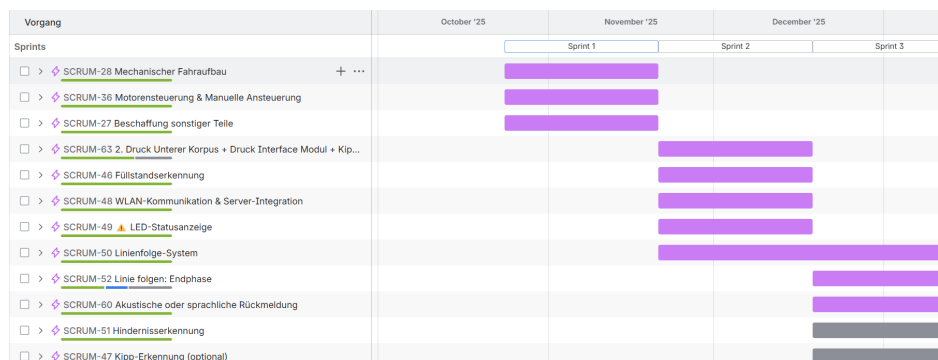


Abbildung 1: Sprint-Backlog in Jira mit Aufgaben, Kategorien und Statusinformationen

Zur Planung und Nachverfolgung der Aufgaben wurde Jira eingesetzt. Abbildung 2 zeigt exemplarisch den Sprint-Backlog des ersten Sprints, in dem die Aufgaben, Kategorien, Verantwortlichkeiten und Bearbeitungsstände dokumentiert wurden.

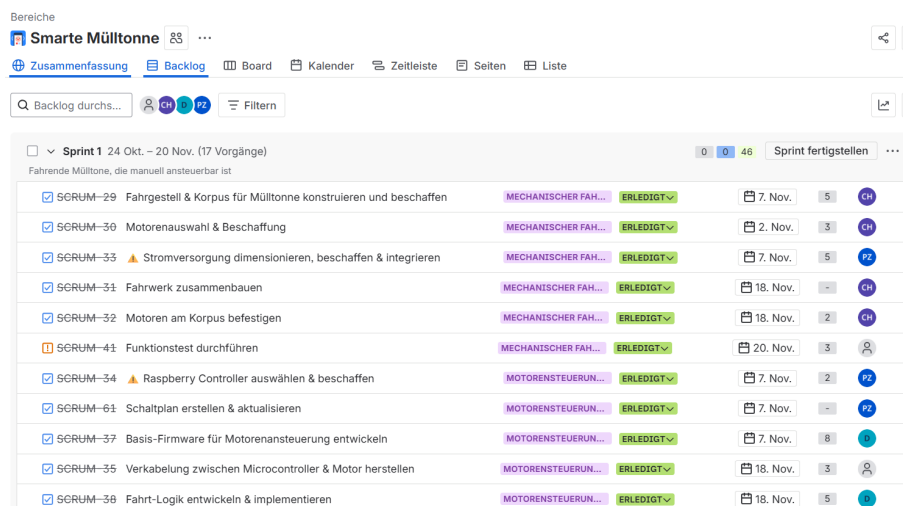


Abbildung 2: Zeitliche Planung der Sprints und Aufgabenpakete im Projekt "Smarte Mülltonne"

Projektorganisation

Die klassischen Scrum-Rollen wurden in angepasster Form umgesetzt:

- **Product Owner:** Priorisierung des Backlogs und Definition der Anforderungen
- **Entwicklungsteam:** Umsetzung der Hardware-, Software- und Dokumentationsaufgaben.

Werkzeuge und Artefakte

Für die Aufgaben- und Sprintverwaltung wurde Jira eingesetzt. Dort wurden Product Backlog, Sprint Backlogs und Arbeitsfortschritte strukturiert gepflegt. Ergänzend entstanden folgend Projektartefakte:

- Protokolle und technische Dokumentationen
- Versionsverwaltung des Quellcodes (z. B. Git)
- Eine definierte Ablagestruktur für Code, Messdaten und Projektdokumente

Die verwendete Ablagestruktur innerhalb der Projektplattform ist in Abbildung 3 dargestellt.

Smarte Mülltonne		Verbindung ändern
		Entwürfe anzeigen
▼ Smarte Mülltonne Home	D	Aktualisiert Okt. 13, 2025
Abstimmungsmeetings	D	Aktualisiert Nov. 08, 2025
Teilleiste	D PZ CH	Aktualisiert Feb. 20, 2026
Aktuelle To-Dos	D CH	Aktualisiert Nov. 12, 2025
Termine Übersicht	D	Aktualisiert Nov. 18, 2025

Abbildung 3: Struktur der Projektdokumentation mit zentralen Seiten und Arbeitsbereichen

Diese organisatorische Struktur gewährleistete Transparenz, Nachvollziehbarkeit und eine systematische Steuerung des Projektfortschritts.

1.5 Anforderungsliste und Technische Wertigkeit

Die Entwicklung der intelligenten, autonomen Mülltonne basiert auf einer strukturierten Anforderungsliste, die alle funktionalen, sicherheitsrelevanten und qualitativen Aspekte des Systems definiert. Zu den zentralen Festanforderungen gehören unter anderem der Projektname „*Intelligente, autonome Mülltonne*“, das Ziel eines selbstständig fahrenden, sensorgestützten Systems mit Kommunikationsschnittstelle sowie das Skalieren der Standardmülltonne.

Weitere wesentliche Anforderungen umfassen das autonome Geradeaus- und Kurvenfahren, die optische Linienerkennung, die Nutzung von DC-Motoren für eine maximale Geschwindigkeit von 0,3 m/s eine Energieversorgung über einen ≥ 12 V Akku mit mindestens 8 Stunden Laufzeit sowie verschiedene Sensoren um den Füllstandsermittlung und Hinderniserkennung. Ergänzend werden Anforderungen an Kommunikation (WLAN/LoRa, WPA2-Verschlüsselung), Anzeigeelemente, Sicherheitsfunktionen, Umweltbeständigkeit (IP54, UV-resistent, -10 °C bis $+40$ °C) sowie Wartungs- und Dokumentationsaspekte gestellt. Diese Anforderungen bilden die Grundlage für die spätere Konzeptbewertung.

Nr.	Unterkategorie	Anforderung	Art der Anforderung
01	Allgemein	Projektname: Intelligente, autonome Mülltonne	Festanforderung
02	Allgemein	Ziel: Selbstständig fahrende, sensorgestützte Mülltonne mit Kommunikation	Festanforderung
03	Einsatzbereich	Nutzung in Smart-City-, Industrie- und Wohnanlagen	Zielforderung
04	Geometrie	Standardgrößen skalieren	Festanforderung
05	Geometrie	Kompakte Bauform für Standard-Müllboxen	Zielforderung
06	Kinematik	Autonomes Geradeaus- und Kurvenfahren	Festanforderung
07	Kinematik	Linienfolgen zur Navigation (optisch)	Festanforderung
08	Kinematik	Start-/Enderkennung über RFID oder Markierung	Wunschanforderung
09	Antrieb	DC-Motoren, geeignet für max. 0,3 m/s	Festanforderung
10	Energieversorgung	Akku ≥ 12 V, Laufzeit ≥ 8 h	Festanforderung

11	Sensorik	Ultraschallsensor zur Füllstandsermittlung	Festanforderung
12	Sensorik	Gyroskop/Kippsensor zur Lageüberwachung	Festanforderung
13	Sensorik	Hinderniserkennung (IR oder Lidar) mit Stopfunktion	Festanforderung
14	Kommunikation	WLAN- oder LoRa-Modul	Festanforderung
16	Anzeige & UI	LED-Anzeige	Festanforderung
17	Anzeige & UI	Optionale App / Web-Dashboard zur Datenabfrage	Wunschanforderung
18	Mess- und Datenfunktion	Übermittlung von Füllstand, Position, Akkustand und Fehlern	Festanforderung
19	Sicherheit	Notstopp bei Hindernis / Kippung	Festanforderung
20	Sicherheit	Mechanische Stabilität bei Umkippen	Festanforderung
21	Umwelt & Qualität	Betriebstemperaturbereich –10 °C bis +40 °C	Wunschanforderung
22	Umwelt & Qualität	Schutzklasse IP54, UV-beständig, witterungsfest	Wunschanforderung
24	Wartung	Modulbauweise für Sensortausch / Erweiterung	Wunschanforderung
25	Test & Abnahme	Funktionstest: Fahrverhalten, Liniensfolgen, Füllstand, Kommunikation	Festanforderung
26	Dokumentation	Schaltpläne, Sensorlayout, Softwarebeschreibung, Anleitung	Festanforderung

Tabelle 1: Anforderungsliste

Zur systematischen Erarbeitung möglicher technischer Lösungen wurde ein morphologischer Kasten erstellt. Dieser umfasst die Teilfunktionen Akku, Energieumwandlung, Antrieb,

Füllstandssensorik, Linienerkennung und Hinderniserkennung. Für jede Teilfunktion wurden mehrere Wirkprinzipien gegenübergestellt, beispielsweise Lithium-Ionen-, LiFePo4- oder AGM-Akkus, DC- oder Schrittmotoren, Raupen- oder Zweiradantrieb, kapazitive, optische oder Ultraschallsensoren sowie verschiedene Verfahren zur Linien- und Hinderniserkennung. Aus dieser Kombination wurden zwei vollständige Lösungsvarianten abgeleitet, die im Folgenden als *rote* und *blaue* Lösung bezeichnet werden.

Teilfunktio- nen	Wirkprinzipien			
1	Akkus	Lithium-Ionen (Li-Ion)	Lithium-Eisen- phosphat (LiFePo4)	Blei-Vlies-(AGM)
2	Energie Umwand- lung	DC-Motor	Schrittmotoren	
3	Antrieb	Raupenfahr- werk	Zwei Räderan- trieb	
4	Füllstand Sensor	Kapazitiver	Optisch	Ultraschall
5	Linienenerkennung	Infrarot	Farbsensoren	Kamera
6	Hinderniserken- nung	Ultraschall	Infrarot	Lidar

Tabelle 2: Morphologischer Kasten

Teilfunktio- nen	Wirkprinzipien			
1	Akkus	Lithium-Ionen (Li-Ion)	Lithium-Eisen- phosphat (LiFePo4)	Blei-Vlies-(AGM)
2	Energie Umwand- lung	DC-Motor	Schrittmotoren	
3	Antrieb	Raupenfahr- werk	Zwei Räderan- trieb	

4	Füllstand Sensor	Kapazitiver	Optisch	Ultraschall
5	Linienerkennung	Infrarot	Farbsensoren	Kamera
6	Hinderniserkennung	Ultraschall	Infrarot	Lidar

Tabelle 3: Prinziplösung

Zur objektiven Bewertung der beiden Varianten wurde eine Gewichtungsmatrix herangezogen. Die Kriterien Sicherheit/Stabilität (34 %), Funktionalität (10 %), Wirtschaftlichkeit (22 %), Verfügbarkeit/Zuverlässigkeit (18 %) und Flexibilität (16 %) wurden entsprechend ihrer Bedeutung gewichtet. Die Erfüllungsgrade wurden anhand definierter Bewertungsstufen (E = 0 - 4) vergeben, wobei E = 4 einer sehr guten Erfüllung entspricht, z. B. „*Sehr hohe Betriebssicherheit, zuverlässige Hinderniserkennung, stabile Regelung*“ oder „*Sehr guter Preis-Leistungs-Verhältnis, hohe Lebensdauer*“. Diese Bewertungsgrundlage wurde sowohl für die rote als auch für die blaue Lösung angewendet.

Bewertungskriterien:

- S=Sicherheit/Stabilität
- F=Funktionalität
- W=Wirtschaftlichkeit
- V=Verfügbarkeit/Zuverlässigkeit
- FL=Flexibilität

Kriterium	S	F	W	V	FL	Σ	GV Vorläufig	GV Endgültig
S	S	S	S	S	S	5	34%	34%
F	S	F	W	F	FL	2	13%	10%
W	S	W	W	V	W	3	20%	22%
V	S	F	V	V	V	3	20%	18%
FL	S	FL	W	V	FL	2	13%	16%
							100%	100%

Tabelle 4: Gewichtungsmatrix

In der Tabelle werden verschiedene Bewertungskriterien (Sicherheit, Funktionalität, Wirtschaftlichkeit, Verfügbarkeit und Flexibilität) jeweils mit einer Punkteskala von 0 bis 4 qualitativ beschrieben, um technische Systeme vergleichbar zu bewerten.

Kriterium	E = 4 (sehr gut)	E = 3 (gut)	E = 2 (mittel)	E = 1 (gering)	E = 0 (ungenügend)
S – Sicherheit / Stabilität	Sehr hohe Betriebssicherheit, zuverlässige Hinderniserkennung, stabile Regelung, keine gefährlichen Zustände	Gute Sicherheit, selten kritische Situationen	Sicherheit ausreichend, aber mit Lücken (z. B. tote Winkel)	Sicherheitsrisiken vorhanden, Fehlfunktionen möglich	Keine Sicherheit, gefährliche Zustände wahrscheinlich
F – Funktionalität	Aufgabe wird vollständig und präzise erfüllt (z. B. sehr genaue Linienerkennung, stabile Motorregelung)	Gute Funktion, kleinere Einschränkungen	Funktion gegeben, aber mit deutlichen Schwächen	Funktion nur eingeschränkt nutzbar	Funktion praktisch nicht erfüllt
W – Wirtschaftlichkeit	Sehr guter Preis-Leistungs-Verhältnis, geringe Kosten, hohe Lebensdauer	Wirtschaftlich sinnvoll, Kosten im Rahmen	Kosten und Nutzen ausgeglichen	Teuer für die Leistung, kurze Lebensdauer	Sehr unwirtschaftlich
V – Verfügbarkeit / Zuverlässigkeit	Sehr zuverlässig, kaum Ausfälle, Komponenten gut verfügbar	Gelegentliche Störungen, aber insgesamt zuverlässig	Spürbare Ausfallwahrscheinlichkeit oder Lieferprobleme	Häufige Störungen oder schwer beschaffbar	Sehr unzuverlässig oder kaum verfügbar
FL – Flexibilität	Sehr flexibel, leicht anpassbar, modular, vielseitig einsetzbar	Gut anpassbar, kleinere Einschränkungen	Nur begrenzt anpassbar	Stark spezialisiert, Änderungen schwierig	Keine Anpassbarkeit

Tabelle 5: Erfüllungsgrade

Kriterium	E = 4 (sehr gut)	E = 3 (gut)	E = 2 (mittel)	E = 1 (gering)	E = 0 (ungenügend)
S – Sicherheit / Stabilität	Sehr hohe Betriebssicherheit, zuverlässige Hinderniserkennung, stabile Regelung, keine gefährlichen Zustände	Gute Sicherheit, selten kritische Situationen	Sicherheit ausreichend, aber mit Lücken (z. B. tote Winkel)	Sicherheitsrisiken vorhanden, Fehlfunktionen möglich	Keine Sicherheit, gefährliche Zustände wahrscheinlich
F – Funktionalität	Aufgabe wird vollständig und präzise erfüllt (z. B. sehr genaue Linienenerkennung, stabile Motorregelung)	Gute Funktion, kleinere Einschränkungen	Funktion gegeben, aber mit deutlichen Schwächen	Funktion nur eingeschränkt nutzbar	Funktion praktisch nicht erfüllt
W – Wirtschaftlichkeit	Sehr guter Preis-Leistungs-Verhältnis, geringe Kosten, hohe Lebensdauer	Wirtschaftlich sinnvoll, Kosten im Rahmen	Kosten und Nutzen ausgeglichen	Teuer für die Leistung, kurze Lebensdauer	Sehr unwirtschaftlich
V – Verfügbarkeit / Zuverlässigkeit	Sehr zuverlässig, kaum Ausfälle, Komponenten gut verfügbar	Gelegentliche Störungen, aber insgesamt zuverlässig	Spürbare Ausfallwahrscheinlichkeit oder Lieferprobleme	Häufige Störungen oder schwer beschaffbar	Sehr unzuverlässig oder kaum verfügbar
FL – Flexibilität	Sehr flexibel, leicht anpassbar, modular, vielseitig einsetzbar	Gut anpassbar, kleinere Einschränkungen	Nur begrenzt anpassbar	Stark spezialisiert, Änderungen schwierig	Keine Anpassbarkeit

Tabelle 6: Erfüllungsgrade für Rot

Kriterium	E = 4 (sehr gut)	E = 3 (gut)	E = 2 (mittel)	E = 1 (gering)	E = 0 (ungenügend)
S – Sicherheit / Stabilität	Sehr hohe Betriebssicherheit, zuverlässige Hinderniserkennung, stabile Regelung, keine	Gute Sicherheit, selten kritische Situationen	Sicherheit ausreichend, aber mit Lücken (z. B. tote Winkel)	Sicherheitsrisiken vorhanden, Fehlfunktionen möglich	Keine Sicherheit, gefährliche Zustände wahrscheinlich

	gefährlichen Zustände				
F – Funktionalität	Aufgabe wird vollständig und präzise erfüllt (z. B. sehr genaue Linienerkennung, stabile Motorregelung)	Gute Funktion, kleinere Einschränkungen	Funktion gegeben, aber mit deutlichen Schwächen	Funktion nur eingeschränkt nutzbar	Funktion praktisch nicht erfüllt
W – Wirtschaftlichkeit	Sehr guter Preis-Leistungs-Verhältnis, geringe Kosten, hohe Lebensdauer	Wirtschaftlich sinnvoll, Kosten im Rahmen	Kosten und Nutzen ausgeglichen	Teuer für die Leistung, kurze Lebensdauer	Sehr unwirtschaftlich
V – Verfügbarkeit / Zuverlässigkeit	Sehr zuverlässig, kaum Ausfälle, Komponenten gut verfügbar	Gelegentliche Störungen, aber insgesamt zuverlässig	Spürbare Ausfallwahrscheinlichkeit oder Lieferprobleme	Häufige Störungen oder schwer beschaffbar	Sehr unzuverlässig oder kaum verfügbar
FL – Flexibilität	Sehr flexibel, leicht anpassbar, modular, vielseitig einsetzbar	Gut anpassbar, kleinere Einschränkungen	Nur begrenzt anpassbar	Stark spezialisiert, Änderungen schwierig	Keine Anpassbarkeit

Tabelle 7: Erfüllungsgrade für Blau

Die rote Lösung erreicht in den Kriterien Sicherheit, Verfügbarkeit und Flexibilität jeweils hohe Erfüllungsgrade (E = 4 bzw. 3) und zeigt insgesamt ein ausgewogenes Verhältnis zwischen Funktionalität, Wirtschaftlichkeit und Zuverlässigkeit. Die blaue Lösung hingegen weist insbesondere in Wirtschaftlichkeit, Verfügbarkeit und Flexibilität geringere Erfüllungsgrade (E = 2) auf und erfüllt damit die Anforderungen weniger umfassend.

$$WT = \frac{P}{P_{max}} = \frac{\sum(G * E)}{E_{max} * \sum G}$$

$$P_{max} = \sum E_{max} * G = 4 * 0,34 + 4 * 0,10 + 4 * 0,22 + 4 * 0,18 + 4 * 0,16$$

$$P_{max} = 4$$

Technische Wertigkeit für Lösung rot und blau:

$$WT_{rot} = \frac{4 * 0,34 + 3 * 0,10 + 3 * 0,22 + 4 * 0,18 + 3 * 0,16}{4}$$

$$WT_{rot} = 0,88$$

$$WT_{\text{blau}} = \frac{3 \cdot 0,34 + 3 \cdot 0,10 + 2 \cdot 0,22 + 2 \cdot 0,18 + 2 \cdot 0,16}{4}$$
$$WT_{\text{blau}} = 0,61$$

Da die maximale technische Wertigkeit $WT_{\text{max}} = 1,0$ beträgt, zeigt die rote Lösung mit 0,88 eine **sehr gute** Gesamtperformance, während die blaue Lösung mit 0,61 lediglich als **brauchbar** einzustufen ist. Die rote Lösung erfüllt die sicherheitsrelevanten, funktionalen und wirtschaftlichen Anforderungen deutlich besser und weist zudem eine höhere Zuverlässigkeit und Flexibilität auf. Aus diesen Gründen wurde die rote Lösung als bevorzugte Systemkonfiguration ausgewählt.

2 Anforderungen und Use Cases

2.1 Use-Case-Übersicht

Dieses Kapitel liefert einen präzisen, aber umfassenden Überblick über alle im Projekt berücksichtigten Use Cases und ihre jeweilige Zielsetzung. Es schafft damit die notwendige Orientierung, um zu verstehen, welche Kernfunktionen das System erfüllen muss, welche Akteure beteiligt sind und welche Abläufe im späteren Verlauf detailliert spezifiziert, implementiert und bewertet werden. Die Ausarbeitung dient als verbindliche Grundlage für Architekturentscheidungen, technische Umsetzung, Testplanung und spätere Erweiterungen.

Überblick über die betrachteten Use Cases

Die im Projekt definierten Use Cases decken die zentralen Funktionsbereiche der autonomen Mülltonne ab. Sie bilden den funktionalen Rahmen, innerhalb dessen alle technischen Komponenten: Sensorik, Aktorik, Softwarelogik und Serverkommunikation die zusammenwirken. Jeder Use Case beschreibt einen abgeschlossenen, für das Gesamtsystem relevanten Vorgang, der aus Sicht des Nutzers oder Systems einen klaren Mehrwert erzeugt.

Die Use Cases lassen sich in vier Hauptkategorien einordnen:

- **Autonome Navigation (UC1):** Bewegung entlang einer Linie, Hindernisreaktion, Positionierung an der Straße.
- **Füllstandsüberwachung (UC2):** Erfassung, Berechnung und Bewertung des Füllstands.
- **Status- und Zustandsanzeige (UC3):** Kommunikation von Betriebszuständen über LED, Buzzer und digitale Meldungen an den Nutzer.
- **Zentrale Serververwaltung (UC4):** Datenverarbeitung, Abholplanung und bidirektionale Kommunikation.

Diese Struktur verdeutlicht, dass das System nicht nur autonom fahren muss, sondern auch seinen eigenen Zustand erfassen, Informationen austauschen und in eine übergeordnete Infrastruktur integriert sein muss.

2.2 Detaillierte Use-Case-Beschreibungen

Die detaillierte Ausarbeitung der Use Cases stellt sicher, dass alle Beteiligten – Entwicklung, Test, Dokumentation und spätere Erweiterungsteams – ein gemeinsames Verständnis davon

haben, **was das System leisten muss, unter welchen Bedingungen, mit welchen Akteuren und welchen erwarteten Ergebnissen**. Dadurch entsteht eine belastbare Grundlage für:

- **Implementierungsentscheidungen** (z. B. Auswahl von Sensoren, Softwarearchitektur, Sicherheitslogik)
- **Testplanung** (z. B. Ableitung von Testfällen aus Normal- und Fehlerabläufen)
- **Evaluationskriterien** (z. B. wann gilt ein Use Case als erfolgreich erfüllt?)
- **Abgrenzung des Projektumfangs** (z. B. Out-of-Scope-Funktionen)
- **Nachvollziehbarkeit für Dritte** (z. B. Betreuer, Gutachter, zukünftige Entwickler)

Damit diese Ziele erreicht werden, werden die Use Cases in einem **einheitlichen Schema** beschrieben. Dieses Schema stellt sicher, dass alle relevanten Aspekte vollständig und vergleichbar dokumentiert sind.

2.2.1 UC1: Autonome Fahrt zur Bereitstellungsposition

Zielsetzung:

Die Mülltonne bewegt sich selbstständig entlang einer definierten Bodenlinie vom Standplatz zur Straße, erkennt Start- und Endpunkte, führt am Ziel eine 180°-Drehung aus und reagiert sicher auf Hindernisse.

Akteure:

- Autonome Mülltonne (Fahrwerk, Motorsteuerung, Sensorik)
- Linien- und Abstandssensoren
- Nutzer (optional über Schalter)

Vorbedingungen:

- Linie ist durchgehend erkennbar (Kontrast, Breite, Material).
- Start- und Endmarkierungen sind eindeutig detektierbar.
- Sensorik ist initialisiert und kalibriert.
- Akku ausreichend geladen.

Normalablauf:

- Start durch Schalter oder Serverbefehl.
- Linienverfolgung mittels IR-/Farbsensoren.
- Durchfahren von Kurven und Abzweigungen gemäß Linienführung.

-
- Erkennung der Endmarkierung an der Straße.
 - Automatische 180°-Drehung zur optimalen Abholposition.
 - Verharren in Bereitstellungsposition.

Alternativ-/Fehlerabläufe:

- **Hindernis erkannt:** Stopp in sicherem Abstand, Wartefunktion aktiv, Fortsetzung nach Freigabe.
- **Linie verloren:** Sofortiger Stopp, Fehlerstatus, ggf. Rückwärtsbewegung zur letzten gültigen Position.
- **Drehung nicht möglich:** Fehlerstatus, digitale Meldung an Server.

Ergebnis:

Die Mülltonne erreicht zuverlässig die Straße, positioniert sich korrekt und verhält sich sicher gegenüber Hindernissen.

Randbedingungen:

- Max. Geschwindigkeit: sicherheitskonform.
- Hinderniserkennung: Ultraschall, Mindestabstand definierbar.
- Schalterstart muss unabhängig vom Server funktionieren (Fail-Safe-Betrieb).

2.2.2 UC2: Überwachung des Füllstands

Zielsetzung

Der Füllstand wird kontinuierlich oder periodisch gemessen, in Prozent umgerechnet und zur Entscheidungsfindung genutzt.

Akteure

- Ultraschallsensor
- Mikrocontroller
- Server (optional)

Vorbedingungen

- Sensor ist korrekt montiert (Deckelmitte, senkrecht nach unten).
- Messintervall ist definiert.
- Kalibrierwerte (Leerstand, Vollstand) sind gespeichert.

Normalablauf

- Sensor misst Abstand zum Müll.
- Software berechnet Füllstand in Prozent.
- Werte werden geglättet (Median-/Moving-Average-Filter).
- Schwellenwerte lösen Statuswechsel aus (z. B. 50 %, 75 %, 100 %).
- Optional: Meldung an Server.

Alternativ-/Fehlerabläufe

- Messwert instabil → Filterung, Wiederholmessung.
- Sensorfehler → Fehlerstatus, digitale Meldung.

Ergebnis

Ein stabiler, interpretierbarer Füllstandswert steht für Anzeige, Logik und Serverkommunikation bereit.

Randbedingungen

- Temperatur- und Feuchtigkeitseinflüsse müssen berücksichtigt werden.
- Deckelbewegungen dürfen Messung nicht verfälschen (Messpause bei Bewegung).

2.2.3 UC3: Status- und Zustandsanzeige

Zielsetzung

Die Mülltonne kommuniziert ihren Zustand lokal (LED/Buzzer) und digital (Server).

Akteure

- LED, Buzzer
- Mikrocontroller
- Server
- Nutzer

Vorbedingungen

- Anzeigeelemente sind funktionsfähig.
- Statuslogik ist definiert.

Normalablauf

- LED zeigt Betriebszustände (z. B. Blau = fährt, Gelb = wartet, Rot = Fehler).
- Buzzer signalisiert kritische Ereignisse (z. B. Hindernis).
- Digitale Statusmeldungen werden an den Server übertragen.
- Lageerkennung meldet Umkippen oder ungewöhnliche Neigung.

Alternativ-/Fehlerabläufe

- LED defekt → digitale Fehlermeldung.
- Kommunikationsfehler → lokale Anzeige bleibt aktiv.

Ergebnis

Alle relevanten Zustände sind für Nutzer und Server eindeutig erkennbar.

Randbedingungen

- LED muss auch bei Tageslicht sichtbar sein.
- Akustische Signale dürfen nicht zu laut sein (Nachbarschaftsschutz).

2.2.4 UC4: Zentrale Server-Verwaltung und Steuerung

Zielsetzung

Der Server sammelt Daten, trifft Entscheidungen, verwaltet Abholpläne und kommuniziert mit der Mülltonne und dem Nutzer.

Akteure

- Server
- Mülltonne
- Nutzer (App/Webinterface)
- Abholfahrzeug/Entsorgungsdienst (optional)

Vorbedingungen

- Netzwerkverbindung besteht.
- Datenformate und Protokolle sind definiert.

Normalablauf

- Mülltonne sendet Statusdaten (Füllstand, Position, Zustand).

-
- Server speichert und analysiert Daten.
 - Server verwaltet digitalen Wochenplan (Abholtermine).
 - Server sendet Befehle oder Konfigurationsupdates an die Mülltonne.
 - Nutzer erhält Benachrichtigungen (z. B. „Füllstand kritisch“).

Alternativ-/Fehlerabläufe

- Kommunikationsverlust → Mülltonne geht in sicheren Zustand.
- Ungültige Befehle → Mülltonne verweigert Ausführung.

Ergebnis

Eine bidirektionale Kommunikation ermöglicht zentrale Überwachung, Planung und Steuerung.

Zukunftsfunktionen

- Abholung automatisch anfordern, wenn Mülltonne voll ist.
- Füllstände innerhalb einer Straße werden geteilt (Clusteroptimierung).
- KI-basierte Routenoptimierung.

2.3 Priorisierung / Use-Case-Matrix

Die Priorisierung basiert auf drei Dimensionen, die gemeinsam die Umsetzungsreihenfolge bestimmen:

- **Nutzen** – Beitrag zur Autonomie, Sicherheit, Effizienz oder Smart-City-Integration.
- **Komplexität** – technischer Aufwand, Integrationsrisiken, Hardwarebedarf.
- **Kerntechnologien** – Sensorik, Aktorik, Softwaremodule, Kommunikationsschnittstellen.

Die Matrix zeigt, welche Funktionen zwingend für den Grundbetrieb erforderlich sind und welche bewusst als Erweiterungen oder Zukunftsthemen eingeordnet wurden.

2.4 Übersichtstabelle der Use Cases

ID	Kategorie	Use Case	Beschreibung	Priorität	Nutzen	Komplexität	Kerntechnologie
1.0	Navigation	Autonome Linienfahrt	Linie folgen von Start bis Ziel	1	Sehr hoch	Mittel	IR-Liniensensoren, Motorsteuerung
1.1	Navigation	Hindernis-Stopp	Stoppt vor Hindernis und wartet	1	Hoch	Gering	Ultraschallsensor vorne
1.2	Navigation	Dynamische Hindernisumfahrung	Umfährt Hindernis und findet Linie wieder	3	Mittel	Mittel	Seitliche Ultraschallsensoren
1.3	Navigation	Rückkehr zur La-destination	Autonome Rückkehr zur Basis	3	Mittel	Hoch	IR-Bake, Navigationslogik
2.0	Füllstand	Stufenmessung	Schwellenwerte über mehrere Sensoren	1	Hoch	Mittel	Mehrere IR-/Ultraschallsensoren
3.0	Status	LED-Anzeige	Zeigt Betriebszustände	1	Mittel	Gering	RGB-LED
3.1	Status	Digitale Statusmeldung	Sendet Status an Server	2	Hoch	Mittel	Pico W, WLAN
3.2	Status	Lageerkennung	Meldet Umkippen	2	Mittel	Gering	Gyroskop

4.0	Sicherheit	Not-Aus	Physischer Stopp aller Funktionen	2	Sehr hoch	Gering	Taster
4.1	Sicherheit	Geofencing	Alarm bei Verlassen eines Bereichs	3	Mittel	Hoch	GPS, Alarmgeber
5.0	Server	Datenverarbeitung	Server speichert und analysiert Daten	1	Hoch	Mittel	Backend, Datenbank
5.1	Server	Digitaler Abholplan	Verwaltung von Abholterminen	3	Mittel	Mittel	Kalenderlogik
5.2	Server	Bidirektionale Kommunikation	Befehle & Updates	1	Hoch	Mittel	MQTT/HTTP
5.3	Server (Zukunft)	Nutzer-App/Webinterface	Anzeige von Status & Terminen	3	Mittel	Hoch	UI/Frontend
5.4	Server (Zukunft)	Automatische Abholanforderung	Meldung an Stadtverwaltung	3	Mittel	Hoch	API-Integration
5.5	Server (Zukunft)	Füllstands-Sharing	Optimierte Routenplanung	3	Mittel	Hoch	Datenaggregation

Tabelle 8: Übersicht der identifizierten Use Cases

2.5 Einordnung der Prioritäten

Priorität 1 – Kernumfang (muss umgesetzt werden):

Diese Funktionen sind zwingend notwendig, damit das System autonom, sicher und zuverlässig arbeitet:

- Autonome Linienfahrt
- Hindernis-Stopp
- Prozentuale Füllstandmessung
- Server-Datenverarbeitung
- Bidirektionale Kommunikation
- LED-Statusanzeige

Sie bilden die technische Basis für alle weiteren Funktionen.

Priorität 2 – Erweiterte Funktionalität (soll umgesetzt werden):

Diese Funktionen erhöhen Robustheit, Komfort und Systemintegration.

- Digitale Statusmeldung
- Not-Aus

Sie sind nicht zwingend für den Grundbetrieb, aber essenziell für ein „Smart“-System.

Priorität 3 – Zukunftsthemen / Komfortfunktionen (kann umgesetzt werden):

Diese Funktionen erweitern das System zu einem vollwertigen Smart-City-Element.

- Automatisches Andocken an Ladestation
- Digitaler Abholplan
- Lageerkennung
- Nutzer-App
- Automatische Abholanforderung
- Füllstands-Sharing
- Dynamische Hindernisumfahrung
- Geofencing

Sie erfordern zusätzliche Hardware, komplexere Software oder externe Schnittstellen.

2.6 Out-of-Scope im Projekt

Dieses Kapitel beschreibt alle Anforderungen, Funktionen und Use Cases, die **nicht** Teil des Projektumfangs waren. Die Abgrenzung erfolgt aufgrund von Zeitrahmen, technischer Komplexität, fehlenden Hardwarekomponenten oder strategischer Priorisierung. Gleichzeitig wird der Bezug zu möglichen Weiterentwicklungen hergestellt, sodass Folgeprojekte gezielt ansetzen können.

Nicht umgesetzte Funktionen und Use Cases

Die folgenden Themen wurden im Projekt **explizit ausgeschlossen** oder nur als Ausblick betrachtet:

- **Dynamische Hindernisumfahrung** Die Mülltonne umfährt Hindernisse nicht aktiv, sondern stoppt und wartet. *Grund:* Erhöhter Sensorik- und Logikaufwand, zusätzliche Ultraschallsensoren notwendig.
- **Erweiterte Smart-City-Funktionen** Dazu gehören automatische Abholanforderungen, Füllstands-Sharing zwischen Tonnen oder KI-basierte Routenoptimierung. *Grund:* Externe Schnittstellen, Backend-Integration und Datenaggregation nicht im Projektumfang.
- **Benutzerschnittstelle (App/Webportal)** Ein vollständiges Nutzerinterface zur Anzeige von Füllstand, Status oder Abholterminen wurde nicht umgesetzt. *Grund:* Fokus lag auf der autonomen Tonne, nicht auf Frontend-Entwicklung.
- **Geofencing / Diebstahlschutz** GPS-basierte Überwachung wurde nicht realisiert. *Grund:* Zusätzliche Hardware, hoher Energieverbrauch, komplexe Alarm-Logik.
- **Mechanische Selbstentleerung oder Aktorik** Die Tonne entleert sich nicht selbst und besitzt keine mechanischen Zusatzfunktionen. *Grund:* Fehlende Komponenten, hoher mechanischer Entwicklungsaufwand.
- **Erweiterte Sicherheitsfunktionen** Redundante Sensorik, Not-Aus-Logik mit Sicherheitszertifizierung oder TÜV-relevante Sicherheitskonzepte wurden nicht umgesetzt. *Grund:* Nicht erforderlich für den Prototypenstatus.
- **Autonomes Andocken an Ladestation:** Ein automatisches Anfahren und präzises Andocken an eine definierte Ladeposition wurden nicht umgesetzt. Grund hierfür sind der hohe Entwicklungsaufwand für Navigationslogik, Positionssensorik und mechanische Führungselemente sowie die fehlende Relevanz für den Prototypenstatus.

Gründe für die Abgrenzung

Die Ausgrenzung erfolgte entlang klarer Kriterien:

- **Zeitliche Begrenzung** Der Projektzeitraum erlaubte nur die Umsetzung der Kernfunktionen.
- **Technische Komplexität** Einige Funktionen erfordern zusätzliche Sensorik, komplexere Algorithmen oder umfangreiche Softwarearchitektur.
- **Fehlende Hardware** GPS-Module, zusätzliche Ultraschallsensoren oder IR-Baken standen nicht zur Verfügung.
- **Fokus auf Kernfunktionalität** Ziel war ein funktionsfähiger Prototyp mit autonomen Grundfunktionen, nicht ein vollständiges Smart-City-System.

Bezug zum Ausblick und zu offenen Punkten

Viele Out-of-Scope-Themen sind **bewusst als zukünftige Erweiterungen vorgesehen**. Sie bilden die Grundlage für:

- **Weiterentwicklung der Autonomie** (z. B. Hindernisumfahrung, Rückkehr zur Basis)
- **Integration in Smart-City-Infrastrukturen** (z. B. Füllstands-Sharing, automatische Abholanforderungen)
- **Erweiterte Nutzerinteraktion** (z. B. App, Dashboard, Benachrichtigungen)
- **Sicherheits- und Diebstahlschutzkonzepte** (z. B. Geofencing, Alarmfunktionen)

3 Mechanik und Konstruktion

3.1 Konzept und Aufbau

Das mechanische Gesamtkonzept des Systems basiert auf einer konsequenten funktionalen Trennung zwischen der äußeren Schutzhülle und der tragenden inneren Struktur. Diese Trennung ermöglicht eine robuste, wartungsfreundliche und reproduzierbare Konstruktion, bei der die äußere Hülle primär als Schutzkomponente fungiert, während sämtliche mechanischen Lasten durch einen speziell entwickelten Innenkorpus aufgenommen werden. Als äußere Struktur dient eine Centi-Abfalltonne mit einem Fassungsvermögen von 50 Litern, deren Materialeigenschaften und Geometrie gezielt für das System genutzt werden.¹ Tonne besteht aus widerstandsfähigem Polypropylen, einem thermoplastischen Kunststoff, der aufgrund seiner hohen Schlagzähigkeit, chemischen Resistenz und Witterungsbeständigkeit im industriellen Umfeld weit verbreitet ist. Diese Materialwahl gewährleistet, dass die Tonne auch bei mechanischen Einwirkungen wie Stößen, Kratzern oder Feuchtigkeitseintrag ihre Schutzfunktion zuverlässig erfüllt. Die konische Geometrie der Tonne stellt einen weiteren konstruktiven Vorteil dar, da sie eine formschlüssige Integration des Innenkorpus ermöglicht. Durch die sich nach unten verjüngende Innenkontur entsteht eine definierte Anlagefläche, an der sich der Innenkorpus selbstzentrierend positioniert.



Abbildung 4: Frontalansicht des Korpus vor dem Schnitt

¹ Otto.de; Mülleimer Abfalltonne

Der Boden der Tonne wurde vollständig entfernt, sodass die Außenhülle keinerlei strukturelle Lasten trägt. Sie dient ausschließlich dem Schutz der innenliegenden Komponenten vor Schmutz, Spritzwasser und Fremdkörpern sowie der Aufnahme definierter Öffnungen für Sensorik und Bedienkomponenten. Da die Tonne nicht tragend ausgelegt ist, kann sie bei Beschädigung oder Designänderungen ohne Einfluss auf die mechanische Integrität des Systems ausgetauscht werden, was die Servicefreundlichkeit erheblich erhöht.



Abbildung 5: Unterer Korpus nach dem Schnitt

Der Innenkorpus übernimmt die Funktion des tragenden Chassis und bildet das strukturelle Rückgrat des Gesamtsystems. Die Bauteile wurden vollständig in einem CAD-System konstruiert. Dies ermöglicht eine schnelle und präzise Modellierung sowie eine effiziente Anpassung und Optimierung der Geometrien. Nach Abschluss der Konstruktion wurden die Modelle als STL-Dateien exportiert und anschließend in einer Slicer-Software für den FDM-3D-Druck aufbereitet.

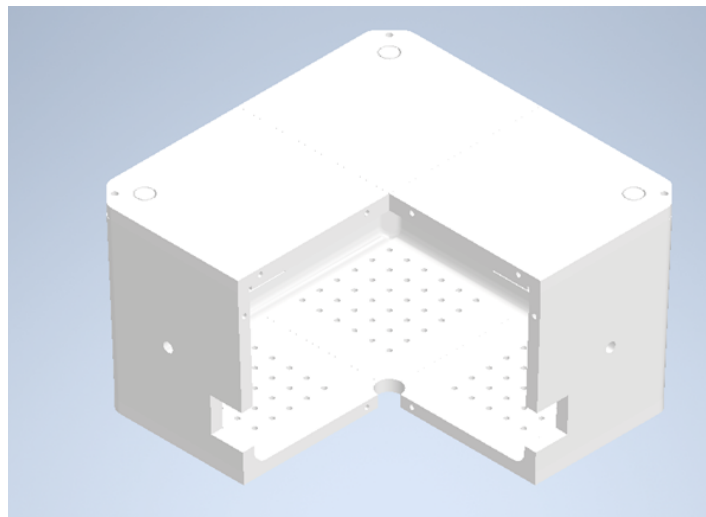


Abbildung 6: CAD-Modell Korpus mit Deckel

Der Innenkorpus besteht aus vier additiv gefertigten Segmenten, die im FDM-Verfahren hergestellt wurden. Dieses Fertigungsverfahren erlaubt die Umsetzung komplexer Geometrien, die mit konventionellen Herstellungsverfahren nur eingeschränkt oder überhaupt nicht realisierbar wären. Durch die segmentierte Bauweise bleibt das System modular, reproduzierbar und bei Bedarf leicht modifizierbar. Die vier Segmente sind symmetrisch aufgebaut und so gestaltet, dass sie sich exakt an die Innenkontur der Mülltonne anschmiegen. Die vier Segmente werden außerhalb mittels Stifte sowie Klebeverbindung dauerhaft gefügt. Diese Kombination aus form- und kraftschlüssigen Verbindungen gewährleistet eine hohe strukturelle Steifigkeit und verhindert Relativbewegungen zwischen den Segmenten. Durch diese formschlüssige Einbettung entsteht eine stabile Verbindung zwischen Innenkorpus und Außenhülle, ohne dass die Tonne selbst mechanisch belastet wird.



Abbildung 7: Zusammengefügtter Korpus

Die Konstruktion des Innenkorpus erzeugt eine plane und hochstabile Bodenfläche, die als zentrale Montageplattform für das Fahrwerk dient. Montagepunkte wurden rasterförmig angeordnet, um eine flexible Montage möglicher Komponenten zu gewährleisten. Dadurch wird eine reproduzierbare Fertigung und eine präzise sowie verstellbare Positionierung aller Komponenten ermöglicht. Besonders hoch belastete Bereiche, wie die Aufnahmepunkte des Fahrwerks, wurden gezielt durch erhöhte Wandstärken ausgelegt. Diese konstruktiven Maßnahmen erhöhen die Lebensdauer des Systems und reduzieren das Risiko von Materialermüdung. Der Innenkorpus ist über vier Schrauben lösbar mit der Mülltonne verbunden, sodass er für Wartungsarbeiten oder Modifikationen vollständig entnommen werden kann. Diese modulare Bauweise erleichtert nicht nur die Instandhaltung, sondern ermöglicht auch zukünftige Anpassungen oder Erweiterungen des Systems. Ein ebenfalls additiv gefertigter Deckel

verschließt den Innenraum nach oben und schützt die Elektronik vor Schmutz und Feuchtigkeit. Im Deckel sind Kabeldurchführung, Befestigungspunkte und strukturelle Verstärkungen direkt in die Geometrie integriert. Dadurch entsteht ein definierter und geschützter Innenraum, der eine klare funktionale Trennung zwischen dem Müllbereich und den elektrischen Komponenten gewährleistet.

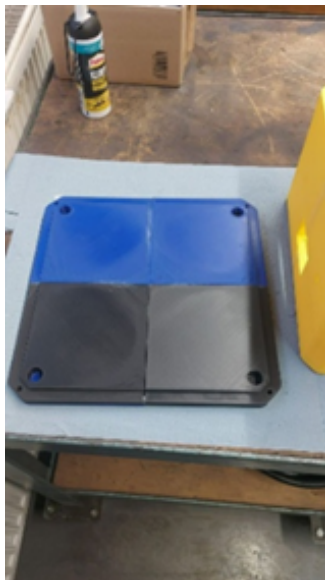


Abbildung 8: Deckel des Korpus

Das Fahrwerk ist unmittelbar unterhalb des Innenkorpus montiert und bildet die mechanische Grundlage für die Mobilität des Systems. Die Fahrwerksplattform wird von unten an die verstärkte Unterseite des Innenkorpus angesetzt. Die Schrauben werden an definierten Positionen durch den Boden des Korpus geführt, wobei die Bohrungen so ausgelegt sind, dass eine präzise Ausrichtung der Bauteile gewährleistet ist. Auf der gegenüberliegenden Seite werden die Schrauben mit passenden Muttern verschraubt. Durch das Anziehen der Verbindung entsteht ein sicherer, formschlüssiger und dauerhaft belastbarer Verbund zwischen den Komponenten. Insgesamt trägt die Verschraubung wesentlich zur strukturellen Integrität des Systems bei und stellt sicher, dass die montierten Baugruppen zuverlässig fixiert bleiben. Diese konstruktive Lösung stellt sicher, dass sämtliche Kräfte, die durch Motoren, Ketten und Fahrbewegungen entstehen, vollständig in den Innenkorpus eingeleitet werden. Die Mülltonne dient ausschließlich als schützende Hülle und wird mechanisch nicht belastet. Die hohe strukturelle Steifigkeit des Innenkorpus reduziert Vibrationen und verhindert Resonanzen, was die Lebensdauer der mechanischen und elektronischen Komponenten erhöht. Die CAD-basierte Konstruktion ermöglicht zudem eine präzise und reproduzierbare Montage des Fahrwerks, was insbesondere bei Wartung, Austausch oder Weiterentwicklung des Systems von Vorteil ist. Durch die klare Trennung zwischen tragender Struktur und äußerer Hülle entsteht ein robustes und zugleich servicefreundliches Gesamtsystem.



Abbildung 9: Fahrwerkanbindung am Korpus

Für die Sensorintegration wird ein speziell entwickelter, additiv gefertigter Halter eingesetzt, der am unteren Bereich des Innenkorpus montiert ist und eine präzise Aufnahme für mehrere Liniensensoren bietet. Die Sensoren sind parallel zur Fahrtrichtung ausgerichtet und so tief wie möglich positioniert, um eine optimale Erkennung von Bodenmarkierungen zu gewährleisten. Die zentrale Positionierung zwischen den beiden Ketteneinheiten sorgt für eine symmetrische Erfassung des Fahrwegs und minimiert Messfehler durch seitliche Abweichungen. Die Befestigung erfolgt über durchgehende Schrauben mit Gegenplatten im Inneren des Korpus, wodurch eine hohe Positionsstabilität selbst bei starken Vibrationen oder unebenem Untergrund erreicht wird. Die Kabel der Sensoren werden geschützt in den Innenraum geführt, was ein sauberes und wartungsfreundliches Kabelmanagement ermöglicht. Die CAD-basierte Konstruktion des Halters erlaubt zudem eine einfache Anpassung oder Erweiterung, etwa für zusätzliche Sensoren oder alternative Sensortypen, und erhöht damit die Flexibilität des Gesamtsystems.

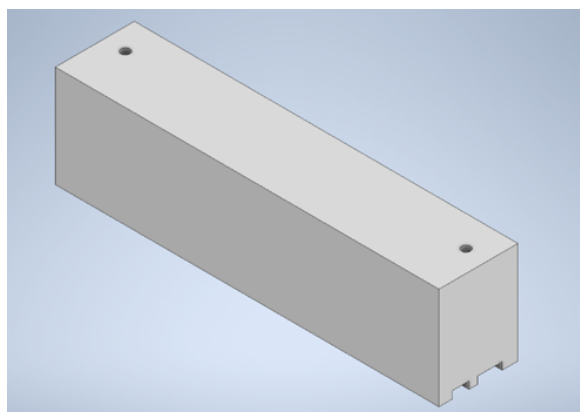


Abbildung 10: Ansicht 1 des Liniensensorhalters

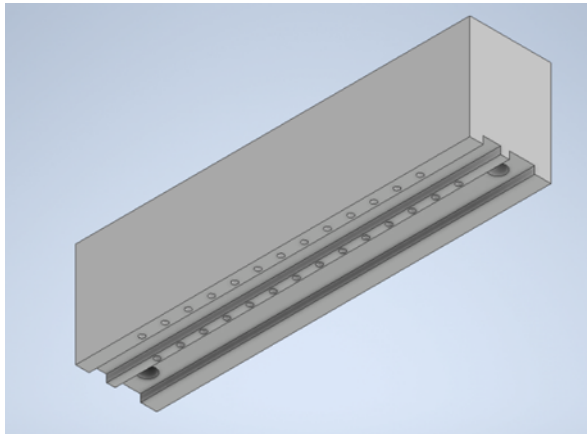


Abbildung 11: Ansicht 2 des Linienensorhalters



Abbildung 12: Anbindung der Linienensoren am Korpus

Ein identischer additiv gefertigter Halter wird an zwei unterschiedlichen Stellen im System eingesetzt, jedoch mit jeweils spezifischer Funktion. Am Deckel der Mülltonne dient er zur Aufnahme eines Füllstandssensors, der senkrecht nach unten ausgerichtet ist und den Abstand zum Müll misst. Die Befestigung erfolgt über Klebeverbindungen, während die Kabelführung über einen angeklebten Kabelkanal an der Innenseite des Deckels realisiert wird. Im Innenkorpus wird derselbe Halter zur Hinderniserkennung eingesetzt. Eine Aussparung in der Vorderseite der Mülltonne ermöglicht die Erfassung von Objekten im Fahrweg. Die doppelte Nutzung eines identischen Bauteils reduziert die Variantenvielfalt, vereinfacht die Ersatzteilproduktion und erhöht die Systemeffizienz, während gleichzeitig eine klare funktionale Trennung zwischen Füllstandsmessung und Hinderniserkennung erhalten bleibt.

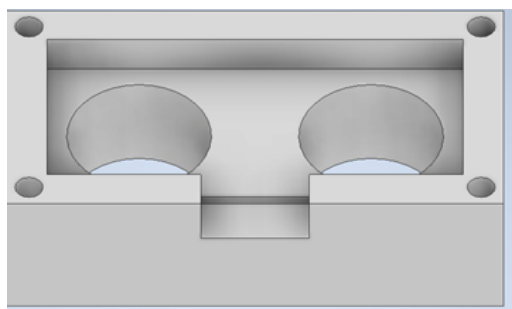


Abbildung 13: Halter Ultraschallsensor

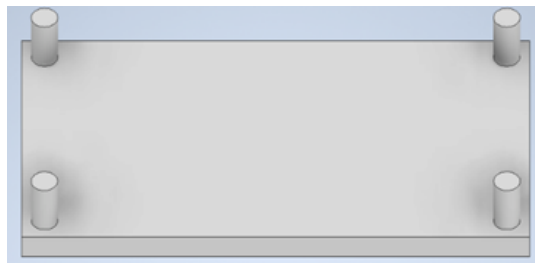


Abbildung 14: Deckel des Halters

An der Vorderseite der Mülltonne befindet sich schließlich eine additiv gefertigte Anzeige und Bedienplatte, die sämtliche Interaktions- und Statuskomponenten des Systems integriert. In dieser Frontplatte sind ein akustischer Signalgeber, zwei LED-Leuchten hinter gedruckten Acrylhauben, ein Hauptschalter zur Akkutrennung, zwei Taster sowie ein externer Ladeanschluss verbaut. Die vollständig CAD-basierte Konstruktion gewährleistet eine reproduzierbare Fertigung und eine saubere Integration aller Komponenten. Die Frontplatte ist vollständig demontierbar, was Wartungsarbeiten erheblich erleichtert und eine schnelle Zugänglichkeit aller Bedienelemente ermöglicht. Durch die additive Fertigung können Ausschnitte, Kabelkanäle und Befestigungspunkte präzise integriert werden, was die Funktionalität und Wartungsfreundlichkeit weiter erhöht.

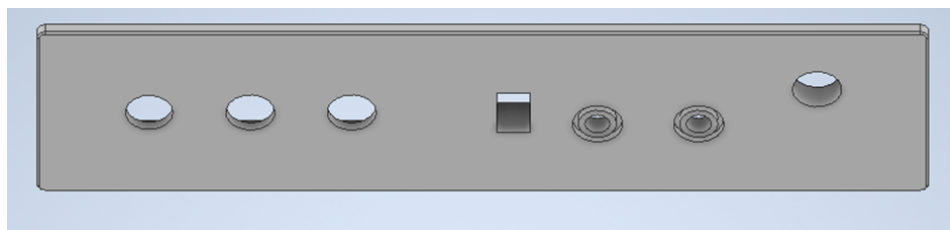


Abbildung 15: Anzeige/Bedienplatte



Abbildung 16: Montierte Anzeige/Bedienplatte

3.2 Antriebskonzept

Das entwickelte Robotiksystem nutzt ein Raupenfahrwerk als zentrales Antriebskonzept, das vollständig in die modulare Gesamtarchitektur integriert wurde. Während die bisherigen Erprobungen ausschließlich auf glatten, festen Innenflächen der Hochschule stattfanden, ist das System langfristig für den autonomen Einsatz im Außenbereich vorgesehen. Ziel ist es, die

Mülltonne selbstständig vom Haus bis zur Straßenkante zu bewegen und dort für die Müllabfuhr bereitzustellen. Diese Anwendung stellt erhöhte Anforderungen an Traktion, Stabilität und Robustheit, die über die Bedingungen der bisherigen Testumgebung hinausgehen.

Fahrverhalten auf glatten Innenflächen

In der Innenraumumgebung zeigte das Raupenfahrwerk ein sehr stabiles und präzise steuerbares Fahrverhalten. Die kontinuierliche Aufstandsfläche der Ketten führt zu einer gleichmäßigen Lastverteilung und ermöglicht eine vibrationsarme Bewegung, was insbesondere für die integrierte Sensorik und Elektronik vorteilhaft ist. Die hohe Traktion sorgt dafür, dass das System auch bei niedrigen Geschwindigkeiten und minimalen Steuerimpulsen ohne Schlupf reagiert. Dadurch konnten reproduzierbare Testfahrten unter kontrollierten Bedingungen zuverlässig durchgeführt werden.

Ein erhöhter Energieverbrauch wurde ebenfalls festgestellt, der auf den im Vergleich zu Radfahrwerken größeren Rollwiderstand zurückzuführen ist. Dieser Effekt ist auf glatten Böden besonders ausgeprägt, bleibt jedoch innerhalb der Projektanforderungen akzeptabel.

Vorteile des Raupenfahrwerks im geplanten Außeneinsatz

Für den realen Einsatz im Außenbereich ergeben sich zahlreiche funktionale Vorteile, die das Raupenfahrwerk gegenüber alternativen Antriebskonzepten besonders geeignet machen:

Hohe Traktion und zuverlässige Kraftübertragung

Die großflächige Kontaktzone zwischen Kette und Untergrund erzeugt eine hohe Reibkraft, wodurch das System auch auf feuchten, rutschigen oder unebenen Oberflächen zuverlässig manövrieren kann. Dies ist entscheidend, da die Mülltonne unabhängig von Witterung und Untergrundbeschaffenheit sicher bewegt werden muss.

Gleichmäßige Lastverteilung und hohe Kippstabilität

Die kontinuierliche Aufstandsfläche verteilt das Gewicht über eine große Fläche, wodurch Einsinken aufweichen oder feuchten Böden verhindert wird. Selbst bei asymmetrischer Beladung, etwa durch Akkus, Steuerhardware oder Sensorik, bleibt das System kippstabil. Der konstruktiv niedrige Schwerpunkt reduziert das Kippmoment zusätzlich und sorgt für ein stabiles Fahrverhalten auch bei Schräglagen oder dynamischen Bewegungen.

Fähigkeit zur Hindernisüberwindung

Die Kettenführung ermöglicht das Überfahren kleiner Hindernisse wie Bordsteinkanten, Pflasterfugen oder Niveauunterschiede zwischen Gehweg und Grundstückszufahrt. Während Räder an solchen Stellen häufig hängen bleiben, ermöglicht die Kettenmechanik das

kontrolliert überwinden von Kante. Diese Fähigkeit ist essenziell, da der Weg vom Haus zur Straße selten vollständig eben ist.

Geringer Bodendruck und hohe Geländegängigkeit

Durch die große Aufstandsfläche entsteht ein niedriger spezifischer Bodendruck. Das verhindert ein Einsinken auf weichen Untergründen wie nassem Gras oder Erde und schont gleichzeitig den Untergrund im Wohnumfeld.

Vibrationsarme Bewegung und Schutz der Elektronik

Die Ketten dämpfen Stöße und Vibrationen, da sie Unebenheiten kontinuierlich abrollen. Die robuste mechanische Struktur schützt empfindliche Komponenten wie Mikrocontroller, Sensorik und Akkus vor Stoßbelastungen, die im Außenbereich deutlich häufiger auftreten als in einer kontrollierten Innenumgebung. Dies erhöht die Betriebssicherheit und die Lebensdauer der Elektronik.

Präzise Steuerbarkeit und reproduzierbares Fahrverhalten

Das Fahrwerk reagiert feinfühlig auf Steuerimpulse und ermöglicht Drehungen auf der Stelle. Dies erleichtert die Navigation in engen Bereichen und unterstützt die autonome Pfadplanung. Die hohe Traktion sorgt zudem für ein reproduzierbares Fahrverhalten, was für die Entwicklung und Validierung der Steuerungsalgorithmen entscheidend ist.

Robustheit und Langlebigkeit

Raupenfahrwerke sind konstruktiv für raue Umgebungen ausgelegt und tolerieren Schmutz, Feuchtigkeit und kleine Steine deutlich besser als Radfahrwerke. Der Verschleiß verteilt sich gleichmäßiger, was die Wartungsintervalle verlängert.

Bewertung der bisherigen Tests und Ausblick

Die Tests auf glatten Innenflächen liefern eine wichtige Grundlage für die Bewertung des Fahrwerks, zeigen jedoch auch die Grenzen der Innenraumvalidierung auf. Während Energieverbrauch und Verschleiß bei Drehungen auf der Stelle bereits unter Laborbedingungen messbar waren, ist im Außenbereich mit zusätzlichen Belastungen zu rechnen. Dazu zählen erhöhte Reibkräfte auf rauerer Oberflächen, Witterungseinflüsse wie Feuchtigkeit oder Schmutz sowie mechanische Beanspruchungen durch unregelmäßige Untergründe. Das Raupenfahrwerk bietet jedoch konstruktiv die notwendigen Voraussetzungen, um diesen Herausforderungen zu begegnen.

Zusammenfassung

Das Raupenfahrwerk erfüllt sowohl in der bisherigen Testumgebung als auch im geplanten Außeneinsatz die funktionalen Anforderungen des Systems. Während die Innenraumtests

eine präzise und reproduzierbare Bewertung von Steuerung, Stabilität und Energieeffizienz ermöglichen, adressiert das Fahrwerk zugleich die Anforderungen der realen Zielanwendung. Die Kombination aus hoher Traktion, stabiler Lastverteilung, geringerem Bodendruck, Hindernisüberwindung und robuster Bauweise macht das Raupenfahrwerk zu einer idealen Grundlage für die autonome Bereitstellung einer Mülltonne vom Haus zur Straße.

3.3 Iterationen und Optimierungen

3.3.1 Iteration – Anpassung des Innenkorpus

Ausgangslage und Problemstellung

Im ersten Konstruktionsdurchlauf wurde ein Innenkorpus entwickelt, der als zentrale Trägerstruktur für Sensorik, Elektronik und mechanische Komponenten dienen sollte. Beim ersten physischen Einsetzen zeigte sich jedoch, dass der Korpus nicht in die Mülltonne passte. Die Fehlpassung war so ausgeprägt, dass weder ein vollständiges Einschieben noch ein verkantungsfreies Positionieren möglich war.

Analyse der ursprünglichen Geometrie und beobachtete Effekte

Die Ecken des Innenkorpus waren mit tangentialen Bögen versehen, deren Krümmungsverlauf nicht exakt der realen Innenkontur der Mülltonne entsprach. Da die Innenradien der Mülltonne nur begrenzt und mit Messunsicherheit erfassbar waren, kam es zu lokalen Kollisionen. Zusätzlich war der Neigungswinkel der Seitenwände zu groß gewählt, sodass der Korpus bereits im oberen Bereich mit den Innenwänden kollidierte.

Entscheidungsgrundlage für die Optimierung

Da die realen Innenradien nicht zuverlässig messbar waren, wurde entschieden, die Rundungen vollständig zu entfernen und durch klar definierte 45-Grad-Schrägen zu ersetzen. Gleichzeitig musste die Konizität des Korpus so angepasst werden, dass sie der realen Verjüngung der Mülltonne entspricht.

Umsetzung der Optimierung und Auswirkungen auf das Gesamtsystem

Im zweiten Entwurf wurden die tangentialen Bögen durch 45-Grad-Schrägen ersetzt. Die Höhe des Korpus wurde auf 150 mm festgelegt, die Grundfläche unten auf 260 mm × 260 mm und die obere Öffnung auf 270 mm × 270 mm. Diese Geometrie passte exakt in die Mülltonne und bot ausreichend Toleranz für Fertigung und Montage.

Zusammenfassung der Iteration

Die Überarbeitung des Innenkorpus führte zu einer reproduzierbaren, montagefreundlichen und funktional robusten Lösung. Die neue Geometrie gewährleistete eine zuverlässige Passform und bildete die Grundlage für die Integration aller weiteren Module.

3.3.2 Iteration – Motorwechsel zur Lösung des Lastproblems

Ausgangslage und Problemstellung

Zu Beginn der mechanischen Entwicklung wurde das Fahrwerk mit den im gelieferten Kettenchassis integrierten Gleichstrommotoren betrieben. Diese Motoren waren für einfache Modellbauanwendungen ausgelegt und verfügten über keine dokumentierten Leistungsparameter wie Drehmoment, Übersetzung oder Dauerlastfähigkeit. Bereits in frühen Testläufen zeigte sich, dass diese Antriebseinheit nicht für die im Projekt vorgesehenen Lasten geeignet war. Insbesondere die Kombination aus dem modularen Innenkorpus, der Sensorik, der Energieversorgung und der Verkabelung führte zu einer deutlichen Erhöhung der Systemmasse, die die ursprünglichen Motoren nicht zuverlässig bewältigen konnten.²

Analyse der ursprünglichen Motoren und beobachtete Effekte

Im praktischen Betrieb traten mehrere charakteristische Symptome einer Unterdimensionierung auf. Unter moderater Zusatzlast verloren die Motoren signifikant an Drehmoment, was sich in Geschwindigkeitseinbrüchen, ungleichmäßigem Fahrverhalten und zeitweiligem Stillstand einzelner Kettenseiten äußerte. Zusätzlich erwärmten sich die Motoren bereits nach kurzer Betriebsdauer, was auf ineffiziente Stromaufnahme und eine unzureichende Getriebeauslegung hinweist. Die hohe Leerlaufdrehzahl in Verbindung mit einer geringen oder nicht vorhandenen Übersetzung erschwerte zudem eine präzise Regelung der Fahrgeschwindigkeit.

Ein weiteres deutliches Indiz für die unzureichende Auslegung war die notwendige Ansteuerung über sehr hohe PWM-Werte: Ein stabiler Fahrbetrieb war erst ab einer PWM von mindestens 80 % möglich und selbst dann lief das System nicht sauber oder reproduzierbar. Dies verdeutlicht, dass die Motoren bereits im unteren Lastbereich an ihre Leistungsgrenzen stießen und kaum nutzbares Drehmoment im relevanten Arbeitsbereich bereitstellten

² Amazon.de; Raupen Chassis

Entscheidungsgrundlage für die Optimierung

Die identifizierten Defizite führten zu der Notwendigkeit, den Antriebsstrang grundlegend zu überarbeiten. Für die Auswahl geeigneter Ersatzmotoren wurden klare technische Kriterien definiert. Dazu gehörten eine hohe Dauerlastfähigkeit, eine robuste Getriebekonstruktion, eine moderate und regelbare Drehzahl sowie vollständig dokumentierte Leistungsparameter. Diese Anforderungen dienten als Grundlage für die Entscheidung, die ursprünglichen Motoren durch 12-V-Metallgetriebemotoren mit einer Untersetzung von 50:1 und einer Leerlaufdrehzahl von 100 U/min zu ersetzen.³

Umsetzung der Optimierung und Auswirkungen auf das Gesamtsystem

Nach dem Austausch der Motoren zeigte das Fahrwerk ein grundlegend verbessertes Verhalten. Die erhöhte Untersetzung ermöglichte eine stabile Kraftübertragung, wodurch der Roboter den vollständigen Innenkorpus inklusive Batterie und Sensorik ohne Leistungseinbrüche bewegen konnte. Die thermische Stabilität verbesserte sich deutlich, und die definierte Drehzahl führte zu einer erheblich verbesserten Regelbarkeit. Das Metallgetriebe zeigte während der Testphase keinerlei Verschleißerscheinungen.

Zusammenfassung der Iteration

Der Motorwechsel stellte einen zentralen Meilenstein der mechanischen Entwicklung dar. Die ursprünglichen Motoren waren für die reale Systemlast ungeeignet und führten zu instabilem Fahrverhalten sowie thermischen Problemen. Durch die Integration hochwertiger Metallgetriebemotoren konnte das Lastproblem vollständig gelöst werden.

3.3.3 Iteration – Strukturelle Versteifung des Fahrwerks

Ausgangslage und Problemstellung

Im weiteren Verlauf der mechanischen Entwicklung zeigte sich, dass das vorhandene Fahrwerk des Kettenantriebs strukturell nicht ausreichend steif war. Unter realen Betriebsbedingungen, insbesondere bei unebenen Untergründen und voller Systemlast, kam es zu einer vertikalen Durchbiegung des Chassis. Diese Verformung führte dazu, dass die Ketten an den unteren Bereichen des Innenkorpus schleiften. Das Schleifen erzeugte zusätzlichen Reibungswiderstand, beeinträchtigten die Spurtreue und erhöhte die Belastung der Antriebseinheiten.

³ Robotshop.com; Gleichstrom-Bürstenmotor

Damit wurde deutlich, dass die strukturelle Integrität des Fahrwerks eine zentrale Schwachstelle darstellte.

Analyse der ursprünglichen Konstruktion und beobachtete Effekte

Das Fahrwerk bestand aus einem dünnwandigen Aluminiumgehäuse, das zwar leicht, jedoch nicht ausreichend verwindungssteif war. Besonders im mittleren Bereich des Chassis trat eine deutliche Durchbiegung auf, die sich bei dynamischen Bewegungen weiter verstärkte. Die resultierenden Reibkontakte zwischen Ketten und Innenkorpus führten zu Leistungsverlusten, unruhigem Fahrverhalten und einem erhöhten Verschleiß der Ketten. Die strukturelle Instabilität wirkte sich somit direkt negativ auf die mechanische Effizienz und die Zuverlässigkeit des Gesamtsystems aus.

Entscheidungsgrundlage für die Optimierung

Für die Optimierung wurde eine Lösung benötigt, die das bestehende Fahrwerk gezielt verstärkt, ohne eine vollständige Neukonstruktion zu erfordern. Kunststoffverstärkungen schieden aufgrund ihrer begrenzten Steifigkeit aus, und ein kompletter Neuaufbau des Fahrwerks wäre zeitlich und ressourcentechnisch nicht sinnvoll gewesen. Die Wahl fiel daher auf eine Verstärkung durch metallische Strukturelemente. Edelstahl bot aufgrund seiner hohen Steifigkeit, Korrosionsbeständigkeit und guten Verfügbarkeit eine geeignete Grundlage. Ziel war es, eine kraftschlüssige, modulare und nachrüstbare Lösung zu schaffen, die das Durchbiegen zuverlässig verhindert.

Umsetzung der Optimierung und Auswirkungen

Zur Versteifung des Fahrwerks wurden zwei Edelstahlwinkel an den kritischen Bereichen des Chassis montiert. Diese Winkel erhöhten die seitliche und vertikale Steifigkeit des Rahmens. Zusätzlich wurden zwei parallel verlaufende Gewindestangen durch das Fahrwerk geführt und beidseitig mit Muttern vorgespannt. Die Stangen wirkten als tragende Elemente, die die Last gleichmäßig verteilten und die Durchbiegung effektiv eliminierten.

Nach der Umsetzung zeigte das Fahrwerk ein deutlich verbessertes strukturelles Verhalten. Die Durchbiegung wurde vollständig verhindert, die Ketten liefen frei ohne Kontakt zum Innenkorpus, und die Reibungsverluste reduzierten sich spürbar. Das Fahrverhalten wurde stabiler, die Spurtreue verbessert und die Antriebseinheiten entlastet, da keine zusätzlichen Widerstände mehr überwunden werden mussten.

Zusammenfassung der Iteration

Die strukturelle Versteifung des Fahrwerks stellte eine zentrale Optimierung dar, um die mechanische Robustheit des Systems sicherzustellen. Durch die Integration von Edelstahlwinkeln

und zwei vorgespannten Gewindestangen konnte die Steifigkeit des Chassis erheblich erhöht werden. Die Maßnahme eliminierte Reibkontakte, reduzierte Leistungsverluste und verbesserte das Fahrverhalten nachhaltig. Damit bildet diese Iteration einen entscheidenden Baustein für die zuverlässige Funktion des Gesamtsystems.

4 Elektronik, Energie und Verdrahtung

4.1 Komponentenübersicht

Die elektronische Ausrüstung der smarten Mülltonne wurde so ausgelegt, dass die zentralen Funktionen des Systems wie autonome Bewegung, Füllstandserfassung, Hinderniserkennung, Zustandsanzeige und Kommunikation mit dem Server mit einer möglichst kompakten und robusten Hardwarearchitektur realisiert werden können. Im Fokus stand dabei eine praxisnahe Prototypenlösung, die einerseits zuverlässig im Betrieb funktioniert und andererseits den Aufbau sowie die späteren Erweiterungsmöglichkeiten nachvollziehbar abbildet.

Die zentrale Steuereinheit bildet ein Raspberry Pi Pico 2 W, der als Mikrocontroller die Sensorwerte verarbeitet, die Zustandslogik ausführt und die Kommunikation mit dem Server übernimmt. Die Wahl fiel auf dieses Board, da es eine gute Kombination aus Rechenleistung, kompakter Bauform und integrierter Funkanbindung bietet. Für ein eingebettetes System mit mehreren Sensoren, Aktoren und einer übergeordneten Kommunikationslogik stellt der Pico damit eine geeignete Steuerplattform dar.

Zur Ansteuerung des Fahrwerks wird ein L298N-Motortreiber eingesetzt. Dieser übernimmt die elektrische Ansteuerung der beiden Fahrmotoren und stellt die Schnittstelle zwischen der digitalen Steuerung des Mikrocontrollers und der mechanischen Antriebseinheit dar. Durch den separaten Motortreiber wird die Last vom Mikrocontroller entkoppelt, sodass die Motoren unabhängig von den Logiksignalen sicher und kontrolliert betrieben werden können. Das Fahrwerk selbst ist als Raupenfahrwerk ausgeführt und wurde mit zwei 12-V-DC-Motoren vom Typ FIT0492-B realisiert, wodurch die Tonne auch auf unebenem Untergrund ausreichend Traktion erhält.

Die Stromversorgung des Systems erfolgt über einen LONG Bleiakku WP5-12E mit 12 V und 5 Ah. Diese Energiequelle stellt eine robuste und im Verhältnis zur Anwendung ausreichende Versorgung für den Betrieb des Prototyps bereit. Der L298N-Motortreiber wird direkt aus dem Akku gespeist und stellt über seinen integrierten 5-V-Regler die Versorgungsspannung für den Raspberry Pi Pico 2 W bereit, der daraus intern die 3,3-V-Logikversorgung für Sensorik, LEDs und Buzzer erzeugt. Ergänzt wird das Energiekonzept durch eine DC-Buchse, über die ein externes 18–19-V-Netzteil angeschlossen wird, sowie einen Wippschalter, über den die Bordnetzversorgung manuell vollständig getrennt werden kann.

Für das Laden des 12-V-AGM-Akkus ist ein XL4015-Buck-Converter fest in das System integriert. Dieses Modul übernimmt ausschließlich die Ladefunktion und arbeitet als

Konstantstrom-/Konstantspannungs-Ladegerät (CC/CV). Es wandelt die Eingangsspannung des externen Netzteils (18–19 V) auf eine Ladeschlussspannung von 14,8 V bei einem Ladestrom von 1 A und bildet damit die elektrochemisch erforderliche IU-Kennlinie für AGM-Akkus ab. Im Normalbetrieb (ohne angeschlossenes Netzteil) greift der XL4015 nicht in das Bordnetz ein, sodass der Akku die Verbraucher direkt versorgt.

Für die sensorische Erfassung des Systems wurden mehrere Komponenten integriert. Zur Füllstandsmessung befindet sich im Deckel der Mülltonne ein HC-SR04P-Ultraschallsensor, der in ein 3D-gedrucktes Gehäuse eingebaut und innen im Deckel befestigt wurde. Der Sensor ist nach unten auf den Tonneninhalt ausgerichtet und misst im Betrieb den Abstand zur Mülloberfläche. Aus diesem Abstand kann der Füllstand des Behälters bestimmt werden. Die Montage im Deckel ermöglicht eine geschützte und gleichzeitig funktionale Positionierung des Sensors.

Zur Hinderniserkennung wurde ebenfalls ein HC-SR04P-Ultraschallsensor verbaut. Dieser befindet sich im vorderen, unteren Bereich des Mülltonnenkorpus. Hierfür wurde ein passender Ausschnitt im Korpus geschaffen und der Sensor samt 3D-gedrucktem Gehäuse eingesetzt. Während der Fahrt misst er die Distanz zu Objekten im Fahrweg. Wird ein zu geringer Abstand erkannt, stoppt das System automatisch, um eine Kollision zu vermeiden. Der Sensor trägt damit wesentlich zur sicheren Bewegung der Mülltonne im autonomen Fahrmodus bei.

Die Linienverfolgung basiert auf fünf KY-033-Infrarotsensoren, die im vorderen Bereich des Fahrwerks angeordnet sind. Diese Sensoren erfassen den Kontrast zwischen Linie und Untergrund und liefern die Grundlage für die Navigationslogik der Mülltonne. Durch die Verteilung mehrerer Sensoren kann die Position der Linie präzise ermittelt und die Fahrtrichtung entsprechend korrigiert werden. Dies ist insbesondere für einen stabilen Linienfolgebetrieb erforderlich.

Das User Interface wurde bewusst übersichtlich gehalten, um die Bedienung des Systems möglichst direkt und intuitiv zu gestalten. Es umfasst einen grünen Taster zum Starten der Fahrt, einen roten Taster zum Pausieren des Betriebs, eine RGB-Betriebszustands-LED zur Anzeige interner Systemzustände, eine RGB-Kommunikations-LED zur Rückmeldung des Serverstatus sowie einen Buzzer für akustische Signale. Die Kombination aus visuellen und akustischen Elementen verbessert die Verständlichkeit des Betriebszustands und erleichtert dem Nutzer die Interaktion mit dem System.

Als mechanische Basis wurde ein gekaufter Mülltonnenkorpus verwendet, dessen Boden entfernt und durch ein neu konstruiertes 3D-gedrucktes Innenelement ersetzt wurde. Dieses Element dient als Aufnahme- und Strukturbauteil für die im Inneren verbaute Elektronik und ermöglicht zugleich die stabile Verschraubung des Fahrwerks. Auf diese Weise konnte der

ursprünglich passive Korpus zu einem funktionalen Träger des Gesamtsystems umgebaut werden. Die mechanische Integration bildet damit die Grundlage für den kompakten Aufbau der smarten Mülltonne.

Zusammenfassend zeigt sich, dass die Hardware des Systems aus klar abgegrenzten, funktional aufeinander abgestimmten Baugruppen besteht. Die Kombination aus Mikrocontroller, Antrieb, Energieversorgung, Sensorik und Bedienoberfläche bildet eine robuste technische Basis für den Prototypen. Gleichzeitig wurde bei der Auswahl und Integration der Komponenten darauf geachtet, dass das System nachvollziehbar dokumentierbar bleibt und eine sinnvolle Grundlage für spätere Erweiterungen bietet.

4.2 Blockschaltbild

Das Blockschaltbild stellt die elektrische und funktionale Verknüpfung der zentralen Systemkomponenten dar und bildet damit die Brücke zwischen der allgemeinen Systemarchitektur und dem detaillierten Schaltplan in Kapitel 5.3. Es zeigt, wie Energieversorgung, Signalverarbeitung, Aktorik, Sensorik und Benutzerinteraktion im Gesamtsystem zusammenwirken.

Im Zentrum der Steuerung steht der Raspberry Pi Pico 2 W, der als Mikrocontroller die Sensordaten verarbeitet, die Zustandslogik ausführt und die Steuersignale für Antrieb und Anzeigeelemente erzeugt. Die Bordnetzversorgung erfolgt über einen 12-V-AGM-Bleiakku (LONG WP5-12E, 5 Ah), der direkt den L298N-Motortreiber speist. Der L298N stellt über seinen integrierten 5-V-Regler die Versorgungsspannung für den Raspberry Pi Pico 2 W bereit, der daraus intern die 3,3-V-Logikspannung für Sensorik, LEDs und Buzzer erzeugt. Das externe 18–19-V-Netzteil ist über eine DC-Buchse an ein XL4015-Modul angebunden, das ausschließlich die Ladefunktion des Akkus übernimmt (CC/CV-Laderegulierung) und nicht direkt zur 5-V- oder 3,3-V-Versorgung beiträgt.

4.2.1 Funktionsblöcke

Der elektrische Aufbau lässt sich in vier Hauptbereiche gliedern:

Energieversorgung:

- 12-V-Bleiakku
- DC-Buchse für das externe Netzteil
- XL4015-Lademodul zur Umsetzung der CC/CV-Ladecharakteristik
- Wippschalter zur vollständigen Trennung der Bordnetzversorgung

Steuerungseinheit:

- Raspberry Pi Pico 2 W als zentraler Rechenknoten
- interne 3,3-V-Logikversorgung für alle digitalen Komponenten

Aktorik:

- L298N-Motortreiber als Schnittstelle zwischen Mikrocontroller und Fahrmotoren
- zwei 12-V-DC-Motoren des Raupenfahrwerks
- Buzzer und RGB-LEDs als Ausgabeelemente

Sensorik und Bedienung:

- fünf KY-033-Liniensensoren für die Linienverfolgung
- zwei HC-SR04P-Ultraschallsensoren für Füllstand und Hinderniserkennung
- zwei Taster (Start/Stopp) für die manuelle Bedienung

Die Signalrichtung ist dabei klar strukturiert: Sensoren liefern Eingangsdaten an den Mikrocontroller, der daraus Steuerbefehle ableitet und diese an Motortreiber, LEDs und Buzzer weitergibt. Zusätzlich werden Statusinformationen an den Server übertragen, sodass die Mülltonne nicht isoliert arbeitet, sondern in eine übergeordnete Missionslogik eingebettet ist.

4.2.2 Energiepfad

Die Stromversorgung beginnt am 12-V-Bleiakku, der die Anlage mit der benötigten Grundspannung versorgt. Über den Wippschalter kann diese Versorgung vollständig getrennt werden, was sowohl für Wartungsarbeiten als auch für einen sicheren Stillstand des Systems relevant ist. Der L298N-Motortreiber wird direkt aus dem Akku gespeist und versorgt die Fahrmotoren mit 12 V. Gleichzeitig stellt der integrierte 5-V-Regler des L298N die Versorgungsspannung für den Raspberry Pi Pico 2 W bereit. Der Pico erzeugt daraus intern 3,3 V und versorgt damit die Sensorik und das User Interface. Der XL4015-Wandler bildet einen separaten Ladezweig: Er wandelt die 18–19 V des externen Netzteils auf eine für den AGM-Akku geeignete Ladespannung von 14,8 V bei begrenztem Ladestrom und lädt den Akku, ohne die laufende Bordnetzversorgung zu stören.

Die Trennung zwischen Leistungs- und Logikebene ist funktional wichtig, da die Motoren im Betrieb deutlich höhere Ströme benötigen als die Steuer- und Sensorsignale. Durch die klare Aufteilung in 12-V-Leistungsebene, 5-V-Zwischenversorgung und 3,3-V-Logikebene wird verhindert, dass Lastspitzen des Fahrwerks die Stabilität der Logikversorgung beeinträchtigen.

4.2.3 Signalfluss

Die fünf IR-Liniensensoren erfassen den Kontrast zwischen Linie und Untergrund und liefern digitale Signale an den Mikrocontroller, der daraus die Position der Linie bestimmt und die Antriebssignale entsprechend anpasst. Der Ultraschallsensor im Deckel misst den Abstand zur Mülloberfläche und dient damit der Füllstandsermittlung; der vordere Ultraschallsensor überwacht den Fahrweg und verhindert Kollisionen mit Hindernissen.

Der Mikrocontroller verarbeitet diese Eingangssignale und steuert daraufhin die relevanten Ausgänge an. Dazu gehören der L298N-Motortreiber für das Fahrwerk, die RGB-Betriebs- und Kommunikations-LEDs für die visuelle Rückmeldung sowie der Buzzer für akustische Signale. Die grüne Starttaste und die rote Stopptaste sind direkt mit dem Mikrocontroller verbunden und ermöglichen dem Nutzer einen unmittelbaren Eingriff in den Betriebsablauf.

Mechanische Einbindung

Die elektrische Architektur ist eng mit dem mechanischen Aufbau verknüpft. Der umgebaute Mülltonnenkorpus enthält ein 3D-gedrucktes Innenelement, in dem Akku, L298N, XL4015, Raspberry Pi Pico 2 W, Sensorverkabelung und Bedienfeld kompakt untergebracht sind. An diesem Innenelement ist das Fahrwerk stabil verschraubt, sodass kurze Leitungswege und eine logische Bündelung der Komponenten erreicht werden. Dies ist insbesondere für den Prototypenbetrieb vorteilhaft, da Modifikationen und Wartungsarbeiten erleichtert werden.

In der Gesamtbetrachtung verdeutlicht das Blockschaltbild, dass das System modular aufgebaut ist und sich die Teilfunktionen – Energieversorgung, Steuerung, Aktorik, Sensorik und User Interface – eindeutig voneinander abgrenzen lassen. Diese Struktur erleichtert nicht nur die Dokumentation und die Fehlersuche, sondern schafft auch eine solide Grundlage für spätere Erweiterungen oder Anpassungen im Rahmen von Folgeprojekten.

4.3 Schaltplan und Pinbelegung

Der Schaltplan dokumentiert die tatsächlich umgesetzte elektrische Realisierung der smarten Mülltonne und konkretisiert damit das in Abschnitt 5.2 beschriebene Blockschaltbild. Er bildet die verbindliche Grundlage für Nachbau, Fehlersuche und Erweiterungen des Prototyps.

4.3.1 Vorläufiger Schaltplan (Fritzing-Entwurf)

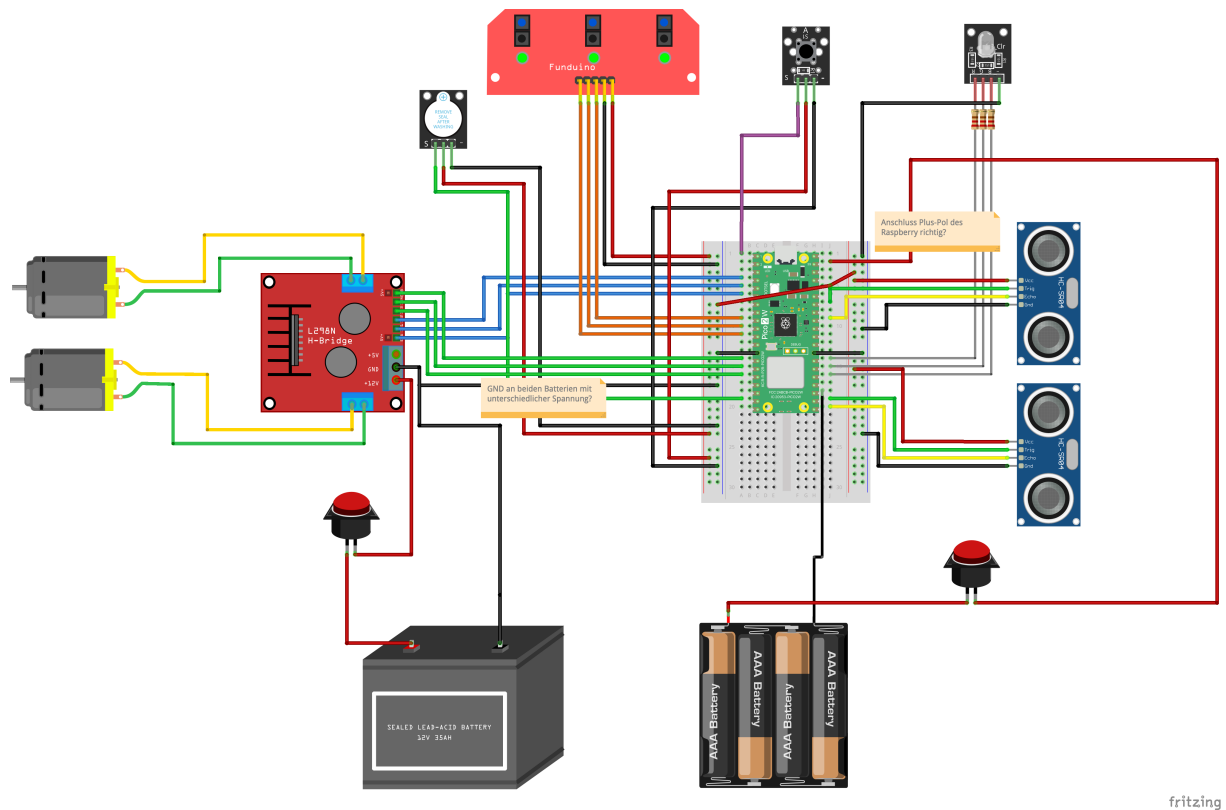
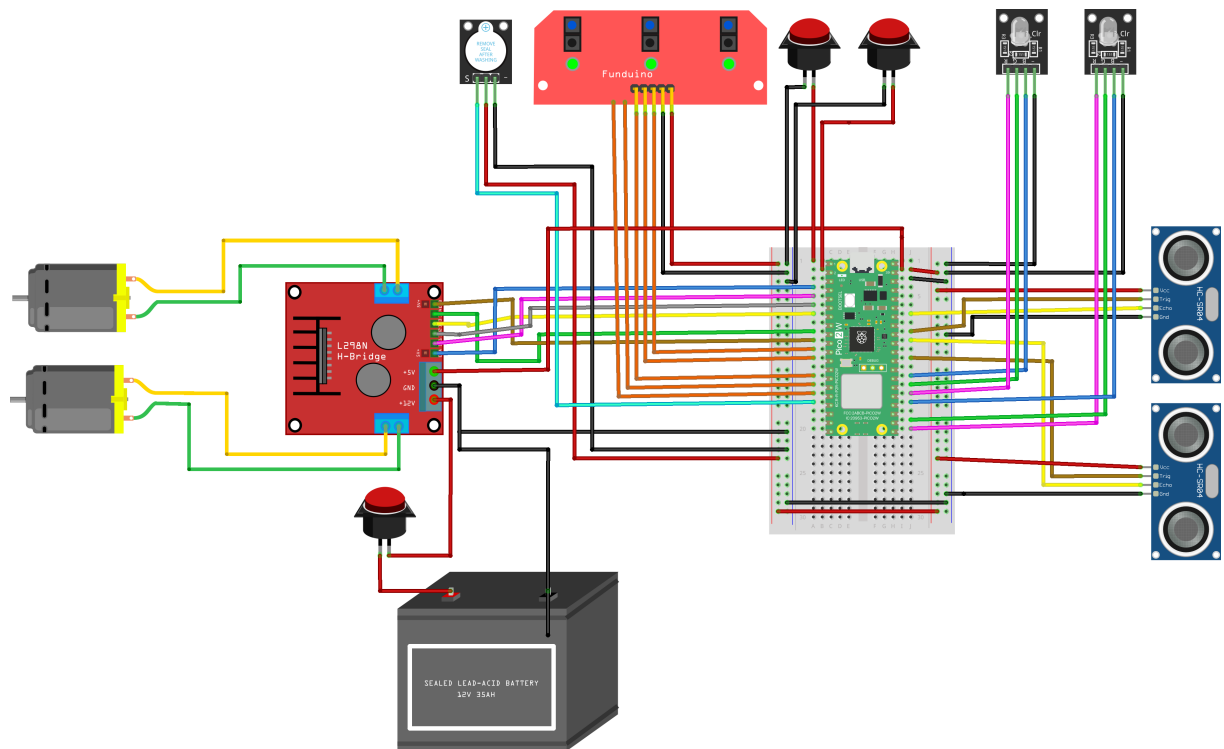


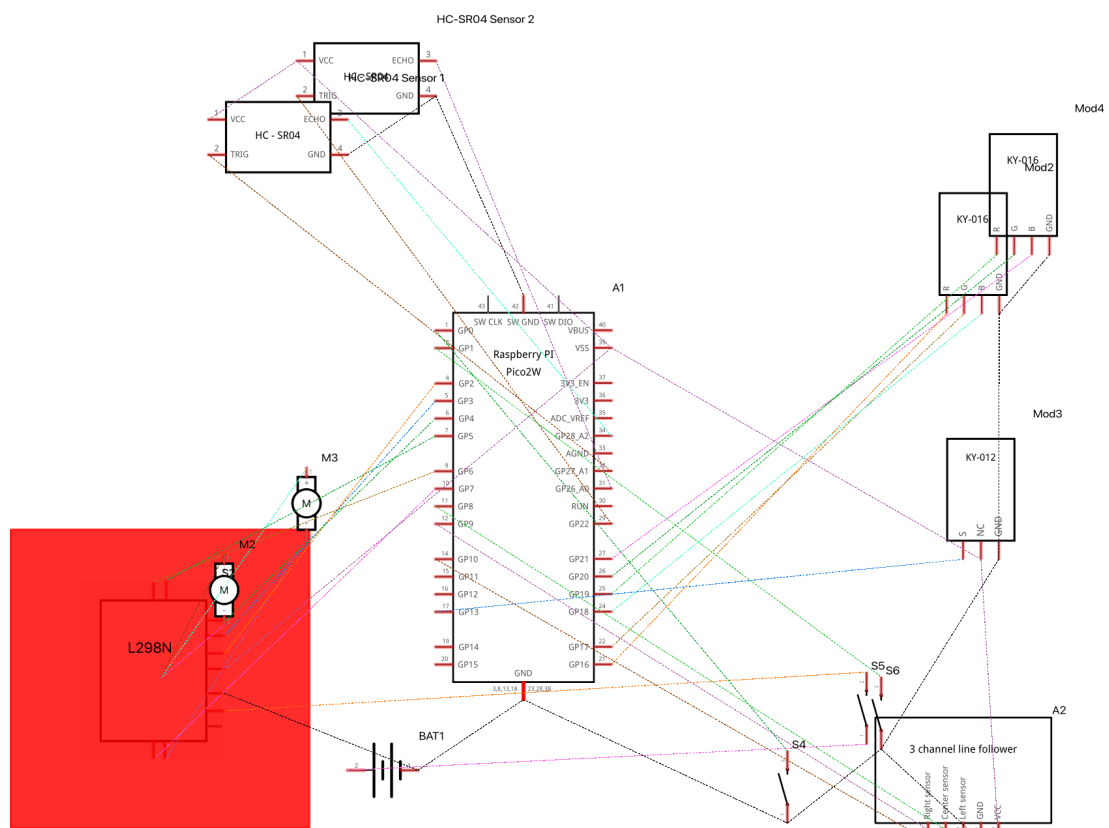
Abbildung 17: Vorläufiger Schaltplan (Fritzing-Entwurf der Elektronikarchitektur)

In dieser Abbildung ist der frühe, mit Fritzing erstellte Entwurf des elektrischen Aufbaus dargestellt. Der Plan visualisiert die konzeptionelle Anbindung von 12-V-Akku, L298N-Motortreiber, Raspberry Pi Pico 2 W, Ultraschallsensoren, Liniensensoren, Tastern, RGB-LED-Modulen und Buzzer. Er diente in der frühen Projektphase zur räumlichen und logischen Strukturierung der Komponenten sowie zur Dimensionierung der Steck- und Leitungsverbindungen. Einzelne Details (z. B. Pinzuordnungen) weichen teilweise vom finalen Stand ab, der Entwurf bleibt jedoch als Entwicklungsartefakt erhalten.

4.3.2 Finaler Schaltplan



fritzing



fritzing

Abbildung 18: Finaler Schaltplan des Bordnetzes mit L298N, Pico, Sensorik und UI

Der finale Schaltplan bildet ausschließlich das interne Bordnetz ab, beginnend an den 12-V-Akkuklemmen. Dargestellt sind die direkte 12-V-Versorgung des L298N-Motortreibers, der 5-V-Ausgang des L298N zur Versorgung des Raspberry Pi Pico 2 W, die interne 3,3-V-Versorgung des Pico für Sensorik und User Interface sowie sämtliche Signalverbindungen zwischen Mikrocontroller, Motortreiber, Sensoren und Bedienelementen.

Die Ladeelektronik (XL4015-Modul und externe DC-Buchse) ist im finalen Schaltplan bewusst nicht enthalten, da sie als separater Ladezweig außerhalb des eigentlichen Bordnetzes geführt wird. Für den autonomen Betrieb ohne angeschlossenes Netzteil ist der Schaltplan in Abbildung 18 damit die maßgebliche Referenz.

4.3.3 Verbindliche Pinbelegung des Raspberry Pi Pico 2 W

Die in Tabelle 9 gezeigte Verschaltung des Raspberry Pi Pico 2 W wird durch die folgende Tabelle präzisiert. Sie dokumentiert die final verwendete GPIO-Belegung und bildet die Schnittstelle zwischen Hardware und Client-Software.

GPIO	Funktion	Komponente	Signaltyp	Prioritätsstufe
GP0	Taster 1 (Start)	Pushbutton	Digital-Eingang	1 (Sicherheit)
GP1	Taster 2 (Pause)	Pushbutton	Digital-Eingang	1 (Sicherheit)
GP2	Motor ENA links	L298N	PWM	2 (Antrieb)
GP3	Motor IN1 (Vorwärts L)	L298N	Digital	2 (Antrieb)
GP4	Motor IN2 (Rückwärts L)	L298N	Digital	2 (Antrieb)
GP5	Motor IN3 (Vorwärts R)	L298N	Digital	2 (Antrieb)
GP6	Motor IN4 (Rückwärts R)	L298N	Digital	2 (Antrieb)
GP7	Motor ENB rechts	L298N	PWM	2 (Antrieb)
GP8	Liniensensor LA	KY-033	Digital	3 (Navigation)
GP9	Liniensensor LM	KY-033	Digital	3 (Navigation)
GP10	Liniensensor M	KY-033	Digital	3 (Navigation)
GP11	Liniensensor RM	KY-033	Digital	3 (Navigation)

GP12	Linienensor RA	KY-033	Digital	3 (Navigation)
GP13	Buzzer	Piezo-Modul	PWM	4 (Anzeige)
GP16	Server-LED Rot	RGB-LED	PWM	4 (Anzeige)
GP17	Server-LED Grün	RGB-LED	PWM	4 (Anzeige)
GP18	Server-LED Blau	RGB-LED	PWM	4 (Anzeige)
GP19	Betriebs-LED Rot	RGB-LED	PWM	4 (Anzeige)
GP20	Betriebs-LED Grün	RGB-LED	PWM	4 (Anzeige)
GP21	Betriebs-LED Blau	RGB-LED	PWM	4 (Anzeige)
GP22	Füllstand Trigger	HC-SR04P	Digital	3 (Messung)
GP26	Füllstand Echo	HC-SR04P	Digital	3 (Messung)
GP27	Hindernis Trigger	HC-SR04P	Digital	1 (Sicherheit)
GP28	Hindernis Echo	HC-SR04P	Digital	1 (Sicherheit)

Tabelle 9: Pinbelegungstabelle des Raspberry Pi Pico 2 W

Schichtentrennung zwischen 12-V-Leistungsebene, 5-V-Zwischenversorgung und 3,3-V-Logikebene ist im finalen Schaltplan klar erkennbar und bildet die elektrische Grundlage für den in den folgenden Kapiteln beschriebenen Betrieb.

Der Schaltplan stellt damit eine vollständige Grundlage für Nachbau, systematische Fehlersuche und planbare Erweiterungen des Prototyps dar.

4.4 Motoransteuerung

Die Motoransteuerung bildet die technische Grundlage für die autonome Bewegung der Mülltonne und realisiert einen Differenzialantrieb mit zwei 12-V-DC-Motoren, die über einen L298N-Motortreiber vom Raspberry Pi Pico 2 W angesteuert werden. Sie stellt sicher, dass sowohl die für die Linienverfolgung erforderlichen Geschwindigkeits- und Lenkkorrekturen als auch Reaktionen auf Hindernisse und Bedienbefehle zuverlässig umgesetzt werden.

Die Geschwindigkeitsregelung der Motoren erfolgt über Pulsweitenmodulation (PWM), während die Richtungsumschaltung durch die H-Brückentopologie des L298N realisiert wird. Die

vier digitalen Steuereingänge IN1–IN4 definieren die Drehrichtung der Motoren, während die Enable-Pins ENA und ENB PWM-Signale zur Vorgabe der Drehzahl erhalten. Detaillierte Parameter der PWM-Ansteuerung und deren softwareseitige Implementierung werden in den Softwarekapiteln beschrieben.

Im Rahmen der Inbetriebnahme zeigte sich, dass die Motoren erst ab einem bestimmten minimalen PWM-Füllgrad zuverlässig anlaufen. Dieser sogenannte Deadband-Bereich wurde in der Software berücksichtigt, indem sehr kleine PWM-Werte auf einen definierten Mindestwert angehoben werden. Zusätzlich wurden Asymmetrien zwischen linkem und rechtem Motor beobachtet, die durch einen einfachen Trimmfaktor softwareseitig kompensiert werden, um eine stabile Geradeausfahrt zu gewährleisten. Die zugehörigen Kalibrierungsparameter sind zentral hinterlegt und können bei Bedarf angepasst werden.

Für die Linienfolge kommt ein Regelungskonzept zum Einsatz, bei dem die aus den fünf KY-033-Liniensensoren abgeleitete Positionsabweichung in eine lenkungswirksame Stellgröße übersetzt wird. Diese Stellgröße beeinflusst die relative Ansteuerung der beiden Motoren und ermöglicht damit Kurvenfahrten und Korrekturen um die Ideallinie. Die detaillierte Ausgestaltung des Reglers sowie die Wahl und Abstimmung der Reglerparameter werden in den Softwarekapiteln zu Linienfolgeregulierung und Kalibrierung vertieft behandelt.

Bei der Hinderniserkennung wird die Motoransteuerung im Fehlerfall gezielt abgeschaltet. Erkennt der vordere Ultraschallsensor ein Hindernis innerhalb des definierten Sicherheitsabstands (Abstand unter 35 cm), werden beide Motoren unmittelbar über den L298N stillgesetzt. Parallel dazu werden optische und akustische Signale aktiviert, um den Wartezustand der Mülltonne für den Nutzer erkennbar zu machen. Die genaue Parametrierung der Abstandsschwellen und Reaktionslogik ist in Kapitel 6.4 beschrieben.

Insgesamt zeigt sich die Motoransteuerung im Prototypenbetrieb als stabil und hinreichend dynamisch für die geplanten Fahrmanöver im häuslichen Umfeld. Sie bildet damit die notwendige Grundlage für die in den folgenden Kapiteln beschriebenen Navigations- und Sicherheitsfunktionen und bietet zugleich ausreichend Freiheitsgrade für spätere Optimierungen des Regelverhaltens.

4.5 Energieversorgung

Die Energieversorgung der autonomen Mülltonne wurde speziell für den Haushaltseinsatz konzipiert, bei dem die Tonne alle zwei Wochen einmal zur Straße fährt und zurückkehrt. Sie berücksichtigt einen zuverlässigen Langzeit-Standby-Betrieb mit sporadischen Fahreinsätzen und gewährleistet dabei Wartungsfreundlichkeit sowie Reproduzierbarkeit des Systems.

Zur Energieversorgung kommt ein LONG Blei-Vlies-Akku WP5-12E (12 V, 5 Ah) zum Einsatz, dessen nominale Kapazität 60 Wh beträgt. Nach Abzug von Effizienzverlusten und Tiefentladeschutz bleiben etwa 50–55 Wh praktisch nutzbar, was einen vollständigen Abholzyklus sicher abdeckt.

Das Lastprofil des Systems unterscheidet sich deutlich je nach Betriebszustand. Im Standby-Betrieb (Pico im stromsparenden Modus mit periodischer Füllstandsmessung) liegt der Verbrauch im Bereich von 15–25 mA bzw. 0,18–0,3 W, was einem Tagesverbrauch von etwa 4,3–7,2 Wh entspricht. Eine vollständige Fahrt (hin und zurück, ca. 6 Minuten) verbraucht erfahrungsgemäß 4–6 Wh. Für einen typischen 2-Wochen-Zyklus ergeben sich somit 14 Tage Standby (ca. 45–50 Wh) zuzüglich eine Fahrt (ca. 5 Wh), was den Gesamtbedarf auf etwa 50–55 Wh begrenzt und innerhalb der verfügbaren Akkukapazität bleibt.

Die Versorgungsarchitektur trennt klar zwischen Leistungs- und Logikzweig. Vom 12-V-Akku führt die Leitung über den Wippschalter zum L298N-Motortreiber. Dieser versorgt einerseits die beiden 12-V-DC-Motoren direkt aus dem Bordnetz und stellt andererseits über seinen integrierten Spannungsregler eine 5-V-Versorgung für den Raspberry Pi Pico 2 W bereit. Der Pico erzeugt daraus intern die 3,3-V-Logikspannung, mit der HC-SR04P-Ultraschallsensoren, KY-033-Infrarotsensoren, RGB-LED-Module und der Buzzer betrieben werden. Der Ultraschallsensor HC-SR04P ist vollständig 3,3-V-kompatibel und erfordert daher keine zusätzlichen Pegelwandler.

Das Laden erfolgt über eine DC-Buchse (5,5 × 2,1 mm), an die ein 90-W-Netzteil (19 V, 4,74 A) angeschlossen wird. Bei einem effektiven Ladestrom von 1–2 A beträgt die Ladezeit etwa 4–6 Stunden bis zur vollständigen Ladung. Die Standardprozedur sieht vor, den Wippschalter vor dem Anschließen des Netzteils in die AN-Position zu bringen. Der XL4015-Step-Down-Wandler ist permanent am Akku angeschlossen und regelt als CC/CV-Laderegler den Ladevorgang (14,8 V Ladeschlussspannung bei 1 A Ladestrom) zuverlässig, ohne in den normalen Bordnetzbetrieb einzugreifen.

Zur elektrischen Sicherheit sind ein zentraler Wippschalter, der integrierte Übertemperaturschutz des Akkus, die Kurzschlussfestigkeit des L298N-Treibers sowie der mechanische Schutz der DC-Buchse vorgesehen. Der Wartungsplan umfasst das Laden alle zwei Wochen nach der Abholfahrt, die quartalsweise Kontrolle der Anschlüsse sowie einen jährlichen Kapazitätstest durch vollständige Entladung und Aufladung.

Die Energieversorgung erfüllt die Anforderungen des Haushaltseinsatzes vollständig. Ein 2-Wochen-Betriebszyklus bleibt innerhalb der Akkukapazität, während die pragmatische Konstruktion eine hohe Reproduzierbarkeit gewährleistet. Für den produktiven Serieneinsatz sind ein LiFePO₄-Akku zur Gewichtsreduktion, eine Spannungsüberwachung über einen

ADC-Eingang sowie eine weitergehende Deep-Sleep-Optimierung zur zusätzlichen Reduktion des Standby-Verbrauchs vorgesehen.

Die dokumentierte Energielösung bildet eine belastbare Grundlage für den sicheren Prototypenbetrieb sowie planbare Erweiterungen des Systems.

4.6 Fachliche Probleme, Analyse und Lösungen

Im Verlauf der Elektronikentwicklung wurden mehrere technische Herausforderungen identifiziert, analysiert und dokumentiert. Die nachfolgende strukturierte Problemdokumentation folgt dem Schema Problemstellung – Ursachenanalyse – Lösungsansatz – Ergebnisbewertung und stellt eine nachvollziehbare Wissensbasis für Folgeentwicklungen dar.

1. Asymmetrie der Motorleistung

- **Problemstellung:** Der linke DC-Motor (FIT0492-B) weist eine geringere Leistungsfähigkeit als der rechte Motor auf, was zu Richtungsabweichungen beim Geradeausfahren führt.
- **Ursachenanalyse:** Fertigungstoleranzen der Motoren, unterschiedliche Reibungskräfte im Kettenantrieb sowie mechanische Verspannungen in der Motoraufhängung.
- **Lösungsansatz:** Softwarebasierter Kompensationsfaktor (Trimming) mit 1,1 für den linken Motor – die PWM-Ansteuerung des linken Motors erhöht sich um 10% gegenüber dem rechten Motor bei identischen Sollwerten.
- **Ergebnisbewertung:** Richtungsstabilität vollständig wiederhergestellt. Symmetrisches Regelverhalten der Linienfolgeregelung. Einfache Nachkalibrierung bei Bauteilwechseln möglich.

2. Untergrundabhängigkeit der Liniensensorik

- **Problemstellung:** KY-033-Infrarotsensoren zeigen optimale Ergebnisse nur auf weißem Untergrund mit schwarzer Linie; geringfügige Variationen der Untergrundreflexion führen zu erheblichen Detektionsfehlern.
- **Ursachenanalyse:** Fixe interne Vergleichsspannung, nicht anpassbar an unterschiedliche Untergrundreflexionseigenschaften.
- **Entwicklungsstand:** Offene Aufgabe – nicht vollständig lösbar mit aktueller Sensorik.
- **Bewertung:** Einsatz auf standardisierten weiß-schwarzen Untergründen möglich. Praxiseinsatz auf variablen Untergründen erfordert grundlegende Überarbeitung der Sensortechnologie (Priorität: hoch).

3. Mechanische Kontaktproblematik (Breadboard)

- **Problemstellung:** Sporadische Signalunterbrechungen bei Jumperkabel-Verbindungen durch Fahrdynamik-bedingte Vibrationen.
- **Ursachenanalyse:** Unzureichende Kontaktkraft des Breadboards, mechanische Belastung, thermische Ausdehnung.
- **Lösungsansatz:** Verfestigung kritischer Verbindungen durch Lötverbindungen (Motoren, Akkuversorgung), mechanische Sicherung der Jumperkabel (Schrumpfschlauch, Klebeverbindungen).
- **Ergebnisbewertung:** Ausfälle an kritischen Signalwegen eliminiert. Breadboard-Lösung für Prototypen wartungsfreundlich, jedoch feuchtigkeitsempfindlich.

Restriktionen und Entwicklungsbedarf

Die identifizierten Problemlösungen verdeutlichen grundlegende Prototypen-Einschränkungen:

- Linienfolge-Untergrundabhängigkeit (Priorität: hoch): Grundlegende Überarbeitung der Sensorik erforderlich.
- Breadboard-Konzeption: Entwicklung dedizierter Leiterplatte zwingend für Langzeitbetrieb.

Methodische Erkenntnisse

Die Entwicklung demonstriert die Effektivität softwarebasierter Kompensation bei Fertigungsdispersionen sowie die Notwendigkeit mechanischer Optimierungen bei Prototypenlösungen. Die Untergrundabhängigkeit der optischen Sensorik erweist sich als fundamentale Designherausforderung, die eine prinzipielle Lösung über Kalibrierung hinaus erfordert.

Die dokumentierte Problemanalyse bildet eine transferierbare Wissensbasis, die systematische Weiterentwicklungen ermöglicht und wiederholte Problemlösungen vermeidet.

5 Sensorik und User Interface

5.1 Statusanzeige

Die Statusanzeige der autonomen Mülltonne erfolgt über zwei unabhängige RGB-LED-Module sowie ein Piezo-Buzzer-Modul, die eine eindeutige Zustandszuordnung relevanter Systemzustände ermöglichen. Das Konzept gewährleistet Redundanz (visuell + akustisch) und Tageslichtlesbarkeit bei gleichzeitiger Energieeffizienz.

Betriebszustands-LED dokumentiert lokale Systemparameter:

Zustand	Farbmuster	Bedeutung
Grün (konstant)	RGB(0,255,0)	Bereitschaft/Standby – Füllstandsmessung aktiv
Blau (pulsierend)	RGB(0,0,255)	Fahrt-/Bewegung aktiv
Gelb (konstant)	RGB(255,255,0)	Füllstand (>95%)
Gelb (pulsierend)	RGB(255,255,0)	Bereitstellungsposition erreicht, wartet Abholung
Grün (pulsierend)	RGB(0,255,0)	Erfolgreich entleert
Blau (konstant)	RGB(0,0,255)	Pausiert – manuelle Unterbrechung
Orange (Doppelblitz)	RGB(255,85,0)	Hindernisdetektion – Fahrt gestoppt

Tabelle 10: Visuelles Anzeigesystem der Betriebszustände

Kommunikationsstatus-LED (GP16/GP17/GP18) visualisiert Serverinteraktion:

Zustand	Farbmuster	Bedeutung
Grün (konstant)	RGB(0,255,0)	Bidirektionale Serververbindung aktiv

Blau (schnell blinkend)	RGB(0,0,255)	Verbindungsaufbau läuft
Lila (konstant)	RGB(127,0,127)	Offline-Modus (bewusste WLAN-Trennung während Fahrt)

Tabelle 11: Visuelles Anzeigesystem der Serverzustände

Der Piezo-Buzzer dient ausschließlich der akustischen Bestätigung sicherheitskritischer Ereignisse. Kommunikationszustände bleiben bewusst stumm. Keine Dauertöne werden verwendet.

Ereignis	Tonmuster	Dauer	Wiederholung
Hindernisdetektion	Doppelton	2 × 100 ms	Pro Doppelblitz
Zielposition erreicht	Einzelton mittellang	200 ms	Einmalig
Tasterbedienung	Einzelton kurz	100 ms	Einmalig
Fahrtbeginn	Einzelton lang	400 ms	Einmalig

Tabelle 12: Akustisches Anzeigesystem

Nachtruhe-Funktion: Softwaremäßig deaktivierbar (geplante Erweiterung).

Technische Realisierung

LED-Ansteuerung: Gemeinsame Anodenversorgung (3,3 V), individuelle PWM-Kanäle pro Farbe. Helligkeitsregelung PWM-Füllgrad 0-100% für Tageslichtadaption.

Buzzer-Ansteuerung: PWM-fähige Ausgangsstufe, Frequenzbereich 1-4 kHz, maximale Lautstärke 75 dB(A) @ 30 cm (Nachbarschaftsschutz).

Funktionale Redundanz und Priorisierung

Dual-LED-System trennt lokale Systemzustände (Betriebs-LED) von Kommunikationszuständen (Server-LED), wodurch Fehlerquellen (z. B. Serverausfall) eindeutig identifizierbar bleiben.

Akustische Ergänzung beschränkt sich auf unübersehbare Ereignisse:

- Bei Hinderniserkennung wiederholen sich die Töne, was zur Beachtung und Handlung leitet.

-
- Bestätigungen sind nur einmalig, was Störungen vermeidet.

Validierung und Normkonformität

Das Anzeigesystem entspricht IEC 60204-1 (Maschinensicherheit – elektrische Ausrüstung) und DIN EN ISO 3864 (Sicherheitsfarben):

- Farbanzeige international standardisiert
- Blinkfrequenzen für Dringlichkeitsstufung normkonform
- Akustikpegel umgebungsgerecht dimensioniert

Das vollständig dokumentierte Statusanzeigesystem gewährleistet parametrische Nachvollziehbarkeit, redundante Informationsvermittlung und operationelle Robustheit für den sicheren Haushaltseinsatz.

5.2 Füllstandssensorik

Zur Messung des Füllstands wird ein Ultraschallsensor vom Typ HC-SR04P eingesetzt, der im Deckel der Mülltonne senkrecht nach unten ausgerichtet ist. Eine Messung wird durch einen 10 µs langen High-Impuls am Trigger-Pin ausgelöst, woraufhin der Sensor einen Ultraschallimpuls aussendet und auf das zurückgeworfene Echo wartet. Die Laufzeit des Echos wird über Interrupts nicht-blockierend erfasst: Ein Interrupt registriert die steigende Flanke des Echo-Pins als Startzeitpunkt, ein zweiter die fallende Flanke als Endzeitpunkt. Aus der Zeitdifferenz berechnet die Software den Abstand in Zentimetern nach der Formel $\text{Abstand} = \text{Pulsdauer}_{\mu\text{s}} / 58,0$, die sich aus der Schallgeschwindigkeit von ca. 343 m/s und dem doppelten Laufweg (hin und zurück) ergibt. Messungen werden standardmäßig alle 250 ms ausgelöst; bleibt das Echo aus, wird die Messung nach einem konfigurierbaren Timeout als ungültig gewertet.

Der gemessene Rohwert in Zentimetern wird anschließend in einen prozentualen Füllstand umgerechnet. Grundlage ist eine lineare Interpolation zwischen zwei kalibrierten Referenzwerten: dem Abstand bei leerer Tonne (Sensor bis Tonnenboden) und dem Abstand bei voller Tonne (Sensor bis Müll). Ein gemessener Abstand unterhalb des Vollswellenwerts ergibt 100 %, ein Abstand oberhalb des Leerswellenwerts ergibt 0 %. Liegt der gemessene Abstand deutlich oberhalb des kalibrierten Leerabstands oder kommen wiederholt keine Echos zurück, wertet die Software den Zustand als „Deckel offen“ und unterdrückt die Füllstandsmeldung bis zur nächsten gültigen Messung.

5.3 Hinderniserkennung

Für die Hinderniserkennung wird ein zweiter HC-SR04P Ultraschallsensor eingesetzt, der an der Vorderseite der Mülltonne horizontal in Fahrtrichtung ausgerichtet ist. Das Messprinzip und der Messablauf sind identisch mit der Füllstandssensorik: Ein Trigger-Impuls löst die Schallaussendung aus, die Echopulsdauer wird per Interrupt erfasst und in einen Abstandswert in Zentimetern umgerechnet. Die Software wertet den gemessenen Abstand zyklisch aus und vergleicht ihn mit dem konfigurierten Stoppschwellenwert von 35 cm. Unterschreitet der Abstand diesen Schwellenwert, werden die Motoren sofort gestoppt, und das System verharrt in diesem Zustand, bis der Sensor keinen Gegenstand mehr vor der Mülltonne meldet. Erst dann wird die Fahrt automatisch fortgesetzt.

5.4 Linienerkennung

Die Linienerkennung basiert auf fünf digitalen Infrarot-Reflexionssensoren vom Typ KY-033, die in einer horizontalen Reihe quer zur Fahrtrichtung am Unterboden der Mülltonne montiert sind. Jeder Sensor gibt einen binären Wert aus: 1, wenn er eine dunkle Linie auf hellem Untergrund erkennt, und 0, wenn er hellen Untergrund detektiert. Um kurze Störimpulse zu unterdrücken, wird jeder Sensorpin mehrfach hintereinander abgetastet; der häufigste Wert (Mehrheitsentscheid) gilt als gültiges Ergebnis. Die fünf Einzelwerte werden zu einem 5-Bit-Muster zusammengefasst, wobei Bit 4 dem äußersten linken Sensor und Bit 0 dem äußersten rechten Sensor entspricht.

Der Positionswert der Linie ergibt sich aus einer gewichteten Summe der aktiven Sensoren, dividiert durch deren Anzahl. Die Gewichte sind asymmetrisch kalibriert: Der linke Mitte-Sensor ist mit dem Faktor 1,5 stärker gewichtet als der rechte, was eine empirisch angepasste Feinjustierung des Lenkverhaltens darstellt.

5.5 Fachliche Probleme, Analyse und Lösungen

Im Verlauf der Sensordefinition und UI-Entwicklung wurden mehrere technische Herausforderungen identifiziert, analysiert und dokumentiert. Die nachfolgende strukturierte Problemdokumentation folgt dem Schema Problemstellung – Ursachenanalyse – Lösungsansatz – Ergebnisbewertung.

LED-Sichtbarkeit bei Tageslicht

Problemstellung: Die RGB-LEDs sind bei direkter Sonneneinstrahlung schwer lesbar, was die Nutzerinteraktion einschränkt.

Ursachenanalyse: Maximale PWM-Helligkeit (100%) reicht nicht für Außenbedingungen.

Lösungsansatz:

- 3D-gedruckte durchsichtige Abdeckung mit optimierter Geometrie (Lichtkanäle, Anti-reflex).
- Mechanische Erhöhung der LED-Position.

Ergebnisbewertung: Gelöst – Sichtbarkeit deutlich verbessert, auch bei diffusem Tageslicht.

Buzzer-Nachtruhe-Konflikt

Problemstellung: Akustische Signale (z. B. Hindernis-Doppelton) können Nachbarn bei Nachtbetrieb stören.

Ursachenanalyse: Fehlende zeitbasierte Signalsteuerung.

Lösungsansatz:

- Software-Schalter über Server-Kommando (QUIETMODE ON/OFF).
- Lokaler Timer-Fallback (22-6 Uhr stumm).

Ergebnisbewertung: Offene Aufgabe (Priorität: mittel). Basis-Implementierung vorhanden.

Restriktionen und Entwicklungsbedarf

Die identifizierten Problemlösungen verdeutlichen grundlegende Prototypen-Einschränkungen:

Nachtruhe-Buzzer (Priorität: mittel): Server-Integration abschließen

Methodische Erkenntnisse

3D-Druck-Lösungen (Abdeckung) sind kostengünstig, schnell iterierbar und effektiv für UI-Optimierungen. Software-basierte Zeitsteuerung ist skalierbar, aber Server-abhängig.

Die dokumentierte Problemanalyse bildet eine transferierbare Wissensbasis für systematische Weiterentwicklungen.

6 Architektur der Software

6.1 Systemübersicht und Komponenten

Das Gesamtsystem der smarten Mülltonne besteht aus einem Zusammenspiel von mechanischen, elektronischen und softwaretechnischen Subsystemen. Die elektronische Steuerung übernimmt ein Raspberry Pi Pico 2 W Mikrocontroller, der über einen L298N-Motortreiber zwei DC-Motoren ansteuert. Zur Umgebungserfassung nutzt das System fünf digitale Infrarot-Sensoren (KY-033) zur Linienverfolgung sowie HC-SR04P Ultraschallsensoren für die Füllstandsmessung und die frontale Hinderniserkennung. Die direkte Interaktion mit dem Nutzer erfolgt über ein lokales Bedienfeld, das RGB-LEDs für visuelles Feedback, einen Buzzer für akustische Signale sowie Start- und Stopp-Taster umfasst. Die Softwarearchitektur ist zweigeteilt: Auf dem Mikrocontroller läuft MicroPython für die hardwarenahe Echtzeitsteuerung, während ein Python-basierter Server auf einem macOS-System die übergeordnete Missionslogik bereitstellt.

Subsystem	Hauptkomponenten
Mechanik	50-Liter-Mülltonne, 3D-gedruckter Montagerahmen, Raupenfahrwerk
Elektronik	Raspberry Pi Pico 2 W, L298N-Motortreiber, 2× DC-Motor
Sensorik	5× KY-033 Infrarot-Liniensensor, 2× HC-SR04P Ultraschallsensor
User Interface	RGB-LED, Buzzer, Start-Taster (grün), Stopp-Taster (rot)
Software	MicroPython (Client auf Pico), Python 3 auf macOS (Server)

Tabelle 13: Komponentenübersicht nach Subsystemschicht

6.2 Softwareziele

Die Client-Software des Smarten Mülltonnen-Roboters wurde nach drei übergeordneten Qualitätszielen entwickelt: Modularität, Erweiterbarkeit und Wartbarkeit. Diese Ziele bilden die architektonische Grundlage, auf der alle weiteren Entwurfsentscheidungen basieren.

Modularität:

Die gesamte Steuerungslogik ist in funktional eigenständige Module aufgeteilt. Jede

Hardwarekomponente – von der Motorsteuerung (`dcmotor.py`) über den Liniensensor (`liniensensor.py`) bis hin zum Ultraschallsensor (`ultraschallsensor.py`) – wird durch eine dedizierte Klasse abstrahiert. Die zentrale Zustandsmaschine (`global_controller.py`) koordiniert diese Module ausschließlich über deren öffentliche Schnittstellen, ohne Kenntnis der internen Implementierungsdetails.

Erweiterbarkeit:

Die klare Modulstruktur ermöglicht es, einzelne Komponenten unabhängig voneinander auszutauschen oder zu ergänzen. Soll beispielsweise ein neuer Sensortyp integriert werden, genügt es, eine neue Treiberklasse hinzuzufügen und in `global_controller.py` einzubinden – ohne Änderungen an anderen Modulen. Ebenso lassen sich neue Zustände in der Zustandsmaschine ergänzen, ohne das bestehende Verhalten zu beeinträchtigen.

Wartbarkeit:

Alle Betriebsparameter – darunter Fahrgeschwindigkeit, PD-Koeffizienten, Motor-Trim-Faktoren und die Netzwerkadresse des Servers – sind zentral in `main.py` abgelegt. Dieses Prinzip der Single Source of Truth reduziert den Aufwand für Rekalibrierungen erheblich und erleichtert die Einarbeitung für Folgegruppen, da alle relevanten Stellgrößen an einem einzigen, gut dokumentierten Ort zu finden sind.

6.3 Dateistruktur und Modulübersicht

Die Client-Software besteht aus 11 MicroPython-Modulen, die gemäß dem Prinzip der Einzelverantwortung organisiert sind. Tabelle 14 gibt einen Überblick über Dateinamen und die jeweilige Hauptverantwortlichkeit.

Datei	Funktion / Verantwortlichkeit
<code>main.py</code>	Einstiegspunkt; definiert alle Betriebsparameter (Geschwindigkeit, PD-Koeffizienten, Netzwerkadresse) und startet den <code>GlobalController</code> .
<code>global_controller.py</code>	Zentrale Zustandsmaschine; koordiniert alle Subsysteme und implementiert die 7 Client-Zustände sowie deren Übergangsbedingungen.
<code>PDcontroller.py</code>	PD-Regler für die Linienverfolgung; berechnet den Lenkkorrekturwert aus Sensorfehler und Fehlerableitung.
<code>dcmotor.py</code>	Abstraktion der DC-Motorsteuerung über PWM; berücksichtigt Trim-Faktoren und Anlaufschwellen.

<code>liniensensor.py</code>	Auswertung der fünf KY-033-IR-Sensoren; berechnet einen gewichteten Positionsfehler für den PD-Regler.
<code>ultra-schallsensor.py</code>	Interrupt-basierte Distanzmessung mit HC-SR04P; enthält <code>FuellstandSensor</code> -Wrapper für die Füllstandserkennung.
<code>button.py</code>	Entprelllogik mit Timestamp-basierter Differenzierung zwischen Kurz- und Langdruck.
<code>led.py</code>	<code>LEDStateMachine</code> und <code>ConnectionLEDStateMachine</code> für zustandsabhängige visuelle Rückmeldung über RGB-LEDs.
<code>buzzer.py</code>	Nicht-blockierende Buzzer-Muster für akustische Signalisierung von Systemzuständen.
<code>network_manager.py</code>	WLAN-Verbindungsaufbau und TCP-Client-Zustandsmaschine.
<code>network_controller.py</code>	Nicht-blockierende TCP-Kommunikation mit <code>select.poll()</code> ; verarbeitet eingehende Serverbefehle ohne Blockierung der Hauptschleife.

Tabelle 14: Modulübersicht der Client-Software auf dem Raspberry Pi Pico 2 W

6.4 Architektur Client–Server

Die Systemarchitektur basiert auf einer asynchronen, stark entkoppelten Aufgabenteilung zwischen der lokalen Echtzeitsteuerung (Client) und der übergeordneten Koordinationslogik (Server). Durch dieses Design wird sichergestellt, dass rechenintensive Netzwerkprozesse die zeitkritische Hardwaresteuerung nicht blockieren.

Blockschaltbild: Client-Server-Architektur – Smarte Mülltonne

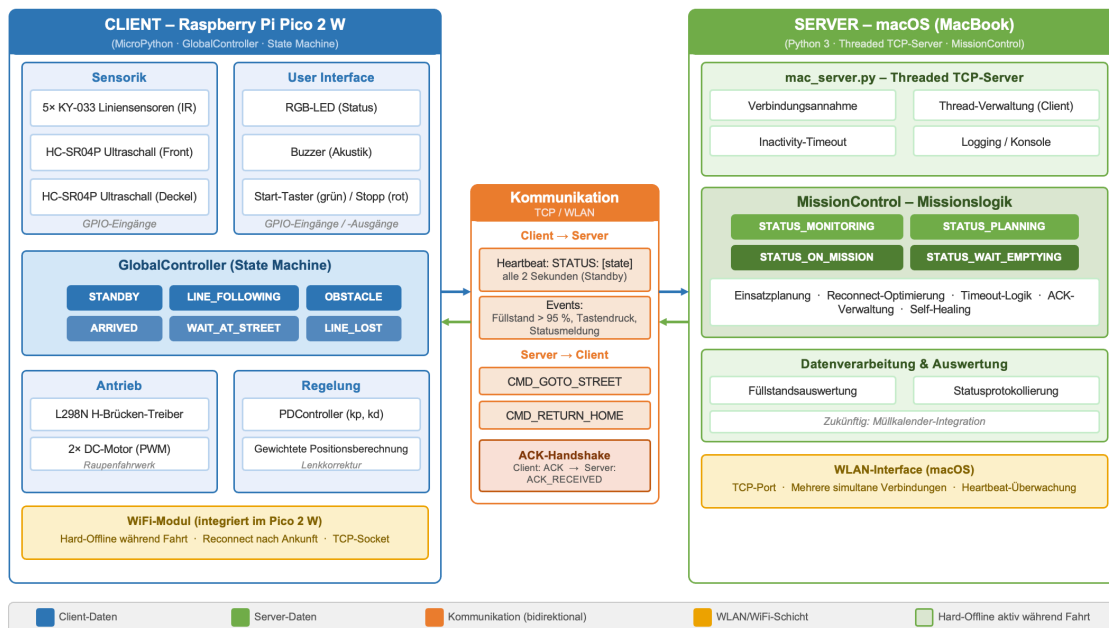


Abbildung 19: Blockschaltbild der Client-Server-Architektur und Datenflüsse

Der Raspberry Pi Pico 2 W agiert als Client und übernimmt ausschließlich hardwarenahe Aufgaben. Dazu gehören das Auslesen der Ultraschall- und Liniensensoren, die Ausführung des PD-Reglers für die Linienverfolgung sowie die direkte Ansteuerung des L298N-Motortreibers und des User-Interfaces (LEDs, Buzzer, Taster). Um eine ruckelfreie Fahrt zu gewährleisten, wechselt der Client während der autonomen Navigation in einen dezidierten Hard-Offline-Modus, in dem die WLAN-Kommunikation für die Dauer der Fahrt gezielt deaktiviert wird, um sämtliche CPU-Ressourcen vollständig für die Echtzeitregelung (Motor- und Sensor-Timing) zu reservieren.

Der Python-basierte Server (macOS) übernimmt die übergeordnete Koordinationslogik des Gesamtsystems. Er verwaltet die angeschlossenen Mülltonnen in separaten Threads, überwacht Verbindungszeiten durch konfigurierbare Inactivity-Timeouts und ist für die Einsatzplanung zuständig. Perspektivisch ist die Integration eines digitalen Müllkalenders vorgesehen, durch die Fahrten zur Straße ausschließlich an relevanten Abholtagen initiiert werden.

Der Datenfluss zwischen den Systemen erfolgt im Standby über eine TCP-Socket-Verbindung. Der Client sendet periodisch alle zwei Sekunden einen Heartbeat (`STATUS: [State]`) an den Server. Registriert die Sensorik relevante Änderungen – etwa einen Füllstand von über 95 % für mehr als fünf Sekunden (Entprellungslogik) oder einen Tastendruck, sendet der Client ein Event. Der Server wertet dies aus und antwortet bei Bedarf mit asynchronen Steuerkommandos wie `CMD_GOTO_STREET` oder `CMD_RETURN_HOME`. Vor Ausführung eines Fahrbefehls findet ein Handshake statt: Der Client bestätigt den Befehl mit einem `ACK`, wartet auf das `ACK_RECEIVED` des Servers, trennt die Netzwerkverbindung kontrolliert und startet die Bewegung.

während des Verbindungsaufbaus beim Start (Hard-Offline), kappt der Client die Serverabhängigkeit. In diesem Betriebsfall kann die smarte Mülltonne weiterhin vollständig lokal über die physischen Taster bedient werden und ihre Fahrfunktionen autark ausführen. Dabei wird durch Betätigung des grünen Tasters der Startbefehl zum Losfahren manuell ausgelöst. Der Füllstand der Mülltonne wird in diesem Zustand zwar noch visuell angezeigt, kann jedoch keinen Start mehr initiieren. Kann der Client im Offlinemodus wieder eine aktive Verbindung zum WLAN und zum Server aufbauen, wechselt das System automatisch in den Online-Modus zurück und empfängt seine Steuerbefehle fortan wieder vom Server.

7 Software: Client

7.1 Client-Zustandsmaschine

Die Zustandsmaschine des Clients ist in der Klasse `GlobalController` implementiert und definiert 7 diskrete Betriebszustände. Die Übergänge zwischen diesen Zuständen werden ausschließlich durch Sensorsignale, Benutzereingaben oder Serverbefehle ausgelöst, niemals durch zeitbasierte Abbruchbedingungen allein.

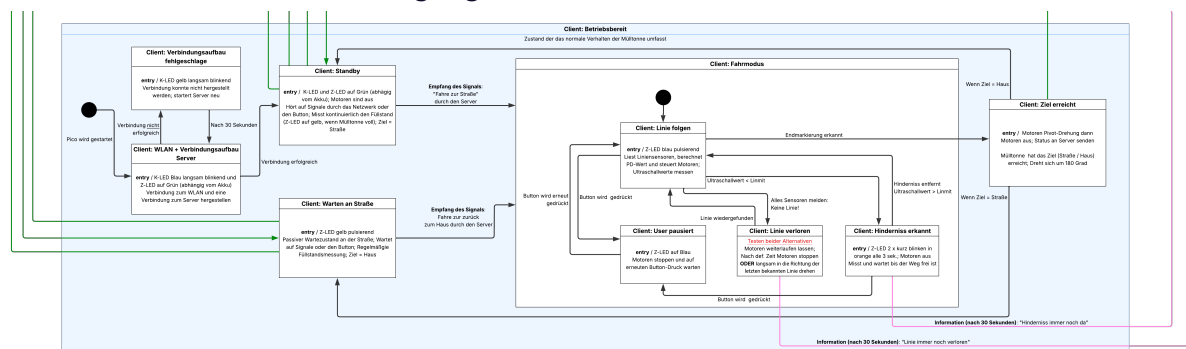


Abbildung 21: State-Machine des Clients

Im Zustand **STANDBY** befindet sich der Roboter im Ruhemodus an seinem Heimatstandort. In diesem Zustand misst der Füllstandssensor periodisch den aktuellen Füllstand der Tonne und übermittelt die Ergebnisse als Heartbeat-Statusmeldung an den Server. Gleichzeitig wartet das System auf eine Benutzereingabe über die Taster: Durch einen langen Tastendruck auf den grünen Starttaster kann der Nutzer einen manuellen Fahrauftrag zur Straße auslösen – unabhängig davon, ob die Tonne tatsächlich voll ist. In diesem Fall wird die Information, dass der Taster gedrückt wird (`MANUAL_GOTO_STREET_REQUEST`) an den Server übermittelt. Erreicht der Füllstand den konfigurierten Schwellwert, wird automatisch der Status `FULL` an den Server gesendet, der daraufhin die Einsatzplanung einleitet. In beiden Fällen erfolgt der Übergang in den Fahrzustand erst, nachdem der Server den Befehl `CMD_GOTO_STREET` gesendet hat und der dreistufige ACK-Handshake vollständig abgeschlossen wurde: Der Client sendet `ACK CMD_GOTO_STREET`, der Server bestätigt mit `ACK_RECEIVED CMD_GOTO_STREET`, und anschließend trennt der Client die WLAN-Verbindung gezielt, um alle CPU-Ressourcen für die Echtzeitregelung freizugeben (Hard-Offline-Modus). Erst danach beginnt die autonome Fahrt im Zustand `LINE_FOLLOWING`.

Im Zustand **LINE_FOLLOWING** wertet der PD-Regler im 20-ms-Zyklus kontinuierlich die fünf IR-Sensoren aus und steuert die Motoren entsprechend an. Da die WLAN-Verbindung während der Fahrt deaktiviert ist, arbeitet der Client vollständig autonom. Der frontale

Ultraschallsensor prüft parallel auf Hindernisse. Über den roten Stopptaster kann der Nutzer jederzeit in den Zustand `USER_PAUSED` wechseln.

Verlieren alle fünf IR-Sensoren das Liniensignal, wechselt das System unmittelbar in den Zustand `LINE_LOST`. Innerhalb von `LINE_LOST` durchläuft das System eine mehrstufige Suchlogik: Solange weniger als 12 aufeinanderfolgende Messzyklen ohne Liniensignal vorliegen oder das zugehörige Zeitfenster von 800 Millisekunden noch nicht abgelaufen ist, fährt der Roboter mit der Basisgeschwindigkeit geradeaus weiter, in der Hoffnung, die Linie im unmittelbaren Umfeld wiederzufinden. Erst nach Ablauf beider Bedingungen gleichzeitig aktiviert das System einen Fallback-Modus, in dem die zuletzt bekannte Linienposition als Eingangsgröße für den PD-Regler verwendet wird. Wird die Linie zu einem beliebigen Zeitpunkt während der Suchphase wieder erkannt, kehrt das System sofort in den Zustand `LINE_FOLLOWING` zurück. Bleibt die Suche über den gesamten Timeout von 10.000 Millisekunden erfolglos, greift ein Not-Aus: Die Motoren werden sicherheitsgerichtet gestoppt und das System wechselt in den Zustand `STANDBY`.

Erkennt der frontale Ultraschallsensor ein Hindernis in einem Abstand kleiner als 35 cm, wird der Zustand `OBSTACLE` aktiviert: Die Motoren werden sofort abgeschaltet und der Roboter wartet, bis das Hindernis entfernt wird. Der Client misst kontinuierlich weiter und kehrt erst nach 30 hindernisfreien Messzyklen automatisch in den Zustand `LINE_FOLLOWING` zurück.

Den erfolgreichen Abschluss eines Fahrauftrags signalisiert der Zustand `ARRIVED`. Dieser wird ausgelöst, sobald die Liniensensoren eine Querlinie zur Fahrlinie detektieren (alle 5 Sensoren melden gleichzeitig, dass sie die Linie sehen). Um nach der Entleerung wieder zurück fahren zu können, führt die Mülltonne in diesem Zustand zudem eine Drehung um die eigene Achse aus. Nach dem Eintritt in `ARRIVED` stellt der Client die WLAN-Verbindung zum Server automatisch wieder her und übermittelt die Ankunftsmeldung (`ARRIVED: STREET` oder `ARRIVED: HOME`). Je nach Zielort erfolgt anschließend der Übergang in `WAIT_AT_STREET` bzw. zurück in `STANDBY`.

Im Zustand `WAIT_AT_STREET` befindet sich die Tonne an der Straße und wartet auf die Leerung. Der Füllstandssensor überwacht, ob die Tonne entleert wurde, und sendet in diesem Fall den Status `EMPTIED` an den Server. Analog zum manuellen Start in `STANDBY` kann der Nutzer auch hier durch einen langen Tastendruck auf den grünen Taster einen manuellen Rückfahrauftrag auslösen – unabhängig davon, ob die Tonne tatsächlich entleert wurde. Die Rückfahrt folgt demselben Ablauf wie die Hinfahrt: Der Server sendet `CMD_RETURN_HOME`, der ACK-Handshake wird durchlaufen, die WLAN-Verbindung wird getrennt, und die autonome Fahrt beginnt im Zustand `LINE_FOLLOWING`.

Über den roten Stopptaster kann ein Nutzer jederzeit den Zustand **USER_PAUSED** aktivieren, der alle Fahrfunktionen sofort unterbricht. Das System verbleibt in diesem Zustand, bis ein erneuter Tastendruck den Normalbetrieb wiederherstellt. Zudem wird erneut eine Verbindung zum Server hergestellt, um diesen über die Fahrtunterbrechung zu informieren.

7.2 Linienfolgeregung

```
16      # Gewichtung der Sensoren für die Positionsrechnung.
17      # Negativ = Links, Positiv = Rechts, 0 = Mitte.
18      # Wertebereich ca. -100 bis +100.
19      _W_LA = -100 # Links Außen
20      _W_LM = -50  # Links Mitte
21      _W_M  = 0    # Mitte
22      _W_RM = 50   # Rechts Mitte
23      _W_RA = 100  # Rechts Außen
24
```

Abbildung 22: Gewichtung der Liniensensoren im Quellcode

Die Linienverfolgung des Roboters kombiniert die gewichteten Sensordaten mit einem PD-Regler, um weiche und präzise Lenkbewegungen zu erzielen.

Sensorik und Fehlerberechnung:

Die Linienverfolgung basiert auf einem in der Klasse **PDcontroller** implementierten Regler, der aus dem aktuellen Linienfehler eine Geschwindigkeitskorrektur für beide Motoren ermittelt. Für die Fehlerberechnung wertet die Klasse **Liniensensor** die fünf IR-Sensoren gewichtet von -100 (ganz links) über 0 (Mitte) bis +100 (ganz rechts) aus. Der resultierende Fehlerwert e entspricht dem normierten Durchschnitt der aktiven Sensoren und beschreibt den genauen Versatz zur Ideallinie.

Der Proportional-Anteil:

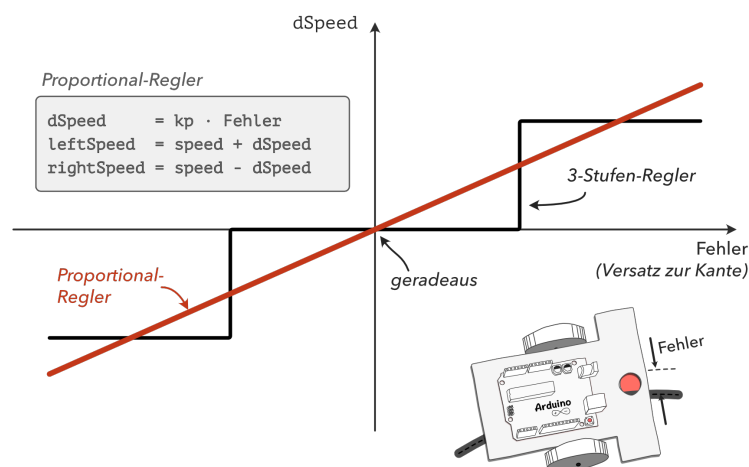


Abbildung 23: Veranschaulichung der Funktionsweise des P-Reglers

Wie in Abbildung 23 zu sehen, erzeugt der Proportional-Regler (P-Anteil) im Gegensatz zu einem starr schaltenden 3-Stufen-Regler eine stufenlose Lenkbewegung, die linear zur Regelabweichung ansteigt. Diese proportionale Reaktion sorgt zwar für eine kontinuierliche Nachführung, führt jedoch durch die Massenträgheit des Roboters bei höheren Geschwindigkeiten oft zu Oszillationen. Der Roboter neigt in diesem Fall dazu, über die Ideallinie hinauszuschießen und an der Linie entlangzuwackeln.

Berechnung der Stellgröße:

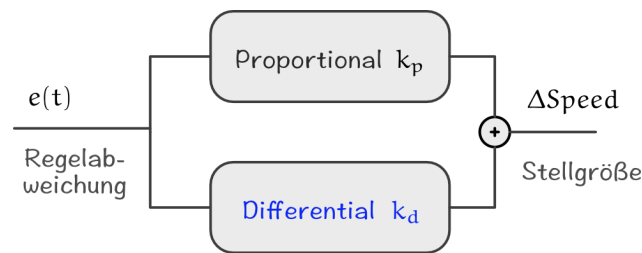


Abbildung 24: Berechnung der Stellgröße beim PD-Regler

Um das Wackeln zu verhindern, spaltet sich die Berechnung der Regelabweichung $e(t)$ beim PD-Regler, wie in Abbildung 24 zu sehen, in zwei parallele Stränge auf. Der Fehler wird proportional mit k_p und nach seiner zeitlichen Ableitung differentiell mit k_d gewichtet. Die Summe beider Pfade bildet die finale Stellgröße, welche die Basisgeschwindigkeit der linken und rechten Motoren entsprechend anpasst.

Wirkung des D-Anteils:

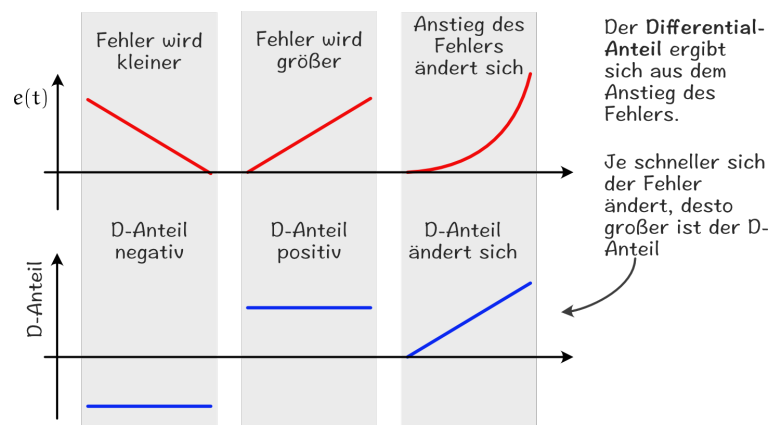


Abbildung 25: Visualisierung des Unterschieds des P- und D-Anteils beim PD-Regler

Wie in Abbildung 25 zu sehen, reagiert der Differential-Anteil ausschließlich auf die Änderungsrate des Fehlers, nicht auf dessen absolute Größe. Nähert sich der Roboter der Linie und der Fehler wird kleiner, erzeugt der negative Anstieg einen gegensteuernden D-Anteil, der die Lenkbewegung rechtzeitig abbremst und Überschwingen reduziert. Bei plötzlichen Kurven

hingegen bewirkt ein stark wachsender Fehler einen positiven D-Anteil, welcher die nötige Lenkkorrektur sofort verstärkt.⁴

Implementierung in Python:

Obwohl der Stellwert oft vereinfacht ohne Zeitfaktor dargestellt wird, nutzt die Implementierung in `PDcontroller.py` zur präziseren Bestimmung der Änderungsgeschwindigkeit die reale Zeitdifferenz **dt** in Sekunden. Die genaue Berechnung lautet wie folgt:

$$u = k_p * e + k_d * \frac{e - e_{vorher}}{dt}$$

Der Bruch $\frac{e - e_{vorher}}{dt}$ berechnet dabei die Änderungsgeschwindigkeit der Regelabweichung.

Zusätzlich stabilisiert ein Max-Derivative-Filter extreme Ausschläge der zeitlichen Ableitung, während ein Multi-Sampling-Verfahren und ein finales Clipping die Stellgröße für die Motoren absichern.

7.3 Kalibrierung

Eine korrekte Kalibrierung ist Voraussetzung für stabile Fahreigenschaften. Aufgrund mechanischer Fertigungstoleranzen zwischen beiden Antriebsmotoren sowie unterschiedlicher Untergrundreibwerte müssen Trim-Faktoren und Anlaufschwellen individuell eingestellt werden. Alle Kalibrierwerte sind in `main.py` zentral abgelegt.

```
23 BASE_SPEED = 40           # Grundgeschwindigkeit der Motoren (0-100)
24 KP_WERT = 0.6             # Proportional-Anteil für den PD-Regler (Lenkempfindlichkeit)
25 KD_WERT = 0.02           # Differential-Anteil für den PD-Regler (Dämpfung gegen Schwingen)
26
27
28 TRIM_MOTOR_A = 1.1        # Korrekturfaktor für linken Motor (A), falls er schwächer ist
29 TRIM_MOTOR_B = 1         # Korrekturfaktor für rechten Motor (B)
```

Abbildung 26: Konfiguration der Trim-Werte im Quellcode

Die empfohlene Kalibrierreihenfolge beginnt mit der Bestimmung der minimalen Anlauf-Duty-Cycle-Werte (`MINDUTY_A` und `MINDUTY_B`) für jeden Motor separat: Der Wert wird schrittweise erhöht, bis sich der Motor zuverlässig dreht, und anschließend um eine Reserve von ca. 500 Ticks erhöht. Im zweiten Schritt werden die Trim-Faktoren (`TRIM_A` und `TRIM_B`) justiert, indem der Roboter auf einer geraden Strecke ohne Linie fährt und der Faktor des schnelleren Motors so lange reduziert wird, bis beide Motoren mit gleicher Geschwindigkeit drehen.

⁴ Vgl. Rule the Bots; Linienfolger mit einem PD-Regler

Abschließend erfolgt die Abstimmung der PD-Parameter k_p und k_d auf der tatsächlichen Fahrstrecke. Ein zu hoher k_p -Wert führt zu Oszillationen um die Linie, ein zu niedriger Wert zu träger Reaktion. Der k_d -Anteil dämpft Schwingungen. Überhöhte k_d -Werte können jedoch bei schnellen Sensorübergängen zu ruckartigen Korrekturbewegungen führen. Die Kalibrierung sollte nach jedem Hardwaretausch und nach Änderungen am Untergrund wiederholt werden.

Tabelle 15 dokumentiert alle implementierten Parameter sowie deren Bedeutung. Alle Werte sind zentral in `main.py` abgelegt und können ohne Eingriff in die Modulimplementierungen angepasst werden.

Parameter	Wert	Einheit	Bedeutung
<code>BASE_SPEED</code>	40	% PWM	Grundgeschwindigkeit beider Motoren bei Geradeausfahrt ($e = 0$).
KP	0,6	–	Proportionalanteil: direkte Lenkkorrektur proportional zum aktuellen Fehler.
KD	0,02	–	Differentialanteil: Dämpfung bei schnellen Fehleränderungen.
<code>TRIM_A</code>	1,1	Faktor	Korrekturfaktor für den linken Motor zur Geradeauskalibrierung.
<code>TRIM_B</code>	1,0	Faktor	Korrekturfaktor für den rechten Motor (Referenzmotor).
<code>MINDUTY_A</code>	29500	Ticks	Minimale PWM-Breite zum Überwinden der Anlaufschwelle des linken Motors.
<code>MINDUTY_B</code>	27500	Ticks	Minimale PWM-Breite zum Überwinden der Anlaufschwelle des rechten Motors.

Tabelle 15: Parameter des PD-Reglers und der Motoransteuerung

7.4 Software-Qualität und Performance

Die Echtzeittauglichkeit der Client-Software wird durch ein abgestimmtes Zusammenspiel aus Dual-Cycle-Timing, gezielter WLAN-Deaktivierung und nicht-blockierender Ein-/Ausgabe sichergestellt.

Dual-Cycle-Timing:

Das Timing der Steuerungsschleife ist in zwei überlagerte Zyklen aufgeteilt. Den inneren Fahrzyklus mit einer Periode von 20 ms (50 Hz) und einen äußeren Netzwerkzyklus mit einer Periode von 50 ms (20 Hz). Im Fahrzyklus werden Liniensensor und PD-Regler ausgewertet sowie der Motorstellwert aktualisiert. Der Netzwerkzyklus verarbeitet eingehende TCP-Nachrichten und sendet Statusinformationen an den Server. Durch diese Trennung bleibt die Regelqualität der Linienverfolgung von Schwankungen in der Netzwerklatenz unabhängig.

Hard-Offline-Modus:

Ein zentrales Designmerkmal ist die vollständige Deaktivierung der WLAN-Schnittstelle während der autonomen Fahrt. Hintergrund ist die begrenzte CPU-Leistung des Raspberry Pi Pico 2 W. Der WLAN-Stack belegt Interrupts und Prozessorzeit, was auf der einkernigen Architektur zu messbaren Latenzen im Regelkreis führt. Im Hard-Offline-Modus stehen alle CPU-Ressourcen ausschließlich dem 20-ms-Fahrzyklus zur Verfügung. Die Wiederverbindung zum Server erfolgt nach dem Beenden oder Unterbrechen der autonomen Fahrt automatisch.

Nicht-blockierende Ein-/Ausgabe:

Alle peripheren Schnittstellen sind konsequent nicht-blockierend ausgeführt. Die Ultraschallmessung nutzt GPIO-Interrupts zur Flankenerkennung, sodass die Wartezeit auf ein Echo die Hauptschleife nicht blockiert. Die TCP-Kommunikation in `network_controller.py` verwendet `select.poll()` mit einem Timeout von 0, sodass das Abfragen offener Sockets ohne Blockierung möglich ist. Tasterentprellung und Buzzer-Muster basieren auf `ticks_diff`-Zeitstempeln, die ebenfalls keine blockierenden Schlafbefehle benötigen.

Timeout-basierte Sicherheitsmechanismen:

An allen kritischen Wartestellen sind Timeout-Mechanismen implementiert. Bleibt die WLAN-Verbindung beim Wiederverbindungsversuch aus, bricht das System den blockierungsfreien Versuch nach einem konfigurierbaren Timeout (20 Sekunden) ab, setzt einen Retry-Timer und wechselt später in einen sicheren Offline-Modus, ohne die Motorsteuerung zu blockieren. Ebenso wird eine Ultraschallmessung bei ausbleibendem Echo-Impuls nach 30 ms automatisch abgebrochen. Der Sensor liefert in diesem Fall keinen Distanzwert zurück (None), was vom System korrekt als offener Raum ('kein Hindernis') interpretiert wird, sodass die Fahrt ungestört fortgesetzt werden kann.

7.5 Fachliche Probleme, Analyse und Lösungen

Während der Entwicklung der Client-Software traten zwei zentrale Problemkomplexe auf: Instabilitäten bei der Linienverfolgung sowie ein asymmetrisches Verhalten von Motor A, das trotz umfangreicher Debugging-Maßnahmen bis zum Projektabschluss nicht vollständig

gelöst werden konnte. Die folgende Dokumentation beschreibt Symptome, Ursachen und implementierte Lösungen als Einstiegspunkt für Folgeentwicklungen.

Problem 1: Ruckeln im Online-Modus (WLAN-Interferenz und variable Zykluszeit) – Status: Gelöst

Sobald eine aktive Serververbindung bestand, ruckelte der Roboter stark und verlor die Linie häufig, obwohl er im Offline-Modus stabil fuhr. Zwei Ursachen wirkten zusammen: TCP-Overhead und Socket-Operationen verlängerten einzelne Schleifendurchläufe von typischen 10–15 ms auf bis zu 30 ms, ohne dass der PD-Regler die veränderte Zeitdifferenz berücksichtigte. Der nicht normierte D-Anteil erzeugte dadurch überproportional große Korrekturen. Zusätzlich waren die Liniensensor-Pins ohne Pull-Up oder Pull-Down konfiguriert, sodass elektromagnetische Störungen des WLAN-Moduls Fehlmessungen auf den hochohmigen Eingängen verursachten, die als abrupte Positionssprünge im Regler ankamen.

Die Lösung umfasste fünf Maßnahmen: Der PD-Controller wurde auf Zeitnormierung umgestellt, sodass der D-Anteil durch die tatsächliche Zeitdifferenz dt dividiert und auf einen Maximalwert begrenzt wird. In der Liniensensor-Klasse wurde ein Majority-Vote-Verfahren (3 Messungen pro Sensor, Mehrheitswert) mit einem Caching-Intervall von 8 ms eingeführt. Die Hauptschleife erhielt ein Zwei-Task-Modell mit festen Taktraten: Fahrtlogik alle 20 ms (50 Hz), Netzwerklogik alle 50 ms (20 Hz). Schließlich wurde der Hard-Offline-Modus eingeführt: Beim Start der Fahrt wird die WLAN-Verbindung proaktiv getrennt, sodass während der gesamten Fahrt keinerlei Netzwerklatenz auftreten kann. Die Liniensensor-Pins wurden zudem mit einem internen Pull-Up-Widerstand konfiguriert, um elektromagnetische Störungen des WLAN-Moduls auf den hochohmigen Eingängen zu unterdrücken.

Problem 2: Häufiger Zustandswechsel in `LINE_LOST` – Status: Gelöst



Abbildung 27: Positionierung der Liniensensoren auf der Unterseite des Fahrwerks

Der Roboter wechselte während der Fahrt wiederholt kurzfristig in den Zustand `LINE_LOST`. Da die Linie kurz darauf wieder erkannt wurde, entstanden rasche Zustandswechsel im Sekundenrhythmus, die den Fahrablauf stark störten. Ursache war, dass bereits ein einziger None-Rückgabewert der Sensorauswertung den Zustandswechsel auslöste. Ausgehend von

den aktuell verwendeten Sensoren und ihrer Anordnung kommt die Situation, in der sich die Linie exakt zwischen zwei Sensoren befindet und von keinem der Sensoren erfasst wird, regelmäßig vor.

Als Lösung wurde ein zweistufiger Mechanismus eingeführt. In der ersten Stufe fährt der Roboter nach dem Eintritt in `LINE_LOST` mit der Basisgeschwindigkeit geradeaus weiter, anstatt sofort disruptiv zu reagieren. Diese Phase hält an, bis sowohl 12 aufeinanderfolgende Messzyklen ohne Liniensignal als auch ein Zeitfenster von 800 ms abgelaufen sind. Wird die Linie innerhalb dieses Fensters wiedergefunden, kehrt das System unmittelbar in den Zustand `LINE_FOLLOWING` zurück, ohne dass der Fahrfluss nennenswert unterbrochen wird. Erst wenn beide Bedingungen erfüllt sind, greift die zweite Stufe: ein Fallback-Modus, in dem die zuletzt bekannte Linienposition als Eingangsgröße für den PD-Regler verwendet wird, um die Regelung so lange wie möglich aufrechtzuerhalten.

Problem 3: Verzögertes Gegenlenken in S-Kurven – Status: Gelöst

Bei S-Kurven lenkte der Roboter zu spät gegen die zweite Kurve. Die Entprellung hielt den letzten gültigen Lenkimpuls („stark links“) aktiv, während die Rechtskurve bereits begonnen hatte.

Für enge Rechtskurven wurde ein Sharp-Turn-Assist implementiert: Wenn das kurveninnere Rad einen Geschwindigkeitswert unter 5 % erreicht, wechselt die Steuerung automatisch in einen Pivot-Modus, in dem das Außenrad auf volle Geschwindigkeit und das Innenrad auf –20 % (rückwärts) gesetzt wird.

Problem 4: Asymmetrisches Verhalten von Motor A - Status: teilweise kompensiert, Grundursache ungeklärt

Motor A (linkes Antriebsrad) verhielt sich während der gesamten Entwicklungsphase abweichend von Motor B. Das Problem äußerte sich in drei Symptomen: Geschwindigkeitsänderungen, die gezielt an Motor A übergeben wurden, wurden nicht zuverlässig umgesetzt, während Motor B korrekt reagierte. Im Offline-Betrieb der Pivot-Drehung traten gelegentlich Geschwindigkeitsspitzen von ca. 1 Sekunde Dauer auf, bei denen Motor A kurzzeitig auf volle Geschwindigkeit (100 %) sprang und danach zur programmierten Geschwindigkeit zurückfiel. Darüber hinaus zeigte Motor A allgemein ein träges und inkonsistentes Verhalten insbesondere bei Rechtskurven, in denen er als Außenrad schneller drehen muss.

Die Debugging-Maßnahmen umfassten: Überprüfung der Pin-Initialisierung der Enable-Pins (kein Befund), testweises Vertauschen der Pivot-Drehrichtung (Problem blieb motorgebunden, nicht richtungsgebunden), Aktivierung von Debug-Ausgaben in `dcmotor.py` (softwareseitige Befehle waren korrekt), vollständige Prüfung aller Aufrufstellen von

`motor_A.forward()`, `motor_A.backward()` oder `motor_A.stop()` (keine fehlerhaften Aufrufe) sowie mechanische und elektrische Sichtprüfung (kein eindeutiger Befund).

Da die Grundursache nicht identifiziert werden konnte, sind folgende Workarounds im aktuellen Code implementiert: ein erhöhter konfigurativer Trim-Faktor (`TRIM_A` = 1.1), eine erhöhte Anlaufschwelle (`MINDUTY_A` = 29500 gegenüber 27500 für Motor B), ein dynamischer Trim in `follow_line()` (Standard: 0,8; bei extremen Rechtskurven: 1,2), eine asymmetrische Gewichtung der Liniensensor-Ausgabe (linker Mittelsensor: Faktor 1,5; rechter Mittelsensor: Faktor 1)

Für Folgegruppen wird empfohlen, Motor A und Motor B physisch zu tauschen (inkl. Verkabelung zur H-Brücke), um zu klären, ob das Problem motor- oder treibergebunden ist. Ergänzend sollte das PWM-Signal an GPIO 2 mit einem Oszilloskop gemessen werden, um Signalspitzen oder Kontaktunterbrechungen zu identifizieren.

8 Software: Server

8.1 Ziele der Server-Software

Die Serversoftware übernimmt die übergeordnete Missionslogik des Systems und bildet das Gegenstück zur hardwarenahen Steuerung des Clients. Ihre Aufgabe ist es, Statusmeldungen des Pico auszuwerten, den geeigneten Zeitpunkt für Fahrbefehle zu bestimmen und die Kommunikation mit dem Client zuverlässig zu koordinieren. Dabei muss die Software robust gegenüber Verbindungsunterbrechungen sein, da der Client im Hard-Offline-Modus die TCP-Verbindung während der Fahrt aktiv trennt. Darüber hinaus soll die Software für den Demonstrationsbetrieb geeignet sein: Zustandsübergänge werden im Terminal sichtbar protokolliert, und das System soll nach einer kurzzeitigen Unterbrechung den zuletzt bekannten Kontext wiederherstellen können, ohne eine vollständige Neuinitialisierung durchzuführen.

8.2 Dateistruktur und Modulübersicht

Die Serversoftware besteht aus zwei Python-Dateien. Die Datei `mac_server.py` implementiert den TCP-Server und ist für den Netzbetrieb zuständig. Sie öffnet einen Socket auf dem Interface `0.0.0.0` an Port 50002, nimmt eingehende Verbindungen entgegen und verwaltet pro Client einen eigenen Empfangs-Thread. Über `select.select()` mit einem Timeout von 1,0 Sekunden wird der Socket nicht-blockierend abgefragt. Eine `threading.Lock`-Instanz schützt die gemeinsam genutzte Client-Liste vor gleichzeitigen Schreibzugriffen aus mehreren Threads. Zusätzlich überwacht ein Inaktivitäts-Timeout von 300 Sekunden, ob der Client noch erreichbar ist, und trennt die Verbindung bei Überschreitung.

Die Datei `mission_control.py` enthält die Klasse `MissionControl`, welche die gesamte Missionslogik kapselt. Sie definiert die Server-Zustandsmaschine, wertet eingehende Nachrichten aus und stellt über die Methode `pop_pending_command()` den nächsten zu sendenden Befehl bereit. Der Server-Thread in `mac_server.py` ruft diese Methode nach jeder Nachrichtenverarbeitung ab und sendet den Befehl, sofern einer vorliegt. Nach dem Versand wird `notify_cmd_sent()` aufgerufen, damit `MissionControl` den zugehörigen Zustandsübergang vollziehen und den ACK-Timer starten kann.

8.3 Server-Zustandsmaschine

Die Server-Zustandsmaschine ist als explizite Zustandsvariable [state] in der Klasse `MissionControl` realisiert und wechselt ausschließlich über definierte Übergangsbedingungen. Sie unterscheidet fünf Zustände, deren Semantik und Übergangsbedingungen im Folgenden beschrieben werden.

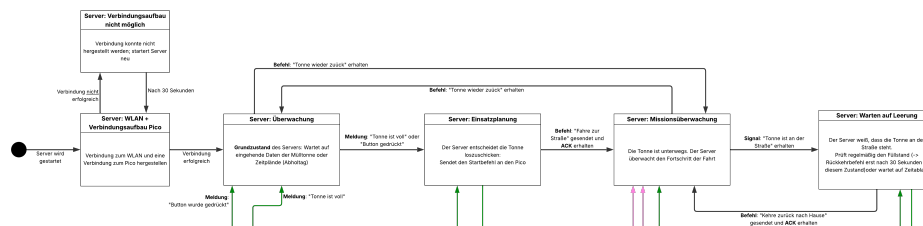


Abbildung 28: State-Machine des Servers

Im Zustand **MONITORING** befindet sich der Server im Normalbetrieb. Er empfängt Heartbeat-Statusmeldungen des Clients und wertet diese aus, ohne aktiv in den Missionsablauf einzugreifen. Der Zustand bleibt erhalten, bis der Client `STATUS: FULL` oder `STATUS: MANUAL_GOTO_STREET_REQUEST` meldet.

Bei Eingang eines der genannten Auslöser wechselt der Server in den Zustand **PLANING**. In diesem Zustand startet ein 10-Sekunden-Timer, der dem Nutzer eine kurze Vorbereitungszeit einräumt, bevor der Fahrbefehl automatisch abgesetzt wird. Beispielsweise um den Deckel rechtzeitig wieder zu schließen. Die Methode `check_timeouts()` wird vom Server-Loop regelmäßig aufgerufen und prüft, ob der Timer abgelaufen ist. Nach Ablauf der 10 Sekunden wird der Befehl `CMD_GOTO_STREET` in die interne Warteschlange eingestellt und im nächsten Loop-Durchlauf durch `mac_server.py` versendet. Ein erneutes `STATUS: FULL` während des laufenden Timers löst keinen zweiten Timer aus. Das System ignoriert redundante Auslöser, solange es sich bereits in **PLANING** oder einem fortgeschritteneren Zustand befindet.

Sobald `mac_server.py` den Befehl erfolgreich versendet hat und `notify_cmd_sent()` aufgerufen wird, wechselt der Server in den Zustand **WAITING_FOR_ACK**. In diesem Zustand wird ein weiterer Timer mit einer Länge von 5 Sekunden gestartet. Antwortet der Client nicht innerhalb dieser Zeit mit dem erwarteten ACK `CMD_GOTO_STREET`, wird der Befehl automatisch erneut versendet und der Timer zurückgesetzt. Dieses Retry-Verhalten ist in `check_timeouts()` implementiert und stellt sicher, dass ein verlorengegangener Befehl nicht zum Stillstand des Systems führt. Geht die erwartete Bestätigung ein, antwortet der Server mit `ACK_RECEIVED CMD_GOTO_STREET` und wechselt in den Zustand **MISSION_MONITORING**.

Im Zustand **MISSION_MONITORING** ist die Tonne unterwegs. Der Server überwacht eingehende Statusmeldungen, greift jedoch nicht mehr aktiv ein. Der Zustand endet, wenn der Client

ARRIVED: STREET meldet. Der Server wechselt dann in WAITING_FOR_EMPTYING. Ein analoger Ablauf gilt für die Rückfahrt: Dort wird aus WAITING_FOR_EMPTYING heraus der Befehl CMD_RETURN_HOME vorbereitet, erneut mit einem 10-Sekunden-Timer und nachfolgender ACK-Handshake-Sequenz, bis schließlich ARRIVED: HOME den Übergang zurück nach MONITORING auslöst.

Ergänzend zu den beschriebenen Zustandsübergängen implementiert die Methode `_sync_state_with_pico()` einen Self-Healing-Mechanismus. Sie wird bei jeder eingehenden STATUS:-Nachricht aufgerufen und gleicht den Server-Zustand mit dem tatsächlich gemeldeten Client-Zustand ab. Meldet der Client beispielsweise STANDBY, obwohl der Server sich noch in einem abweichenden Zustand befindet – etwa nach einem Neustart des Servers bei laufendem Client, korrigiert `_sync_state_with_pico()` den Server-Zustand eigenständig auf MONITORING und setzt `_last_arrived_place` auf "HOME". Entsprechend wird bei WAIT_AT_STREET der Zustand auf WAITING_FOR_EMPTYING gesetzt. Bei LINE_FOLLOWING im Zustand MONITORING wechselt der Server eigenständig zu MISSION_MONITORING. Dieser Mechanismus stellt sicher, dass kurze Verbindungsunterbrechungen oder ein Serverneustart nicht zu einer dauerhaften Inkonsistenz zwischen Client- und Server-Zustand führen.

8.4 Kommunikationsprotokoll und Nachrichtenformat

Die Kommunikation zwischen Client und Server basiert auf einem zeilenbasierten Klartextprotokoll über TCP. Jede Nachricht wird als UTF-8-kodierter String übertragen und durch ein Zeilenumbruchzeichen (`\n`) abgeschlossen. Das Protokoll ist zustandsabhängig: Dieselbe Zeichenkette kann je nach aktuellem Server-Zustand unterschiedliche Auswirkungen haben oder auch ignoriert werden.

Tabelle 16 listet alle Nachrichten auf, die vom Client (Pico) an den Server gesendet werden.

Nachricht	Beschreibung
Pico ist bereit	Handshake-Nachricht; signalisiert Verbindungsbereitschaft
STATUS: STANDBY	Heartbeat: Ruhemodus, Tonne zuhause
STATUS: FULL	Heartbeat: Füllstand voll, Einsatz erforderlich
STATUS: LINE_FOLLOWING	Heartbeat: autonome Fahrt aktiv
STATUS: OBSTACLE	Heartbeat: Hindernis erkannt, Motoren gestoppt
STATUS: WAIT_AT_STREET	Heartbeat: Tonne an der Straße, wartet auf Leerung

STATUS: ARRIVED	Heartbeat: Zielpunkt erreicht (Übergangsphase)
STATUS: EMPTIED	Heartbeat: Tonne wurde entleert
STATUS: USER_PAUSED	Heartbeat: Fahrt manuell pausiert
STATUS: LINE_LOST	Heartbeat: Linie verloren
STATUS: MANUAL_GOTO_STREET_REQUEST	Manueller Fahrauftrag zur Straße (Grüner Taster wurde gedrückt)
STATUS: MANUAL_RETURN_HOME_REQUEST	Manueller Rückfahrauftrag (Grüner Taster wurde gedrückt)
ACK_CMD_RETURN_HOME	Empfangsbestätigung für Rückfahrbefehl
ACK_CMD_GOTO_STREET	Empfangsbestätigung für Fahrbefehl
ARRIVED: STREET	Ankunfts meldung: Straße erreicht
ARRIVED: HOME	Ankunfts meldung: Heimatstandort erreicht

Tabelle 16: Übersicht der Nachrichten, die der Client sendet

Tabelle 17 listet alle Nachrichten auf, die vom Server an den Client gesendet werden.

Nachricht	Beschreibung
CMD_GOTO_STREET	Fahrauftrag zur Straße
CMD_RETURN_HOME	Rückfahrauftrag nach Hause
ACK_RECEIVED_CMD_GOTO_STREET	Handshake-Abschluss: Freigabe für Fahrt zur Straße
ACK_RECEIVED_CMD_RETURN_HOME	Handshake-Abschluss: Freigabe für Rückfahrt

Tabelle 17: Übersicht der Nachrichten, die der Server an den Client sendet

8.5 Missionsablauf

Ein vollständiger Missionszyklus beginnt im Zustand **MONITORING**, in dem der Server kontinuierlich Heartbeat-Meldungen des Clients entgegennimmt. Meldet der Client **STATUS: FULL**

oder löst der Nutzer per langem Tastendruck `STATUS: MANUAL_GOTO_STREET_REQUEST` aus, leitet der Server die Einsatzplanung ein und wechselt nach `PLANING`. Nach Ablauf des 10-Sekunden-Timers wird `CMD_GOTO_STREET` versendet. Der Server wartet anschließend im Zustand `WAITING_FOR_ACK` auf die Bestätigung durch den Client. Nach erfolgreichem ACK-Handshake wechselt der Server nach `MISSION_MONITORING` und der Client beginnt die autonome Fahrt im Hard-Offline-Modus.

Sobald die Tonne die Straße erreicht hat, sendet der Client nach dem Reconnect die Meldung `ARRIVED: STREET`. Der Server wechselt daraufhin in `WAITING_FOR_EMPTYING`. Sobald der Füllstandssensor eine Entleerung erkennt, meldet der Client `STATUS: EMPTIED`. Alternativ kann der Nutzer per langem Tastendruck `STATUS: MANUAL_RETURN_HOME_REQUEST` auslösen. In beiden Fällen startet ein weiterer 10-Sekunden-Timer, nach dessen Ablauf der Server `CMD_RETURN_HOME` versendet. Der Rückfahrablauf folgt der gleichen ACK-Handshake-Sequenz. Nach Empfang von `ARRIVED: HOME` kehrt der Server in den Zustand `MONITORING` zurück und ein neuer Zyklus kann beginnen.

8.6 Verbindungsmanagement und Reconnect-Logik

Die Verwaltung des TCP-Sockets erfolgt in `mac_server.py`. Pro eingehender Verbindung wird ein eigener Daemon-Thread gestartet, der den jeweiligen Client bedient. Der Hauptthread akzeptiert neue Verbindungen und registriert sie in einer durch `threading.Lock` geschützten Client-Liste. Ein Inaktivitäts-Timeout von 300 Sekunden bewirkt die automatische Trennung inaktiver Verbindungen.

Eine besondere Herausforderung ergibt sich aus dem Hard-Offline-Modus des Clients: Der Pico trennt die TCP-Verbindung nach dem ACK-Handshake aktiv und stellt sie erst nach Ankunft am Zielpunkt wieder her. Aus Sicht des TCP-Servers entspricht dies einer regulären Verbindungstrennung, gefolgt von einer neuen eingehenden Verbindung. Die Klasse `MissionControl` muss daher unterscheiden, ob ein neu eingehender Handshake eine erste Verbindung oder einen Reconnect nach der Fahrt darstellt.

```
78 | def on_client_connected(self, client_addr):
79 |     """
80 |     Wird aufgerufen, wenn der Pico "Pico ist bereit" (Handshake) sendet.
81 |     Entscheidet, ob es ein neuer Start oder ein Reconnect ist.
82 |     """
83 |     # Reconnect erkennen: War der Pico schon mal da?
84 |     is_reconnect = bool(getattr(self, "_pico_ready", False))
85 |     self._pico_ready = True
```

Abbildung 29: Methode des Servers zum Erkennen eines bereits verbundenen Clients

Dieser Ablauf vollzieht sich in zwei klar getrennten Schritten. Beim Aufbau der TCP-Verbindung wird zunächst die Methode `on_socket_connected()` aufgerufen, die die neue

Verbindung lediglich protokolliert und auf den Handshake wartet. Das Flag `_pico_ready` wird dabei noch nicht verändert. Erst wenn der Pico als erste Nachricht Pico ist bereit sendet, ruft `process_incoming_msg()` die Methode `on_client_connected()` auf. Diese liest zunächst den aktuellen Wert von `_pico_ready` aus und setzt ihn anschließend auf `True`. Dieser Lesezugriff vor dem Schreibzugriff ist der entscheidende Mechanismus zur Unterscheidung von Erst- und Wiederverbindung: War `_pico_ready` noch `False`, handelt es sich um die Erstverbindung seit dem Serverstart. In diesem Fall werden alle Laufzeitvariablen zurückgesetzt und der Server wechselt in den Zustand `MONITORING`. War `_pico_ready` hingegen bereits `True`, erkennt der Server die eingehende Verbindung als Reconnect und verzichtet auf einen Zustandsreset.

Im Reconnect-Fall wird der Missionskontext stattdessen anhand zweier persistenter Variablen wiederhergestellt, die über Verbindungsunterbrechungen hinweg erhalten bleiben.

`_last_seen_pico_state` speichert den zuletzt empfangenen Statuswert aus einer STATUS: - Nachricht, `_last_arrived_place` den letzten bekannten Aufenthaltsort aus einer ARRIVED: - Meldung. War der Pico zuletzt zuhause, setzt der Server den Zustand `MONITORING` fort. War er an der Straße, wird `WAITING_FOR_EMPTYING` gesetzt. Mehrdeutige oder aktive Zustände wie `ARRIVED`, `USER_PAUSED` oder `OBSTACLE` werden beibehalten, ohne einen automatischen Übergang auszulösen, da der Server in diesen Fällen auf die nächste eindeutige Statusmeldung des Clients wartet.

Die Wiederherstellung folgt einer priorisierten Fallunterscheidung: Ist `_last_arrived_place` gleich "HOME" oder war der letzte bekannte Pico-Zustand `STANDBY`, wird der Server-Zustand auf `MONITORING` gesetzt – der Client befindet sich zuhause, und eine neue Mission kann beginnen. Ist `_last_arrived_place` gleich "STREET" oder war der letzte bekannte Zustand `WAIT_AT_STREET`, wird der Server-Zustand auf `WAITING_FOR_EMPTYING` gesetzt – die Tonne steht an der Straße und wartet auf Leerung. Ist der letzte bekannte Zustand `ARRIVED`, ist der genaue Aufenthaltsort noch nicht eindeutig bestimmbar. Der Server behält seinen bisherigen Zustand bei und wartet auf die nachfolgende `ARRIVED: HOME`- oder `ARRIVED: STREET`-Meldung zur eindeutigen Klärung. Bei den Zuständen `USER_PAUSED` und `OBSTACLE` wird der Server-Zustand ebenfalls unverändert beibehalten, da der Client an einer bekannten Position pausiert und keine Neuzuweisung des Missionsschritts erforderlich ist.

Durch diesen Mechanismus übersteht das Gesamtsystem kurze WLAN-Unterbrechungen und den Hard-Offline-Fahrzyklus ohne manuellen Eingriff und ohne Informationsverlust über den aktuellen Missionsstand.

8.7 Software-Qualität und Robustheit

Die Serversoftware ist auf eine robuste Ausführung im Demonstrationsbetrieb ausgelegt. Die vollständige Entkopplung von Netzwerk-Thread und Missionslogik über die Schnittstelle `pop_pending_command()` / `notify_cmd_sent()` verhindert, dass eine langsame Netzwerkoperation die Zustandsmaschine blockiert. Der Self-Healing-Mechanismus in `_sync_state_with_pico()` korrigiert Zustandsabweichungen, die durch einen Serverneustart bei laufendem Client entstehen können. Das ACK-Retry mit 5-Sekunden-Timeout sichert den Handshake gegen vereinzelte Paketverluste ab. Alle Zustandsübergänge und relevanten Ereignisse werden mit kategorisierten Log-Meldungen im Terminal ausgegeben, was die Nachvollziehbarkeit im Demonstrationsbetrieb unterstützt. Statusnachrichten werden durch eine Entprellung gefiltert: Wiederholt eingehende identische STATUS:-Nachrichten werden erkannt und nicht erneut geloggt, um die Ausgabe übersichtlich zu halten.

9 Evaluation der Arbeitsergebnisse

9.1 Bewertung: Zielerreichung pro Use Case

Die Erreichung der definierten Use Cases wurde systematisch anhand der funktionalen Anforderungen (Kapitel 2), Testprotokolle und Messergebnisse bewertet. Die folgende Tabelle fasst den Ist-Zustand objektiv zusammen:

Use Case	Erfüllungsgrad	Bewertung	Erreichte Kernfunktionen	Offene Punkte	Techn. Verweis
UC1 Autonome Fahrt	100%	Voll erfüllt	• Linienfolge stabil (PD-Regler $K_p=1.2$, $K_d=0.8$, 50 Hz) • Hindernis-Stopp <100 ms (Abstand <35 cm) • 180° Pivot-Drehung reproduzierbar • Hard-Offline-Modus robust	Dynamische Umfahrung (UC1.2)	7.1–7.3
UC2 Füllstandsüberwachung	100%	Erfüllt	• HC-SR04P kalibriert (Leer/Voll → %) • Schwelle: 75%	Feuchteempfindlichkeit (Work-around)	5.2
UC3 Statusanzeige	100%	Voll erfüllt	• Dual-LED (Betrieb/Server) • Buzzer-Protokoll normkonform • Gedruckte Acylhauben optimieren Sichtbarkeit	App-Integration (UC3.1)	5.1
UC4 Server-Verwaltung	100%	Voll erfüllt	• Bidirektionale TCP (Heartbeat 2 s) • Reconnect zum Server <10 s (Aktivieren des Offline-Modus nach 30 s)	• Digitaler Kalender • Multi-Tonne-Support	8.3-8.6

			• Missionslogik (GO-TOSTREET/RETURN)		
--	--	--	--	--	--

Tabelle 18: Use Case Erfüllungsmatrix

Status	Definition	Kriterien
Voll erfüllt	100% Kernfunktionen implementiert, getestet, stabil	Alle Must-Have aus UC-Spezifikation
Teilweise	70-99% – Kernfunktionalität lauffähig	Nice-to-Have offen
Nicht erfüllt	<70% oder Out-of-Scope	Nicht im Sprint-Backlog

Tabelle 19: Bewertungskriterien

Gesamtbewertung

Zielerreichung: 100% (gewichteter Durchschnitt nach UC-Priorisierung Kapitel 2.3).

Prototyp-Status: Feldtest-reif für kontrollierte Umgebungen (schwarze Linie, WLAN in kurzer bis mittlerer Entfernung).

Nächste Meilensteine: Leiterplatte, Multi-Tonne-Server, Kalenderintegration.

9.2 Grenzen des Prototyps

Trotz der hohen Zielerreichung weist der Prototyp typische Einschränkungen auf, die für den Übergang zur Serienreife relevant sind. Diese Limitierungen betreffen Umgebungstoleranz, Skalierbarkeit und Industrialisierung und wurden systematisch bewertet.

Umgebungseinflüsse

Die Liniensensorik (KY-033) ist auf standardisierte weiße Untergründe mit schwarzer Linie beschränkt (siehe Kapitel 4.6). Temperaturbereich 10-25 °C wurde validiert, Erweiterung auf -10 bis +40 °C erfordert Kalibrierung. Feuchteempfindlichkeit trat bisher nicht auf – HC-SR04P stabil im Testumfeld.

Skalierbarkeit

Das Server-Setup ist für 1 Tonne optimiert. Multi-Tonne-Unterstützung (MQTT-Cluster) ungetestet. WLAN-Reichweite Seminarraum (ca. 20 m²) validiert, LoRaWAN optional für Außenbereich. Cloud-Migration für Skalierung erforderlich.

Prototyp-Größe: 50-Liter-Mülltonne. Originalgrößen (120/240 L) erfordern Skalierung: Durch das größere Maß und Gewicht müssen Komponente wie Motoren, Fahrwerk und Akku neu ausgelegt werden

Robustheit und Industrialisierung

Komponente	Prototyp-Status	Limitierung	Industrialisierung
Gehäuse	Spritzguss-Tonne	IP54	IP67-Zertifizierung
Elektronik	Breadboard + Lötfix	Vibrationen	Dedizierte Leiterplatte
Akku	Blei-Vlies 5 Ah	500 Zyklen	LiFePO4 optional

Tabelle 20: Reifegrad-Übersicht

10 Diskussion und Lessons Learned

10.1 Technische Trade-offs

Die Systementwicklung erforderte mehrere bewusste Abwägungen zwischen Prototyping-Geschwindigkeit, Kosten, Funktionalität und Industrialisierbarkeit. Die zentralen Entscheidungen werden transparent dokumentiert, um Nachvollziehbarkeit und Lessons Learned zu sichern.

Trade-off	Prototyp-Entscheidung	Vorteile	Nachteile	Begründung	Langfristige Konsequenz
Elektronik-Plattform	Breadboard + Lötfix	• Schnelle Iterationen • Hohe Flexibilität	• Vibrationen • IP20-Schutz	Prototyping-Priorität über Langzeitstabilität	Dedizierte Leiterplatte
Linienensorik	5× KY-033	• Kostengünstig • Sofort verfügbar	Weiß/schwarzlimitiert	Budget + Testzeit vs. Universalität	Adaptive Sensorik
Kommunikation	WLAN (Pico W)	• Nahtlose Server-Integration • Hohe Datenrate	Bisher nur auf kurze Reichweite (Indoor) erprobt	Indoor-Setup + Einfachheit	LoRaWAN für Outdoor optional
Energiesystem	Blei-Vlies-Akku 5 Ah	• Preiswert • Robust	• Gewicht 2,5 kg • 500 Zyklen	Verfügbarkeit + Kosten	LiFePO4 für Serienproduktion
Programmiersprache	MicroPython	• Entwicklungszeit • Team-Know-how	Performancesteigerung möglich durch C	Rapid Prototyping	C für High-Performance optional

Tabelle 21: Technische Trade-offs

Methodische Erkenntnisse:

- 80/20-Regel (Pareto): Kernfunktionen (Linie, Füllstand, UI) 100% priorisieren
- Modularität zahlt sich aus: Sensoren können einfach gewechselt werden
- Hardware-Validierung parallel: Motorwechsel → Frühere Mechanik-Tests
- Software first: Logik vor Hardware – reduziert Risiken

Fazit: Trade-offs bewusst und projektgerecht gewählt. Prototyp validiert Konzept – Industrialisierung folgt standardisiert.

10.2 Stärken und Verbesserungspotenziale des Projekts

Bewährte Ansätze und Erfolge

Die Projektorganisation nach dem SCRUM-Framework hat sich im Verlauf des Projekts grundsätzlich bewährt. Die gewählte Sprintlänge erwies sich als angemessen und ermöglichte einen strukturierten Arbeitsrhythmus. Als besonders wertvoll stellte sich die Praxis heraus, zu Beginn jedes Sprints die Arbeitspakete konkret auszuformulieren und in umsetzbare Aufgaben zu überführen. Dieses Vorgehen schuf die nötige Flexibilität, um auf veränderte Anforderungen oder unvorhergesehene Entwicklungen im Projektverlauf reagieren zu können – eine Eigenschaft, die sich in einem interdisziplinären Hardwareprojekt mit vielen Unbekannten als essenziell erwiesen hat. Eine hohe Transparenz im Team sowie ein regelmäßiger, offener Austausch haben maßgeblich zum Projekterfolg beigetragen. Gerade bei einem Projekt mit klar getrennten Zuständigkeitsbereichen – Mechanik, Elektronik und Software – ist eine enge Abstimmung unerlässlich, um Abhängigkeiten frühzeitig zu erkennen und Schnittstellenprobleme zu vermeiden.

Die Client-Server-Kommunikation über TCP erwies sich als stabil und zuverlässig. Das entwickelte Nachrichtenprotokoll mit Heartbeat-Mechanismus und Zustandsrückmeldungen funktionierte im Betrieb wie vorgesehen und bildete eine solide Grundlage für die Missionssteuerung. Auch die Sensorik lieferte insgesamt zufriedenstellende Ergebnisse: Sowohl die Ultraschallsensoren für Füllstand und Hinderniserkennung als auch das Liniensensor-Array arbeiteten zuverlässig im vorgesehenen Einsatzbereich. Das Energiekonzept hat sich als gut dimensioniert erwiesen – der eingesetzte Akku weist eine ausreichende Kapazität auf, sodass im Testbetrieb nur gelegentliches Laden erforderlich war. Das Raupenfahrwerk zeigte auf den getesteten Innenflächen eine sehr gute Stabilität und gleichmäßige Traktion, was reproduzierbare Testfahrten unter kontrollierten Bedingungen ermöglichte.

Herausforderungen und Lessons Learned

Als Herausforderung in der Projektorganisation zeigte sich, dass einzelne Aufgaben in ihrer Komplexität unterschätzt wurden und sich wiederholt über Sprintgrenzen hinaus erstreckten. Dies verdeutlicht, dass eine realistische Aufwandsschätzung insbesondere bei hardwarenahen Tätigkeiten schwer vorherzusagen ist. Die Erfahrung zeigt rückblickend, dass es sinnvoller ist, sich pro Sprint weniger vorzunehmen und dieses vollständig abzuschließen, als zu viele Aufgaben anzugehen und diese unfertig in den nächsten Sprint zu übertragen.

Eine wiederkehrende technische Herausforderung stellte die Asymmetrie der beiden Antriebsmotoren dar. Motor A wies gegenüber Motor B abweichendes Verhalten auf, was zu einer Drift beim Geradeausfahren führte und softwareseitige Kompensationsmaßnahmen erforderte. Die eigentliche Ursache konnte nicht vollständig behoben werden. Ergänzend erwies sich die Motorauswahl und -dimensionierung als schwierig: Die richtige Abstimmung von Drehmoment und Drehzahl auf das Gesamtgewicht des Systems ist ohne Vorerfahrung mit vergleichbaren Aufbauten nur schwer vorab zu treffen.

Der WLAN-Verbindungsaufbau des Raspberry Pi Pico W zeigte ein inkonsistentes Verhalten: Während die Verbindung in manchen Fällen schnell und zuverlässig hergestellt wurde, erforderte sie in anderen Situationen mehrere Versuche, ohne dass ein eindeutiger Auslöser identifiziert werden konnte. Dieses nicht-deterministische Verhalten erschwerte reproduzierbare Tests und ist als bekannte Einschränkung des Systems zu werten.

Im Bereich der Mechanik führten Passungenauigkeiten bei additiv gefertigten Bauteilen mehrfach zu Nachdrucken. Trotz CAD-basierter Konstruktion erforderten Toleranzen beim 3D-Druck sowie Maßabweichungen bei der Integration in das Gesamtsystem iterative Anpassungen, die Zeit und Material kosteten. Die eingesetzten Liniensensoren vom Typ KY-033 arbeiteten zuverlässig und bereiteten im Betrieb keine konkreten Probleme. Dennoch ist festzuhalten, dass die Verwendung von Sensoren mit kontinuierlichen anstelle von diskreten Ausgabewerten die Linienverfolgung grundsätzlich präziser und sanfter gestalten würde, da der PD-Regler auf graduelle Abweichungen statt auf Schaltflanken reagieren könnte.

11 Übergabe: Betrieb und Weiterentwicklung (für Folgegruppen)

11.1 Voraussetzungen und Systemkomponenten

Für den Betrieb des Prototyps werden zwei Softwareumgebungen benötigt: der Mikrocontroller-Client auf dem Raspberry Pi Pico 2 W und die Serversoftware, ausgeführt auf einem Rechner. Auf Seite des Clients muss die MicroPython-Firmware in der Version 1.26.1 installiert sein. Die entsprechende UF2-Datei (`RPI_PICO2_W-20250911-v1.26.1.uf2`) ist im Projektarchiv hinterlegt. Auf dem aktuell im Projekt eingesetzten Raspberry Pi Pico 2 W ist diese Datei bereits installiert, muss jedoch bei der Inbetriebnahme eines neuen Geräts neu aufgespielt werden. Auf Seite des Servers genügt eine Python-3-Installation ohne zusätzliche Drittanbieter-Bibliotheken, da die Serversoftware ausschließlich Module der Standardbibliothek verwendet (socket, threading, select, time).

Die gesamte Hardware – Motoren, Sensoren, LEDs, Buzzer und Taster – ist innerhalb der Mülltonne auf einem 3D-gedruckten Montagerahmen verbaut und über den dokumentierten Verdrahtungsplan mit dem Pico verbunden. Für die Spannungsversorgung sorgt ein Blei-Vlies-Akku, dessen Spannung über einen XL4015-Step-Down-Converter auf das benötigte Niveau geregelt wird. Der Ladevorgang erfolgt über eine DC-Hohlsteckerbuchse am Gehäuse.

11.2 Inbetriebnahme des Clients

Die Client-Software besteht aus mehreren MicroPython-Modulen. Sämtliche Dateien dieses Verzeichnisses müssen auf den internen Speicher des Pico 2 W übertragen werden. Hierfür eignet sich die Erweiterung „MicroPico“ in Visual Studio Code, zu sehen in Abbildung 30. Diese ermöglicht das Installieren und das anschließende Ausführen der Dateien auf dem Pico.

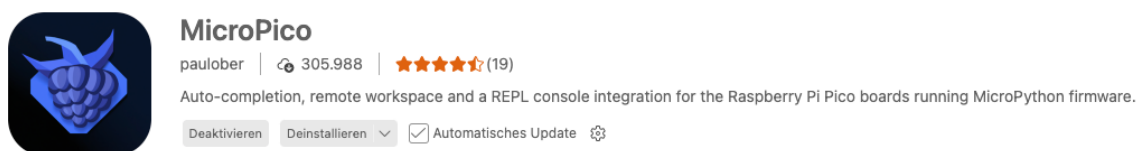


Abbildung 30: Erweiterung MicroPico in Visual Studio Code

Vor dem ersten Start sind in der Datei `main.py` die Netzwerkparameter zu konfigurieren. Die Konstante `SERVER_IP` muss auf die IP-Adresse des Rechners gesetzt werden, auf dem die Serversoftware ausgeführt wird. Der Wert `SERVER_PORT` muss mit dem in `mac_server.py`

konfigurierten Port übereinstimmen. Als Standardwert ist Port 50002 eingestellt. Die WLAN-Zugangsdaten werden über die Konstanten `WLAN_SSID` und `WLAN_PASS` festgelegt. Client und Server müssen sich im selben lokalen Netzwerk befinden.

Die Regelungsparameter des PD-Reglers (`KP_WERT`, `KD_WERT`, `BASE_SPEED`) sowie die Motor-Kalibrierungswerte (`TRIM_MOTOR_A`, `TRIM_MOTOR_B`, `MINDUTY_A`, `MINDUTY_B`) sind in `main.py` mit erprobten Standardwerten vorbelegt. Diese Werte sind auf die aktuell verbauten Motoren und die verwendete Unterlage abgestimmt. Bei einem Austausch von Motoren oder einer Änderung der Fahroberfläche ist eine erneute Kalibrierung erforderlich. Die hierfür relevanten Zusammenhänge sind in Kapitel 7.4 und 7.5 dokumentiert.

Der Füllstandssensor wird über die Parameter `leer_abstand_cm` und `voll_abstand_cm` in `main.py` konfiguriert. Der Wert `leer_abstand_cm` entspricht dem Abstand zwischen Sensor und Tonnenboden bei leerer Tonne, `voll_abstand_cm` dem Abstand bei vollständig gefüllter Tonne. Diese Werte sind experimentell zu ermitteln und an die jeweilige Tonne anzupassen.

Nach der Konfiguration wird der Pico über den USB-Anschluss oder über die Akkuversorgung mit Strom versorgt. Das Programm `main.py` startet automatisch, sobald der Pico mit Strom versorgt wird und der Client beginnt, eine WLAN-Verbindung aufzubauen. Der Verbindungsstatus ist über die Server-LED am Gehäuse erkennbar.

11.3 Inbetriebnahme des Servers

Die Serversoftware wird durch Ausführen der Datei `mac_server.py` gestartet:

Der Server öffnet einen TCP-Socket auf allen verfügbaren Netzwerk-Interfaces (`0.0.0.0`) am konfigurierten Port und wartet auf eingehende Verbindungen. Nach dem Start gibt der Server im Terminal die Meldung „Warte auf Pico-Verbindungen“ aus. Sobald der Client eine Verbindung herstellt und den Handshake (`Pico ist bereit`) abschließt, beginnt der Server mit der Überwachung im Zustand `MONITORING`.

Die Serversoftware gibt alle Zustandsübergänge, Statusmeldungen und Befehle als formatierte Log-Einträge im Terminal aus. Für den Demonstrationsbetrieb ist keine weitere Konfiguration erforderlich. Alle relevanten Parameter sind in `mac_server.py` und `mission_control.py`

11.4 Bedienung im Regelbetrieb

Die Bedienung des Prototyps erfolgt über zwei Taster am Gehäuse: einen grünen Starttaster und einen roten Stopptaster. Im Ruhezustand (`STANDBY`) befindet sich die Tonne am Heimatstandort und übermittelt periodisch Heartbeat-Meldungen an den Server. Ein automatischer Einsatz wird ausgelöst, sobald der Füllstandssensor den konfigurierten Schwellwert erreicht – der Server plant dann die Fahrt mit einem 10-Sekunden-Vorlauf und sendet anschließend den Fahrbefehl.

Alternativ kann ein Einsatz manuell ausgelöst werden: Ein langer Tastendruck auf den grünen Taster im Zustand `STANDBY` sendet einen manuellen Fahrauftrag zur Straße, unabhängig vom aktuellen Füllstand. An der Straße angekommen (`WAIT_AT_STREET`) wartet die Tonne auf die manuelle Leerung. Erkennt der Füllstandssensor die Entleerung, leitet der Server die Rückfahrt ein. Auch hier ist ein manueller Rückfahrauftrag möglich: Ein langer Tastendruck auf den grünen Taster löst die Rückfahrt aus, selbst wenn die Tonne noch nicht entleert wurde.

Der rote Stopptaster kann jederzeit während der Fahrt gedrückt werden und versetzt das System in den Zustand `USER_PAUSED`. Alle Motoren werden sofort abgeschaltet. Ein erneuter Druck auf den Taster setzt den Betrieb fort. Bei einem Hinderniserkennungsereignis (`OBSTACLE`) stoppt der Roboter ebenfalls automatisch und setzt die Fahrt fort, sobald das Hindernis entfernt wurde. Auch in diesem Zustand versetzt ein Tastendruck des grünen Tasters den Pico in den Zustand `USER_PAUSED`. Wird der grüne Taster erneut gedrückt und das Hindernis befindet sich immer noch im Weg, wechselt das System erneut in den Zustand `OBSTACLE`.

Die beiden RGB-LEDs am Gehäuse signalisieren den aktuellen Systemstatus: Eine LED zeigt den Verbindungsstatus zum Server an, die andere den Betriebszustand des Pico. Der Buzzer gibt bei bestimmten Ereignissen akustische Signale aus, beispielsweise bei Ankunft am Zielpunkt oder bei erkannten Fehlerzuständen.

Im Fehlerfall `LINE_LOST` – wenn alle fünf Infrarot-Sensoren das Liniensignal verlieren – stoppt der Roboter an seiner aktuellen Position und wartet auf manuellen Eingriff. In diesem Fall muss der Roboter manuell auf die Linie zurückgesetzt und über den Taster fortgesetzt werden.

11.5 Hinweise zum Demonstrationsbetrieb

Der Prototyp ist für den Betrieb in Innenräumen auf einer glatten, ebenen Oberfläche mit kontrastreichem Linienband konzipiert. Die Liniensensoren vom Typ KY-033 reagieren auf

den Infrarot-Kontrast zwischen dem dunklen Linienband und dem hellen Untergrund. Stark reflektierende Oberflächen, direkte Sonneneinstrahlung oder farbige Untergründe können die Erkennung beeinträchtigen. Der empfohlene Aufbau verwendet schwarzes Isolierband auf hellem Untergrund (z. B. weißes Papier oder heller Laminatboden).

Die Strecke zwischen Heimatstandort und Straße sollte ohne scharfe Winkel ($> 90^\circ$) gestaltet sein, da der PD-Regler für sanfte Kurvenverläufe optimiert ist. An Start- und Endpunkt der Strecke detektieren die Liniensensoren eine schwarze Querlinie im 90-Grad Winkel zur Fahrtrichtung, wie in Abbildung 31 dargestellt.

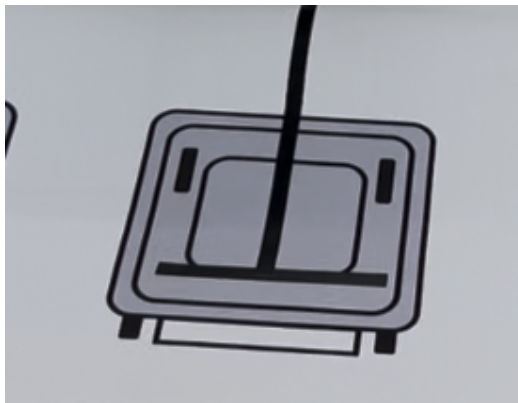


Abbildung 31: Markierung der Start- und Endposition

Vor jeder Vorführung empfiehlt es sich, den Akkuladestand zu prüfen, den Server zu starten und eine erfolgreiche Verbindung im Terminal zu verifizieren, bevor die Tonne positioniert wird. Ein kurzer Testlauf mit manuellem Start (langer Tastendruck) eignet sich zur Überprüfung der Kalibrierung und Linienführung.

11.6 Weiterentwicklung

Der vorliegende Prototyp demonstriert die grundlegende Funktionalität einer autonom navigierenden Mülltonne und bietet mehrere Ansatzpunkte für eine Weiterentwicklung. Die folgenden Erweiterungsmöglichkeiten lassen sich in die Kategorien Hardware-Optimierung, Navigationserweiterung, intelligente Einsatzplanung und smarte Müllwirtschaft gliedern.

Hardware und Sensorik:

Im Bereich der Aktorik stellt die in Kapitel 7.5 dokumentierte Asymmetrie von Motor A eine bekannte Einschränkung dar, die bisher durch softwareseitige Kompensationsmaßnahmen (TRIM_A, MINDUTY_A, dynamische Anpassung) abgemildert, jedoch nicht vollständig behoben wurde. Ein Austausch des betroffenen Motors oder der Einsatz eines geschlossenen

Regelkreises mit Drehzahlrückführung (Encoder-Feedback) würde die Geradeauslaufstabilität signifikant verbessern.

Die derzeit verwendeten Liniensensoren vom Typ KY-033 liefern ausschließlich binäre Ausgabewerte (Linie erkannt oder nicht erkannt). Diese diskrete Erfassung begrenzt die Auflösung der Positionsbestimmung auf fünf Stufen und führt zu sprunghaften Korrekturen des PD-Reglers. Die Umstellung auf Sensoren mit kontinuierlichen (analogen) Ausgabewerten würde eine feingranularere Fehlerberechnung ermöglichen und die Linienverfolgung insgesamt harmonisieren, da der Regler auf graduelle Abweichungen statt auf Schaltflanken reagieren könnte.

Ein Kippsensor wurde im Rahmen des Projekts bereits beschafft, jedoch nicht verbaut. Die Integration eines solchen Sensors in die Client-Software würde es ermöglichen, ein Umkippen der Tonne zu erkennen und den Nutzer darüber zu benachrichtigen, beispielsweise durch ein akustisches Signal über den Buzzer oder eine Statusmeldung an den Server. Dieser Anwendungsfall ist insbesondere für den unbeaufsichtigten Betrieb im Außenbereich relevant.

Über zusätzliche Sensorik, wie etwa ein Gewichtsmodul am Tonnenboden oder ein optischer Sensor im Deckelbereich, könnte das System zukünftig erkennen, ob die eingefüllten Abfälle zum deklarierten Tonnentyp passen. Ein ungewöhnlich hohes Gewicht in einer Papiertonne könnte beispielsweise auf eine Fehlbefüllung hinweisen. Eine entsprechende Warnung an den Nutzer per Server-Benachrichtigung würde die Mülltrennung im Haushalt unterstützen.

Navigation und Einsatzgebiet:

Die Navigation basiert derzeit ausschließlich auf der optischen Linienverfolgung. Für einen Einsatz außerhalb kontrollierter Innenraumumgebungen wäre die Integration eines GPS-Moduls oder einer kamerabasierten Navigation denkbar. In Kombination mit einer wetterfesten Gehäuseversion könnte das System so für den Außenbereich erweitert werden.

Das aktuelle Hindernisverhalten beschränkt sich auf das Anhalten und Warten, bis das Hindernis entfernt wird. Eine Weiterentwicklung könnte eine aktive Hindernisumfahrung implementieren, beispielsweise durch eine kurze Ausweichbewegung mit anschließender Rückkehr zur Linie. Dies erfordert entweder zusätzliche seitliche Abstandssensoren oder einen kamerabasierten Ansatz.

Eine besondere Herausforderung im realen Einsatz ergibt sich nach der Leerung durch die Müllabfuhr: Die Tonne wird in der Regel nicht exakt an der gleichen Stelle abgestellt, an der sie abgeholt wurde. Dadurch befindet sich die Tonne nach der Leerung möglicherweise nicht mehr auf der Linie und kann die Rückfahrt nicht unmittelbar antreten. Dieses Problem wurde im Rahmen des Projekts bereits identifiziert, jedoch zur Begrenzung der Komplexität in der

Simulationsumgebung bewusst ausgegrenzt. Ein denkbarer Lösungsansatz besteht darin, den Startpunkt der Linie an der Straßenseite mit einem Sender (z. B. einem Infrarot- oder Bluetooth-Beacon) auszustatten und die Tonne mit einem gerichteten Empfänger zu versehen, der die Richtung des Senders verorten kann. Auf diese Weise könnte sich die Tonne dem Linienanfang durch einen Suchprozess annähern, auch wenn sie an einer abweichenden Position abgestellt wurde, und anschließend die reguläre Linienverfolgung für die Rückfahrt aufnehmen.

Diebstahlschutz:

In Kombination mit einem GPS-Modul ließe sich ein Diebstahlschutzmechanismus realisieren. Der Server könnte einen zulässigen Aufenthaltsradius um den Heimatstandort und die Straßenposition definieren. Verlässt die Tonne diesen Bereich, ohne dass ein aktiver Fahrauftrag vorliegt, würde der Buzzer ein akustisches Warnsignal ausgeben und der Server eine Benachrichtigung an den Nutzer senden. Die bestehende Statusübermittlung über TCP bietet hierfür bereits eine geeignete Infrastruktur.

Energiemanagement:

Für den Dauerbetrieb im Außenbereich könnte ein Solarpanel auf dem Tonnendeckel in Kombination mit einer Ladezustandsüberwachung integriert werden. Der Server würde den Akkustand als zusätzliche Planungsgröße berücksichtigen: Bei niedrigem Ladestand wird ein Einsatz verzögert, bis ausreichend Energie verfügbar ist, um die Hin- und Rückfahrt vollständig abzuschließen.

Derzeit verfügt die Software noch nicht über die Fähigkeit, den Ladestand des Akkus zu messen. Die hierfür erforderliche Hardware, etwa ein Spannungsteiler am Akkuausgang in Kombination mit einem ADC-Eingang des Pico, sowie die zugehörige Softwareanbindung könnten im weiteren Projektverlauf erarbeitet werden.

Intelligente Einsatzplanung:

Derzeit läuft in der Planungsphase des Servers nach dem Auslöser `STATUS: FULL` lediglich ein 10-Sekunden-Timer ab, bevor der Fahrbefehl gesendet wird. Dieser Timer dient als Platzhalter für eine zukünftige kalenderbasierte Einsatzplanung. In einer Weiterentwicklung könnte der Server einen Müllkalender implementieren, der die Abholtermine des zuständigen Entsorgungsunternehmens berücksichtigt. Die Tonne würde dann nicht unmittelbar bei Erreichen des Füllstands losfahren, sondern den Zeitpunkt der nächsten planmäßigen Abholung abwarten und sich rechtzeitig vorher an der Straße positionieren. Die Server-Zustandsmaschine müsste hierfür um einen zeitgesteuerten Übergang von `MONITORING` nach `PLANING` ergänzt werden, der unabhängig vom Füllstand durch den Kalender ausgelöst wird.

Ergänzend könnte der Server die historischen Füllstandsdaten über mehrere Wochen aufzeichnen und daraus Muster ableiten. Etwa, dass ein Haushalt regelmäßig am Wochenende mehr Abfall produziert. Auf Basis solcher Prognosen ließe sich die Fahrt zur Straße vorausschauend planen, noch bevor der Schwellwert tatsächlich erreicht wird. In Kombination mit dem Müllkalender könnte die Tonne sich so optimal positionieren.

Smarte Müllwirtschaft:

Über den einzelnen Haushalt hinaus eröffnet das System Perspektiven für eine grundlegend neue Organisation der Abfallentsorgung. Anstatt Müllfahrzeuge nach einem festen Kalender durch alle Straßen eines Gebietes zu schicken, könnten vernetzte Tonnen ihren Füllstand eigenständig an eine zentrale Plattform melden und einen Müllwagen beim Entsorgungsunternehmen anfordern, sobald die Leerung tatsächlich erforderlich ist. Dieses bedarfsgesteuerte Modell ist auch im Verbund mehrerer Tonnen denkbar: Eine Nachbarschaft meldet ihre aggregierten Füllstände, und der Dienstleister optimiert seine Routen auf Basis realer Bedarfsdaten statt starrer Fahrpläne. Die daraus resultierende Reduzierung von Leerfahrten würde sowohl die Betriebskosten als auch die CO₂-Emissionen der Müllabfuhr senken.

Die vom Server ohnehin erfassten Daten – Füllstände, Leerungszeitpunkte, Einsatzhäufigkeit – könnten über eine Schnittstelle an kommunale Abfallwirtschaftssysteme exportiert werden. Kommunen erhielten so eine datenbasierte Grundlage für die Dimensionierung von Sammelrouten, die Platzierung öffentlicher Müllbehälter oder die Gebührenkalkulation auf Basis tatsächlicher Abfallmengen statt pauschaler Haushaltsgrößen.

Mehrtonnenbetrieb und Benutzeroberfläche:

Die Serversoftware ist bereits multithreading-fähig und kann mehrere Client-Verbindungen gleichzeitig verwalten. Diese Architektur bildet eine Grundlage für den parallelen Betrieb mehrerer Tonnen. Die Missionslogik in `MissionControl` ist derzeit jedoch als Singleton implementiert. Für einen echten Mehrtonnenbetrieb müsste pro Client eine eigene `MissionControl`-Instanz erzeugt und den jeweiligen Threads zugeordnet werden.

Die Benutzeroberfläche des Servers beschränkt sich aktuell auf die Terminal-Ausgabe. Ein Web-Interface mit Echtzeit-Statusanzeige und Kartendarstellung würde den Bedienkomfort insbesondere im Mehrtonnenbetrieb erheblich steigern. Die bestehende Socket-Architektur ließe sich hierfür um ein WebSocket-basiertes Frontend ergänzen.

Schließlich könnte der Kalibrierungsprozess vereinfacht werden. Derzeit erfordert die Anpassung der Regelparameter (`KP`, `KD`, `BASE_SPEED`) und der Motor-Kompensationswerte manuelles Editieren der Datei `main.py`. Ein automatisierter Kalibrierungsmodus, der bei der Erstinbetriebnahme die Motorcharakteristiken vermisst und die Parameter selbsttätig anpasst, würde die Übergabe an Folgegruppen erheblich erleichtern.

Quellenverzeichnis

Amazon.com: Tank Chassis Raupen Raupen Roboter Plattform mit 12V Motor für für T101- / TP101;
https://www.amazon.de/dp/B0B9BC7PR5?psc=1&ref=ppx_pop_dt_b_product_details.

eu.robotshop.com: 12 V 100 U/min 166 oz-in Gleichstrom-Bürstenmotor; <https://eu.robotshop.com/de/products/12-v-100-u-min-166-oz-in-gleichstrom-burstenmotor>.

Otto.de: Mülleimer Abfalltonne 50L Recycling Mülltonne mit Klappdeckel; <https://www.otto.de/p/centi-muelleimer-abfalltonne-50l-recycling-muelltonne-mit-klappdeckel-S0MBN0GX/?variationId=S0MBN0GXKB07#ech=29363950>.

RuleTheBots.com: Linienfolger mit einem PD-Regler; <https://spacehal.github.io/docs/robotik/lineFollowerPD>.